

UNDERGRADUATE THESIS REPORT

An Application of Integrated Mixed-Integer Linear Programming for Aircraft Maintenance Task Packaging Optimization

Submitted as a fulfilment for undergraduate thesis project for
AE4099 UNDERGRADUATE THESIS II

by:

I Kadek Surya Geryandika
13618009

Supervised by:

Disetujui sebagai Draft Sidang.
Bandung, 9 Desember 2025



Dr. Edy Suwondo

Dr. Ir. Edy Suwondo



AEROSPACE ENGINEERING UNDERGRADUATE PROGRAM
FACULTY OF MECHANICAL AND AEROSPACE ENGINEERING
BANDUNG INSTITUTE OF TECHNOLOGY

2025

APPROVAL PAGE

An Application of Integrated Mixed-Integer Linear Programming for Aircraft Maintenance Task Packaging Optimization

By:

Student Name: I Kadek Surya Geryandika

SID: 13618009

This Undergraduate Thesis Report has been checked and approved by:

Date:

Signature:

(Dr. Ir. Edy Suwondo)

Abstract

Fluctuations in aircraft utilization—such as those observed during COVID-like disruptions—introduce significant variability in maintenance demand, causing calendar-limited tasks to dominate execution and reducing the effectiveness of pre-defined maintenance packages. Under such conditions, manual packaging practices and traditional heuristic approaches struggle to adapt, often leading to inefficient task aggregation, increased maintenance event frequency, and higher operational costs. This research addresses these limitations by formulating aircraft maintenance task packaging as an optimization problem that explicitly captures the inherent characteristics of maintenance task packaging and associated operational constraints.

A high-fidelity Mixed-Integer Linear Programming (MILP) model is developed and implemented through a custom Python-based computational program. The model minimizes total maintenance cost while ensuring strict compliance with airworthiness and operational requirements. Using real Maintenance Planning Document (MPD) data from A320-type aircraft as a case study, the model is evaluated under multiple utilization scenarios. The results demonstrate that the MILP approach consistently generates auditable task-to-package assignment matrices and achieves a reduction in average maintenance cost per flight hour compared to a heuristic model. Moreover, the model exhibits stable and interpretable behavior under utilization variations and aligns with real-world maintenance logic, confirming its suitability as a reliable optimal reference framework for analyzing and supporting maintenance task packaging decisions under changing operational conditions.

Keywords: Maintenance Task Packaging, MILP, Optimization, Utilization Disruption, Airworthiness

Abstrak

Fluktuasi tingkat utilisasi pesawat—seperti yang terjadi pada gangguan operasional berskala besar seperti pandemi COVID—menyebabkan variasi yang signifikan dalam kebutuhan perawatan, sehingga tugas-tugas dengan batas kalender menjadi dominan dan menurunkan efektivitas paket perawatan yang telah ditetapkan sebelumnya. Dalam kondisi tersebut, praktik pengemasan tugas secara manual maupun pendekatan heuristik tradisional menjadi sulit beradaptasi, yang sering kali menghasilkan pengelompokan tugas yang tidak efisien, peningkatan frekuensi kejadian perawatan, serta kenaikan biaya operasional. Penelitian ini bertujuan untuk mengatasi keterbatasan tersebut dengan memformulasikan permasalahan *maintenance task packaging* sebagai suatu masalah optimasi yang secara eksplisit merepresentasikan karakteristik dasar pengemasan tugas perawatan beserta kendala operasional yang terkait.

Sebuah model *Mixed-Integer Linear Programming* (MILP) dengan tingkat fidelitas tinggi dikembangkan dan diimplementasikan melalui program komputasi berbasis Python. Model ini bertujuan meminimalkan total biaya perawatan dengan tetap memastikan kepatuhan penuh terhadap persyaratan kelaikudaraan dan operasional. Dengan menggunakan data *Maintenance Planning Document* (MPD) dari pesawat tipe A320 sebagai studi kasus, model dievaluasi pada beberapa skenario utilisasi. Hasil penelitian menunjukkan bahwa pendekatan MILP secara konsisten menghasilkan matriks penugasan tugas-ke-paket yang dapat diaudit serta mampu menurunkan biaya perawatan rata-rata per jam terbang dibandingkan dengan model heuristik. Selain itu, model menunjukkan perilaku yang stabil dan mudah diinterpretasikan terhadap variasi utilisasi, serta selaras dengan logika perawatan di dunia nyata. Hal ini menegaskan bahwa model yang dikembangkan layak digunakan sebagai kerangka acuan optimal yang andal untuk menganalisis dan mendukung pengambilan keputusan dalam *maintenance task packaging* di bawah kondisi operasional yang berubah.

Kata kunci: Pengemasan Tugas Perawatan, MILP, Optimasi, Gangguan Utilisasi, Kelaikudaraan

Acknowledgment

The completion of this thesis and the research contained within was made possible through the immense support, guidance, and contributions of several individuals and institutions. I wish to express my heartfelt gratitude to the following:

First and foremost, I extend my highest gratitude to my supervisor, **Dr. Ir. Edy Suwondo**. Your unwavering guidance, profound knowledge, patience and the valuable opportunities you provided were instrumental in shaping this research from a nascent idea into a comprehensive study within the field of engineering optimization. Your insights pushed this work to achieve a level of rigor I could not have reached alone.

I am profoundly grateful for the steadfast love and unconditional support of **my family**, especially **my parents**. Your encouragement, understanding, and love provided me with the foundation and well-being necessary to navigate the challenges of this project.

Furthermore, I wish to acknowledge the faculty and staff within the **Faculty of Mechanical and Aerospace Engineering** at **Bandung Institute of Technology** for providing the resources and environment necessary for this study.

Lastly, to the friends, colleagues, and all others who offered support, encouragement, and care for my well-being during this demanding period—your contributions did not go unnoticed.

Thank you all.

I Kadek Surya Geryandika

Bandung, November 2025

Table of Contents

APPROVAL PAGE	i
Abstract	ii
Abstrak	iii
Acknowledgment	iv
Table of Contents	v
Table of Figures	ix
Table of Tables.....	x
Abbreviation.....	xi
Chapter 1	1
Introduction	1
1.1. Background	1
1.2. Problem Statement.....	2
1.3. Research Objectives and Questions	2
1.4. Scope and Limitations	3
1.5. Methodology.....	3
1.6. Report Structure.....	5
Chapter 2	6
Literature Review	6
2.1. Maintenance Task Packaging and Regulatory Context.....	6
2.2. Maintenance Locations and Traditional Maintenance Philosophies.....	7
2.2.1. Block Method	7
2.2.2. Equalized Method	8
2.2.3. Line and Base Maintenance Context.....	8
2.3. Economic Drivers of MRO and Total Cost Modelling	8
2.4. Mathematical Characterization of Maintenance Packaging Problem	10
2.5. Review of Methodologies	11
2.6. Method Selection	15
Chapter 3	16
Model Formulation and MILP Implementation	16

3.1.	Problem Detail, Formulation Model and Experimental Assumptions	16
3.1.1.	Problem Details and Model Formulation Overview.....	17
3.1.2.	Assumption and Simplifications	18
3.2.	Objective Function Formulation	19
3.2.1.	Parameters and Variables	19
3.2.2.	Labor Cost (<i>CLabor</i>) and Material Cost (<i>CMaterial</i>).....	21
3.2.3.	Overhead Cost (<i>COverhead</i>).....	21
3.2.4.	Downtime Cost (<i>CDownTime</i>).....	22
3.2.5.	Penalty Cost (<i>CPenalty</i>).....	23
3.2.6.	Line Cost (<i>CLine</i>)	23
3.2.7.	Interaction and Model Behavior.....	24
3.3.	Output Design Rationale and Method Suitability	27
3.3.1.	Discrete vs. Continuous Results.....	27
3.3.2.	A- and C-interval selection.....	27
3.3.3.	Traceability and Auditability	28
3.3.4.	Correlation with Model Design.....	28
3.4.	Constraints for Design System (MILP-based).....	29
3.4.1.	Policy Selection	29
3.4.2.	Bin Classification	29
3.4.3.	Overhead Trigger Linearization	30
3.4.4.	Bin Compatibility Constraints.....	30
3.4.5.	Task Assignment to Opened Bins.....	30
3.4.6.	Packaging and Line Decisions	30
3.4.7.	Interval Compliance Constraints (Due-Date Rule).....	31
3.4.8.	Bin Workload Aggregation.....	31
3.4.9.	Line Eligibility Restriction.....	31
3.5.	Data Acquisition and Preprocessing Methodology	33
3.6.	Heuristic Model	35
3.7.	System Architecture and User Interface Procedure.....	36
3.7.1.	System Architecture Overview	36
3.7.2.	Step-by-Step User Procedure	38
Chapter 4		42

Result and Analysis	42
4.1. System Implementation and Experimental Setup	42
4.1.1. Software Architecture and Solver Configuration	42
4.1.2. Case Study Inputs and Data Preparation Summary	43
4.2. Auditable Outputs of the Optimization Model.....	43
4.3. Comparative Results and Cost Analysis	46
4.3.1. Comparison Framework and Evaluation Scope.....	46
4.3.2. Interval Selection and Structural Difference	47
4.3.3. Observable Cost Component.....	48
4.3.4. Interpretation of Penalty-Based Component	50
4.3.5. Observable Cost Component.....	50
4.4. Downtime Behaviour and Event-level Analysis.....	50
4.4.1. Downtime Distribution Across the Planning Horizon	51
4.4.2. Relationship between Workload Aggregation and Downtime	52
4.4.3. Stability and Operational Realism	53
4.4.4. Analysis and Discussion for Downtime.....	53
4.5. Utilization Scenario Analysis.....	54
4.5.1. Influence of Utilization on Cost and Downtime.....	54
4.5.2. Optimal Intervals Adjustment for Maintenance Task Packaging.....	56
4.5.3. Interpretation of A-Check Stability and C-Check Sparsity	56
4.5.4. Model Validation Under Utilization Variability.....	57
Chapter 5	58
Conclusion	58
5.1. Conclusion.....	58
5.2. Recommendations	60
References.....	62
Appendix A	67
Detail of Case Study and Raw Data	67
A.1. MPD Raw Data	67
A.2. Full Data Input (Optimizer Input).....	68
A.3. Parameter Data (JSON file).....	80
Appendix B	81

Key Code Snippets and Codebase Link.....	81
B.1. Preprocessing Module	82
B.2. Baseline Heuristic Model	84
B.3. MILP Optimizer	85
B.4. Output Design and Visualization	88
Appendix C	92
Detailed Output Visualization.....	92
C.1. Package Summary Data Result	92
C.2. Event Summary Timeline	92
C.3. Task to Package Matrix	94
C.4. Utilization Scenario Analysis Result	101

Table of Figures

Figure 1.1 Overall Research Methodology Flowchart.....	4
Figure 3.1 Maintenance Planning Document (MPD) of PT.XYZ	17
Figure 3.2 Conceptual Logic Flow of the Optimization Model.....	25
Figure 3.3 Design Output for Optimization Model	28
Figure 3.4 MILP Implementation on Design Decision-Support System.....	32
Figure 3.5 Conceptual Logic Flow of Data Acquisition Process.....	35
Figure 3.6 Engineering Data-to-Decision Pipeline Architecture	37
Figure 3.7 Graphical User Interface of Costum Built Decision-Support System.....	39
Figure 3.8 Input Data Process to System.....	39
Figure 3.9 Run Preprocessing Module.....	40
Figure 3.10 Run Optimizer Modules	41
Figure 4.1 Comparison of observable maintenance cost components for baseline execution, heuristic method, and MILP optimization using real A320 case study data. Penalty-based cost terms are excluded. A logarithmic scale is used to improve visual clarity across cost components of different magnitudes.....	48
Figure 4.2 Average maintenance cost per flight hour for baseline, heuristic, and MILP optimization approaches under identical utilization conditions.....	49
Figure 4.3 Distribution of Downtime Hours Across Planning Horizon (MILP Result)	51
Figure 4.4 Package-Interaction and Workload Distribution	52
Figure 4.5 Effect of Aircraft Utilization on Average Cost and Maintenance Downtime.....	55

Table of Tables

Table 2.1 Existing Studies on Methodologies Solution	12
Table 3.1 Symbol and Parameters for MILP-based Model Formulation.....	20
Table 4.1 Case Study Configuration	43
Table 4.2 Representation of Task-to-Package Assignment.....	44
Table 4.3 Detail Cost Components from Comparative result.....	47
Table 4.4 Summary of MILP Results Across Selected Utilization Scenarios	56

Abbreviation

List of Abbreviations

Terms	Acronym
AD	Airworthiness Directives
ALS	Airworthiness Limitations Section
API	Application Programming Interface
CAMP	Continuous Airworthiness Maintenance Program
DMC	Direct Maintenance Cost
EASA	European Union Aviation Safety Agency
FAA	Federal Aviation Administration
FMEA	Failure Mode and Effect Analysis
GUI	Graphical User Interface
IMC	Indirect Maintenance Cost
LP	Linear Programming
MH	Man-hours
MILP	Mixed-Integer Linear Programming
MPD	Maintenance Planning Document
MRO	Maintenance, Repair, and Overhaul
MRB	Maintenance Review Board
MTP	Maintenance Task Packaging
NP	Non-deterministic polynomial time
OEM	Original Equipment Manufacture
RCM	Reliability-Centered Maintenance
SB	Service Bulletins

Chapter 1

Introduction

1.1. Background

Ensuring the continued airworthiness and safety of commercial aircraft is the core objective of aircraft maintenance engineering, carried out under strict regulatory oversight from authorities such as EASA and the FAA [1]–[4]. Operators are required to establish and comply with an approved Maintenance Programme—part of the Continuing Airworthiness framework—which ensures that every aircraft remains airworthy throughout its service life [1]–[3]. The technical content of this programme originates from the Maintenance Planning Document (MPD), which specifies regulator-approved tasks, intervals, and conditions derived from reliability engineering and structural integrity requirements [4], [5].

In operational settings, these MPD tasks are executed through structured maintenance events such as Line maintenance, A-checks, and C-checks [2], [5], [6]. The process of maintenance task packaging refers to grouping individual MPD tasks into these events while respecting interval limits, manpower requirements, and facility constraints. Effective packaging is essential because it directly influences maintenance cost, downtime, manpower distribution, and the operational feasibility of the Continuous Airworthiness Maintenance Program (CAMP).

A major engineering challenge arises when aircraft utilization (flight hours per day) changes significantly. During the global COVID-19 disruption, for example, utilization dropped sharply across the industry, altering the effective due times of MPD tasks. Lower utilization often accelerates calendar-based requirements, resulting in inefficient packaging, unnecessary event activations, and increased maintenance cost. Since such utilization disruptions can recur, maintenance engineering requires analytical tools capable of using historical utilization profiles to anticipate packaging impacts and evaluate efficient grouping strategies under different utilization conditions.

Traditional packaging practices rely on fixed intervals and manual judgment. These approaches do not systematically evaluate alternative packaging structures or quantify the cost implications of changing utilization levels. To address this limitation, this research develops an application of Mixed-Integer Linear Programming (MILP) model specifically designed to optimize maintenance task packaging for an A320-type aircraft, using realistic cost components and utilization-dependent interval behavior derived from MPD requirements. The model

generates traceable and auditable outputs suitable for engineering validation and alignment with the Continuous Airworthiness Maintenance Program (CAMP).

1.2. Problem Statement

Maintenance task packaging assigns MPD tasks into cluster or group such as: Line, A-check, and C-check events within the Continuous Airworthiness Maintenance Program (CAMP). During major operational disruptions such as the COVID-19 pandemic, aircraft utilization drops sharply, causing many MPD tasks to reach their calendar limits earlier than their flight-hour limits [7]. This disrupts the normal alignment of task intervals and leads to inefficient packaging, more event activations, increased downtime, and unnecessary maintenance cost. Existing manual and rule-based practices are not designed to reorganize packages under such utilization changes, and prior work such as Marvella [8] relied on brute-force enumeration that cannot guarantee optimal results or incorporate realistic operational constraints. The broader literature also provides few MILP formulations that operate at MPD task-level fidelity. Therefore, the core problem addressed in this research is the development of a practical, regulator-consistent MILP-based application model that can generate efficient maintenance task packaging to minimize total maintenance costs under changing utilization conditions, using historical utilization forecast data as a reference for responding to future disruption scenarios.

1.3. Research Objectives and Questions

The objective of this research is to develop a Mixed-Integer Linear Programming (MILP) model for maintenance task packaging of Continuous Airworthiness Maintenance Program (CAMP) in order to minimize maintenance costs while guaranteeing compliance with airworthiness requirements, particularly when aircraft utilization varies significantly as seen during COVID-like disruptions. The model is designed to use MPD-level task data together with historical utilization forecast values to evaluate feasible interval arrangements and produce packaging that minimizes total maintenance cost while satisfying airworthiness constraints. To achieve this, the research pursues three specific objectives:

1. **To formulate a MILP-based mathematical model** that represents the nature of maintenance task packaging and maintenance operational problem.
2. **To develop a computational program** that implements the MILP formulation and generates traceable outputs, including task-to-package assignment, event intervals, and cost components.

3. **To validate the model using real data from case study of A320-type aircraft,** demonstrating how the optimization results respond to variations in utilization inputs and confirming that the model can serve as a reliable reference for handling utilization disruptions in the future.

1.4. Scope and Limitations

This research is limited to the development and evaluation of application Mixed-Integer Linear Programming (MILP) model for maintenance task packaging on Airbus A320-type aircraft. The study focuses strictly on the engineering problem of assigning MPD-defined maintenance tasks into Line, A-check, and C-check maintenance events while complying with regulatory intervals, manpower assumptions, and environment eligibility. The scope does not extend to fleet-level planning, routing, hangar slot allocation, or long-term maintenance scheduling decisions.

The model operates under a deterministic setting using MPD task characteristics, cost parameters, and historical utilization data derived from both normal and COVID-like operational periods. These utilization values are used solely to generate interval conversions and scenario variations; they are not used for forecasting. Several simplifications were made to ensure tractability. Manpower availability is represented using aggregated capacity parameters rather than skill-based resource models. Downtime is approximated using a validated linear formula rather than a full dynamic execution simulation. Uncertainty, operational disruptions, and day-to-day planning variations are not represented. The output reflects an optimal packaging recommendation for the given inputs and assumptions, and is intended as a technical baseline data reference on tasks packaging solutions for disruption scenarios.

1.5. Methodology

This research employs an outcome-oriented design philosophy using an optimization-driven case study approach. The focus is on producing implementable and interpretable maintenance task packaging solutions that reflect real engineering and operational constraints, rather than on theoretical model development alone. **Figure 1.1** presents a high-level overview of the research methodology.

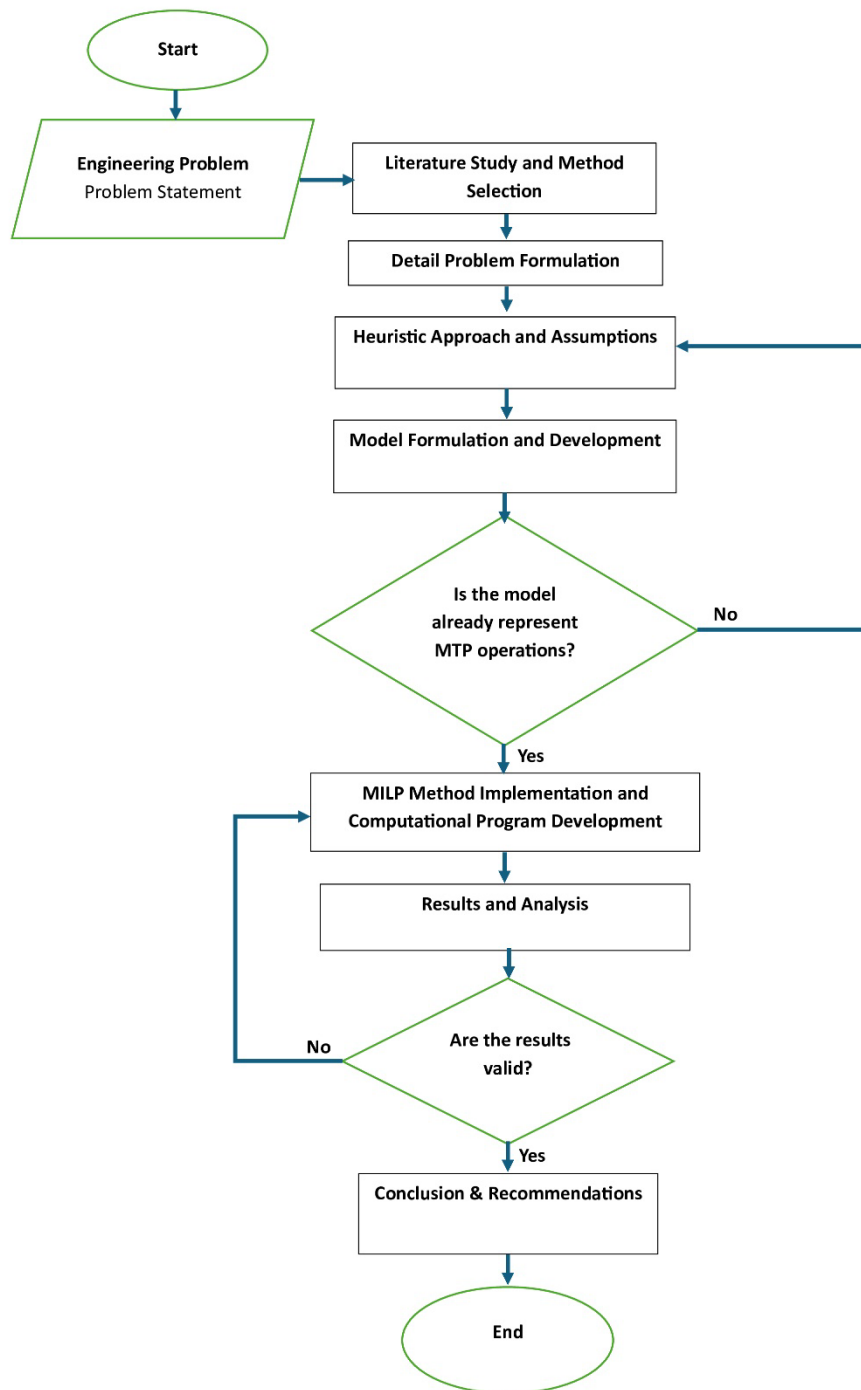


Figure 1.1 Overall Research Methodology Flowchart

The thesis is structured as follows. **Chapter 1** introduces the background of maintenance task packaging, the impact of utilization disruptions, the problem statement, and the objective of developing a computational MILP-based model. **Chapter 2** provides the engineering and methodological foundation for the study, including MPD-based maintenance task

characteristics, airworthiness requirements, cost elements relevant to maintenance packaging, and the reasoning behind selecting MILP as the optimization method. This chapter establishes the basis for the mathematical formulation.

Chapter 3 describes how the MILP method is applied to the maintenance task packaging problem. It explains the assumptions, decision variables, constraints, cost function, and the overall model structure, followed by the custom software pipeline used for implementation. **Chapter 4** presents the optimization results from MILP application model, including comparison with the baseline configuration, heuristic model, and analysis under different utilization forecast values. The chapter also interprets how the results reflect real operational impacts on packaging efficiency.

Finally, **Chapter 5** concludes the study by summarizing the key technical contributions and providing recommendations that remain within the scope of maintenance task packaging.

1.6. Report Structure

This report is organized into five chapters, each designed to clearly guide the reader from the problem motivation to the development, validation, and interpretation of the proposed optimization model.

Chapter 1 – Describes the operational motivation behind the research, particularly the impact of utilization disruption on maintenance task packaging, and outlines the research objectives, scope, and methodological approach.

Chapter 2 – Provides the engineering and regulatory foundations relevant to maintenance task packaging, including MPD task characteristics, interval rules, maintenance event structure, and cost components. This chapter also justifies the selection of Mixed-Integer Linear Programming (MILP) as the optimization method used in this research.

Chapter 3 – Details the formulation of the MILP model, including decision variables, constraints, objective function, and supporting procedures. It also describes the software workflow used to implement the model and how the computational program processes input data to generate optimized task packaging outputs.

Chapter 4 – Presents the case-study data, optimization results, comparative analysis against the baseline configuration, and sensitivity to different utilization profiles. This chapter interprets the outcomes and explains their relevance to maintenance task packaging efficiency.

Chapter 5 – Summarizes the major findings of the research and provides recommendations for further development of the model, particularly in expanding its applicability while staying consistent with the scope of this thesis.

Chapter 2

Literature Review

This chapter presents the foundational concepts, regulatory requirements, and methodological developments relevant to the aircraft maintenance task packaging problem. It synthesizes the engineering principles that govern the Maintenance Planning Document (MPD), the economic factors influencing maintenance decisions, and the combinatorial nature of packaging tasks under interval and feasibility constraints. The chapter also reviews established maintenance philosophies, cost structures, and the classification of the packaging problem within operations research. Finally, a structured assessment of existing solution methodologies is provided to justify the selection of a Mixed-Integer Linear Programming (MILP) framework as the most suitable approach for this research.

2.1. Maintenance Task Packaging and Regulatory Context

Maintenance task packaging is a core process within aircraft maintenance engineering in which individual tasks defined in the Maintenance Planning Document (MPD) are grouped into Line, A-check, and C-check events in accordance with the operator's approved Continuous Airworthiness Maintenance Program (CAMP). The MPD specifies each task's scope, interval, and airworthiness limits, and these limits form binding regulatory requirements under authorities such as EASA and FAA, which mandate that operators perform maintenance in accordance with an approved programme to ensure continued airworthiness throughout the aircraft's service life [1]–[3]. Although MPD task content and intervals are fixed through certification processes—such as damage-tolerance and fatigue evaluation requirements set out in FAA §25.571 [4]—the grouping, or packaging, of these tasks into maintenance events remains an engineering decision. This grouping directly affects maintenance cost, downtime, manpower demand, and the frequency of maintenance event activations.

The primary objective of the regulatory context is compliance; the optimization problem addressed in this report is fundamentally constrained by these non-negotiable safety and regulatory mandates. Effective packaging therefore requires aligning tasks with compatible intervals while preventing unnecessary checks, maintaining compliance with interval limits, and ensuring clear traceability for regulatory oversight, as emphasized in FAA AC 120-16G regarding operator maintenance programmes [9]. These regulatory and engineering foundations form the basis of this research and support the development of a computational

optimization model designed to generate efficient and compliant task packaging solutions under varying aircraft utilization conditions.

2.2. Maintenance Locations and Traditional Maintenance Philosophies

The execution of aircraft maintenance is organized into structured maintenance events—Line maintenance, A-checks, and C-checks—into which individual MPD tasks must be assigned as part of the Continuous Airworthiness Maintenance Program (CAMP). Although MPD tasks come with regulator-approved intervals and task descriptions, operators retain flexibility in determining how these tasks are grouped into appropriate events, provided all interval limits, access needs, and compliance requirements are met. This grouping process, known as maintenance task packaging, is central to ensuring efficient and airworthy operations [2], [5], [6].

Line maintenance represents the lightest category of work, typically carried out on the apron with minimal equipment. A-checks require longer downtime and structured work packages but can often still be performed outside the hangar. C-checks, by contrast, are conducted in a base-maintenance environment and involve deeper inspections and broader aircraft access. The characteristics of each task—such as required man-hours, access zone, or environmental needs—implicitly determine which type of maintenance event it can be assigned to. Although the present research does not explicitly optimize maintenance locations, such feasibility considerations remain embedded in the packaging logic used throughout this work.

Historically, two main philosophies have shaped how operators package tasks within these maintenance events:

2.2.1. Block Method

The **block method**, also known as the letter check system, is a traditional maintenance philosophy where tasks are grouped into predefined checks based on fixed multiples of operating time [5], [6]. The block method (or letter-check system) groups tasks into fixed maintenance blocks based on predefined intervals. For example, an operator might assign all tasks with intervals ≤ 500 flight hours (FH) into an A-check, while tasks with intervals $\leq 4,000$ FH fall into a C-check. This approach is straightforward, easy to document, and highly transparent for regulatory oversight. However, its rigidity can cause inefficiencies: tasks may be carried out earlier than necessary, especially when several interval-based tasks converge into the same fixed block. As utilization patterns change over time, this method becomes increasingly prone to early execution and unnecessary maintenance effort.

2.2.2. Equalized Method

In contrast, the **equalized method** seeks to distribute the total maintenance workload more evenly across a series of smaller, more frequent visits [5], [6]. The equalized method distributes tasks more evenly across a sequence of smaller visits rather than concentrating them into large blocks. By phasing tasks across the cycle, operators reduce workload peaks and avoid large downtime events. This method generally results in more balanced manpower usage and improved aircraft availability. However, balancing hundreds of interval-driven tasks in a consistent and compliant manner is difficult without computational support, and manual approaches cannot guarantee an efficient solution.

2.2.3. Line and Base Maintenance Context

Task packaging must also respect the organizational and physical boundaries between two primary types of maintenance locations, which influence resource availability and cost:

- **Line maintenance:** This covers frequent, low-duration tasks (e.g., transit checks, A-checks, minor repairs) typically performed between flights or overnight at the airport gate or apron. It emphasizes rapid turnaround with minimal downtime and is limited by available ground support equipment. EASA regulations define it as maintenance that can be performed outside of a hangar [2], [5], [12].
- **Base maintenance:** This involves more extensive work, such as scheduled heavy checks (C-checks, D-checks), major structural repairs, and complete overhauls that require the aircraft to be removed from service for a longer period (days to months). This work is conducted in a hangar environment equipped with specialized tools and facilities [2], [5].

The packaging problem must ensure that tasks assigned to each package are compatible with available maintenance resources and locations. While this constraint is usually treated operationally rather than mathematically, it influences packaging feasibility—certain access actions or structural inspections can only be performed under base-maintenance conditions.

These environments are not optimization variables in this research, but they form the practical boundaries within which all packaged tasks must remain feasible. This ensures that the resulting packages generated later in the MILP formulation (Chapter 3) not only satisfy MPD intervals but also reflect real operational executability.

2.3. Economic Drivers of MRO and Total Cost Modelling

Maintenance operations represent a major portion of an airline's operating expenses, and optimization of these activities is therefore essential for maintaining competitiveness [12], [13]. In real operations, the total cost of packaging Line, A-check, and C-check tasks is influenced

by several cost components that must be considered together to represent the actual economic impact of each packaging strategy. An effective optimization model must capture these components comprehensively rather than focusing solely on labor cost, as many simplified formulations in the literature do [14], [15]. This section outlines the cost elements incorporated into this research model and provides the foundation for the mathematical formulation presented in Chapter 3.

1. **Direct Maintenance Cost (DMC)**

These involve fundamental expenditures on labor (technician hours, labor rates) and materials (spare parts, consumable). Literature consistently models these as primary cost drivers that fluctuate with specific task requirements [14], [15].

2. **Indirect Cost and Overhead:**

Indirect costs include hangar fees, apron operations, access equipment setup, tooling requirements, quality control, administration, and aircraft towing for positioning. Industry studies emphasize that these overheads significantly influence maintenance economics and must be included to reflect real operational costs [12], [13], [16].

In this research, event-level overhead is represented as fixed activation cost for each maintenance event (A-check and C-check), with hangar events incurring higher overhead than apron activities. The total number of events generated by a packaging strategy therefore has a direct effect on total maintenance cost.

3. **Downtime and Opportunity Cost:**

Downtime represents the period during which the aircraft cannot generate revenue because it is undergoing maintenance. Research highlights that minimizing the downtime hours is crucial because it is substantial cost driver in itself [14]–[16].

4. **Penalty Cost (Behavioural):**

In optimization models, penalty terms are commonly used to guide task assignment decisions so that the resulting packaging remains compliant and efficient [15]–[17], [24]. Two key behaviors are penalized in this research:

- **Executing tasks significantly earlier than their MPD interval**, which reduces component useful life and causes unnecessary maintenance cost.
- **Excessive shifting of tasks out of packages into individual line executions**, which may increase line workload even if it reduces event overhead.

These penalty functions help the optimization model identify the most cost-effective packaging arrangement that meets regulatory constraints while avoiding unrealistic or operationally undesirable solutions.

5. Task Left Unpackaged (Line Execution)

Some tasks are better left outside A- or C-check packages and executed individually based on their own interval. These tasks:

- avoid hangar overhead,
- follow their MPD interval more closely,
- and reduce unnecessary early execution.

However, performing too many tasks individually increases routine workload, so the model assigns a modest cost to line-executed tasks. This cost is **not a penalty**, but a normal execution cost to balance packaging decisions. The optimization model therefore determines, for each task, whether packaging or individual execution results in the lower total cost.

While the literature acknowledges these varied costs, many existing optimization models tend to simplify the total cost function to manage computational complexity, often focusing only on direct maintenance cost, indirect cost or downtime cost model [17], [18]. This study addresses this limitation by synthesizing these factors into a comprehensive total cost function, as detailed in Chapter 3.

2.4. Mathematical Characterization of Maintenance Packaging Problem

Maintenance task packaging is a **constraint-driven combinatorial optimisation problem** defined by the allowable intervals specified in the MPD [10]. Each MPD task has an indivisible interval limit, and assigning these tasks into Line, A-check, or C-check events requires ensuring that no task exceeds its due threshold while maintaining operational feasibility. Because events occur at discrete multiples of the chosen A and C intervals, every task can only be placed into a small set of valid maintenance events, which creates a tightly constrained decision space.

The interaction between task intervals, event spacing, and resource limitations produces strong dependencies:

- Shorter intervals pull tasks forward more frequently, increasing cost.
- Longer intervals accumulate excessive workloads at fewer events.
- Adjusting intervals or assignments for one group of tasks often affects the feasibility of many others.

This interdependency is the core reason why maintenance task packaging aligns with **bin-packing-type formulations** in operations research [17]–[19]. Each maintenance event functions as a “container” that can receive only tasks compatible with its timing and technical

requirements. As the number of MPD tasks grows, the packaging problem expands rapidly because of overlapping intervals, varying task durations, and cost interactions—matching well-established results that such allocation problems are **NP-hard** in general combinatorial optimisation literature [17], [20], [21].

From an engineering standpoint, packaging is essential because it consolidates tasks that share similar intervals or access requirements, thereby reducing repeated disassembly, minimizing aircraft downtime, and optimizing manpower and resource utilization. However, unlike generic bin-packing problems, MPD tasks cannot be partially executed, split across events, or extended beyond their due limits. Each task must be assigned once, or executed individually when packaging is not cost-effective. Furthermore, some tasks are restricted to base-maintenance conditions while others may be carried out on the apron, introducing additional feasibility boundaries.

These characteristics require a modelling framework that can explicitly capture:

- the nature of maintenance task packaging,
- interval feasibility for each individual MPD task,
- compatibility with Line or Base environments,
- cost interactions across maintenance events, and
- the trade-off between packaging tasks or executing them individually based on economic benefit.

Given these conditions, the problem cannot be reliably solved using manual or heuristic-only methods, as they do not effectively evaluate all valid combinations to guarantee that the final configuration is optimal. A **Mixed-Integer Linear Programming (MILP)** formulation is therefore justified because it enforces compliance with MPD rules, represents the bin-packing-like structure mathematically, and systematically evaluates feasible solutions to identify the cost-optimal packaging arrangement. This formal characterisation directly supports the objective of this research: to develop an application of MILP-based model to minimize total maintenance costs for maintenance task packaging.

2.5. Review of Methodologies

The literature offers a diverse range of methodologies for addressing the aircraft maintenance task packaging problem, reflecting its complexity and the variety of operational contexts in which solutions must be applied. These methods generally fall into three major groups—heuristics and metaheuristics, mixed-integer linear programming (MILP), and

hybrid frameworks—each contributing valuable insights into how maintenance decisions can be modeled and optimized.

- **Heuristics and Metaheuristics**

A substantial portion of research adopts heuristics such as worst-fit decreasing, genetic algorithms, and simulated annealing to construct packaging or scheduling decisions. These methods are attractive because they produce workable solutions quickly and can handle complex, nonlinear interactions in large datasets. Their flexibility makes them particularly suitable for fleet-level planning and long-horizon scenarios. However, they do not provide mathematical guarantees of global optimality, and constraint satisfaction often depends on rule design or parameter tuning. This makes them less reliable for safety-critical, MPD-driven packaging tasks that require strict compliance verification [11], [18], [22].

- **Mixed-Integer Linear Programming (MILP)**

Research also utilizes MILP for maintenance task packaging, particularly where precision and optimality are critical. MILP models excel at handling discrete decisions (binary variables) and complex constraints, guaranteeing an optimal solution if computational limits are respected. The trade-off is often the time required to solve large-scale problems [21], [23].

- **Hybrid Frameworks**

Emerging literature suggests combining methodologies to leverage strengths: using heuristics for high-level fleet management (macro-decisions) and optimization solvers like MILP for tactical, task-level packing problems (micro-decisions) [16], [17].

Research in the broader aircraft maintenance optimization domain varies widely in modeling scope, decision granularity, and the treatment of maintenance costs. Some studies focus on high-level scheduling or facility-level optimization, while others examine detailed task allocation or predictive maintenance decisions. To understand how these methodologies have been applied across different maintenance contexts, Table 2.1 summarizes a selection of studies that illustrate the diversity of modeling approaches as references and review for this research objectives.

Table 2.1 Existing Studies on Methodologies Solution

Reference	Methodology Used	Problem Scope	Cost Formulation	Key Strength	Key Limitation
Ali, Rudinskas & Leonavičiūtė (2025) [21]	Mixed-Integer Linear Programming (MILP)	Small fleet maintenance task allocation (Cessna 172 case study)	Simplified Operational Cost	Explicitly models manpower skill heterogeneity; Rigorous MILP formulation with exact optimization guarantees	Limited to small fleets; MATLAB prototype, focus is on allocation rather than long-horizon packaging
Witteman, Deng & Santos (2021) [17]	Constructive heuristic (bin-packing, worst-fit decreasing)	Multi-year fleet maintenance task allocation	Indirect cost via bin utilization	Computationally efficient and scalable bin-packing formulation	Sacrifices global optimality; heuristic tuning required, Abstracted from regulatory constraints
Filho et al. (2024) [16]	Exact algorithm + Integer Linear Programming (ILP); Optimization-based (MILP/heuristic hybrid)	Task allocation and packing (MSG-3 based requirements)	Penalty-based cost formulation	Incorporates task relationships, preparation tasks, and opportunity costs. (Explicit consideration of feasibility and packing penalties)	Computationally heavy; requires detailed reliability/failure data
Andrade et al. (2021) [22]	Deep Q-learning (Reinforcement Learning)	Long-term fleet check scheduling (A- and C-checks)	Implicit reward-based cost	Adaptive and data-driven scheduling approach; reduces number of checks; robust to disturbances	Requires large training datasets; constraint satisfaction depends on training data and reward design
Pazhooh & Shemirani (2025) [23]	Continuous-Time MILP	Integrated hangar scheduling and layout optimization	Detailed Scheduling Cost	High-fidelity continuous-time modeling; exact optimization feasible	Not MPD-driven; focuses on system- and facility-level scheduling rather than task packaging.

Reference	Methodology Used	Problem Scope	Cost Formulation	Key Strength	Key Limitation
Tseremoglou, Scarpin & Bertolini (2023) [11]	Comparative modeling (MILP, heuristics, metaheuristics)	Condition-based fleet maintenance scheduling	Varies by model	Benchmarks multiple approaches; highlights trade-offs	Review-oriented; not a deployable packaging or scheduling model
Vu et al. (2024) [18]	Genetic Algorithm (GA)	Predictive maintenance optimization in industrial systems	Maintenance-related cost function	Demonstrates GA's adaptability for predictive maintenance; scalable to large datasets; Effective for complex nonlinear search spaces	Conceptual rather than aviation-regulatory specific; requires adaptation to aircraft maintenance context

As shown in Table 2.1, Across the reviewed literature, several consistent insights emerge that help frame the methodological direction of this research.

First, studies vary widely in their modeling focus, ranging from task-level allocation, fleet-level scheduling, hangar layout, to predictive maintenance. This diversity shows that maintenance optimization is a broad field, but it also indicates that only a subset of studies directly address the specific challenge of MPD-driven task packaging.

Second, cost modeling tends to be simplified in many works. Some focus solely on direct labor costs, while others emphasize bin utilization, opportunity cost, or scheduling-related penalties. The lack of comprehensive total cost representation indicates that the economic dimension of packaging—especially for individual aircraft—is not yet fully captured in existing models.

Finally, while some studies explore hybrid or comparative approaches, there remains a gap in models that integrate MPD task structures, operational feasibility, and a detailed economic formulation within a single framework. This gap is most evident at the level of aircraft maintenance task packaging scope focus, where interval alignment, access coordination, and cost accumulation interact directly and require precise modelling.

2.6. Method Selection

The literature demonstrates that while heuristic and metaheuristic approaches offer scalability, they lack the mathematical guarantees required for precise compliance with MPD-defined interval constraints. Conversely, existing MILP approaches often simplify task structures or operational cost elements to maintain tractability, reducing their applicability for detailed engineering-level packaging.

Mixed-Integer Linear Programming (MILP) meets these requirements by enabling discrete assignment decisions while maintaining exact linear constraints. In maintenance task packaging, each task is an indivisible unit with a fixed interval and duration, making it well suited for a binary decision structure. MILP also allows regulatory and engineering limits—such as maximum allowable usage between task executions—to be expressed precisely, which is essential for compliance with CAMP- and MPD-driven requirements. Furthermore, MILP supports explicit cost modeling, allowing direct labor, indirect overhead, downtime cost, and penalty functions to be incorporated in a transparent and interpretable way.

Based on the literature and the specific demands of MPD-based packaging, three core requirements guide the selection of MILP for this research:

1. **Task-level precision** — Each MPD-defined task must be modeled as an indivisible decision unit with fixed intervals and execution durations.
2. **Regulatory fidelity** — Airworthiness and engineering interval constraints must be represented exactly, not through approximations or aggregated structures.
3. **Economic interpretability** — Costs associated with direct labor, overhead, and downtime must be explicitly included to evaluate operational consequences realistically

The MILP framework developed in Chapter 3 is designed to maintain this balance between modeling fidelity and computational feasibility. It integrates engineering constraints, economic drivers, and combinatorial task-placement decisions into a unified formulation capable of producing optimized and regulator-consistent packaging solutions. This high-fidelity baseline establishes the necessary foundation for future extensions, including hybrid or fleet-level methods, while ensuring the core task packaging problem is preserved to maintain a safety-critical engineering context.

Chapter 3

Model Formulation and MILP Implementation

Chapter 3 presents the methodological foundation and formulation of the optimization model developed in this research. Building on the conceptual background and literature insights established in Chapter 2, this chapter translates the aircraft maintenance task packaging problem into a structured, high-fidelity Mixed-Integer Linear Programming (MILP) framework. The chapter begins by detailing the problem context, the engineering assumptions, and the data structures that define how Maintenance Planning Document (MPD) tasks interact with utilization forecasts. It then introduces the objective function and cost components, outlines the full mathematical formulation—including decision variables, constraints, and model logic—and explains how each element supports regulatory compliance and operational feasibility. Complementing the MILP, a heuristic baseline model is also presented to provide a transparent point of comparison. Overall, this chapter establishes the complete technical basis for the optimization workflow used in Chapter 4 to generate, analyze, and validate the model's results.

3.1. Problem Detail, Formulation Model and Experimental Assumptions

To manage the NP-hard nature of the task packaging problem discussed in Section 2.4 while maintaining engineering fidelity, this study restricts its scope to a single aircraft, as defined in Section 1.4. The central goal is to assign indivisible MPD-defined tasks into a sequence of available maintenance packages in a manner that minimizes total cost while strictly respecting engineering intervals and practical feasibility constraints.

To maintain engineering fidelity while keeping the optimization solvable, the research focuses on the packaging of MPD-defined tasks for one aircraft type, following the scope defined in Chapter 1. Each MPD task is treated as an indivisible unit with fixed attributes such as interval type (FH, FC, calendar), access requirement, duration, and classification. The core goal is to determine whether tasks should be grouped into available maintenance packages or executed individually, while minimizing the total cost and ensuring all interval and feasibility constraints are satisfied.

Because real operational environments involve dynamic interactions—such as shifting manpower levels, non-linear downtime effects, aircraft routing, or evolving opportunity costs—this study applies linearized and static assumptions where necessary. These simplifications follow common practices in maintenance optimization research and allow the

MILP model to remain computationally feasible while still reflecting the essential engineering characteristics of the task packaging problem

3.1.1. Problem Details and Model Formulation Overview

This chapter describes how the optimization model is formulated to respond directly to the maintenance task packaging problem introduced in Chapter 1. As discussed earlier, aircraft utilization can change significantly during global disruptions such as the COVID-19 situation, where flight activity dropped sharply. These irregular utilization patterns disturb the natural maturity progression of tasks in the MPD, causing intervals to cluster unexpectedly and making traditional packaging inefficient. This research addresses this issue by developing a Mixed-Integer Linear Programming (MILP) model that uses MPD raw data and utilization forecast inputs to generate cost-efficient and compliant maintenance packages for A320-family aircraft.

In the proposed workflow, the primary raw inputs are:

1. **MPD raw task data** (task intervals, access requirements, man-hours, task classifications).
2. **Utilization forecast data**, which is used to convert interval limits into a uniform flight-hour domain suitable for optimization.

These datasets undergo preprocessing, normalization, and attribute extraction (as detailed in Section 3.5) to produce the structured parameters required by the MILP model. *Figure 3.1* illustrates the MPD raw data used in this study, from which the relevant engineering information is extracted based on the assumptions and simplifications described in Section 3.1.2.

TASK NUMBER	ACCESS	ZONE	DESCRIPTION	SKILL CODE	TASK CODE	100% INTERVAL	SOURCE	TCI	TPS	REFERENCE	MEN	TASK M.H.	ACCESS M.H.	PREP. M.H.	APPLICABILITY
200001-01-1		571 571	HRIFLIGHTNING PROTECTION-WING	EL	FNC	36 MO	MRB LHRF			243200-210-022	1 1	0.50 0.50			PRE 24343 (57-1086)
200001-02-1	521EB 521GB 522AB 621EB 621GB 622AB	511 520 611 620	CHECK BONDING MEASUREMENT OF CLAMPS WITH A LOOP TESTER FROM FUSELAGE TO RIB 9	EL	FNC	72 MO	MRB LHRF			243200-210-023	1 1 1 1	0.50 0.50 0.50 0.50	0.27 0.27	0.02	ALL
200002-01-1	334AB 334BB 334CB 334DB 334EB 344AB 344BB 344CB 344DB 344EB	334 344	CHECK THAT THE CONNECTOR IS CORRECTLY TIGHTENED BY MEASUREMENT OF THE RESISTANCE WITH A LOOP TESTER: - 4005VC, 4007VC, 4009VC, 4011VC, LH SIDE - 4013VC, 4015VC, 4017VC, 4019VC, LH SIDE - 4006VC, 4008VC, 4010VC, 4012VC, RH SIDE - 4014VC, 4016VC, 4018VC, 4020VC, RH SIDE	EL	GVI	36 MO	MRB LHRF			243200-210-018	1 1	0.15 0.15	0.30 0.30		ALL
200003-01-1	321AL	321	HRIFLIGHTNING PROTECTION-VERTICAL FIN	EL	GVI	72 MO	MRB LHRF			243200-210-019	1	0.15	0.35		ALL
200003-02-1	322BL	322	GENERAL VISUAL INSPECTION OF ELEMENTS FITTED ON THE FOLLOWING HARNESSSES: - 4202VB, 4204VB	EL	GVI	72 MO	MRB LHRF			243200-210-020	1	0.15	0.37		ALL
200003-03-1	324AT	324	HRIFLIGHTNING PROTECTION-VERTICAL FIN	EL	GVI	72 MO	MRB LHRF			243200-210-021	1	0.15	0.18		ALL
200121-01-1	121AF 121AL 121BF 121CF 811 122BF	121 122	GENERAL VISUAL INSPECTION OF ELEMENTS FITTED ON THE FOLLOWING HARNESSSES: - 4202VB, 4204VB	AF	RST	48 MO	MRB			243200-100-004	1 1	0.30 0.30	0.12 0.03		ALL
200121-03-1	121AL 811 812	121	CLEANING OF EWIS INSTALLED IN THE FORWARD AVIONIC COMPARTMENT (ONLY IF CONTAMINATED)(EWIS)	AF	GVI	72 MO	MRB			243200-210-028	1	0.50	0.06		ALL

Figure 3.1 Maintenance Planning Document (MPD) of PT.XYZ

The utilization forecast affects interval conversion, task maturity alignment, and planning horizon distribution. The structure of the model outputs (Section 3.3) also determines which MPD attributes and forecast-derived values are extracted. Together, these processed data form the complete input set for the optimization model. Figure 3.1 presents an excerpt of the MPD used in this case study. From this document, only the engineering-relevant task attributes are extracted through preprocessing, normalization, and classification procedures described in Section 3.5.

A reduced task set of 300 tasks was generated using stratified sampling to preserve the statistical characteristics of the full MPD. Tasks were first grouped by interval class (Line, A, C), maintaining the original class proportions. Within each class, tasks were sampled using manhour quantiles to retain the original workload distribution, including the heavy-tail behavior. Interval ranges were preserved to ensure realistic pull-forward feasibility.

These structured parameters then serve as inputs to the MILP formulation. The interaction between MPD attributes and utilization-derived values determines the timing, feasibility, and cost behaviour of each task in the optimization.

3.1.2. Assumption and Simplifications

To ensure the MILP model remains tractable while still representing engineering constraints accurately, the following assumptions are applied:

1. **Aircraft Focus (A320 Family):**

The study evaluates packaging for one aircraft type as a representative narrow-body platform. Only MPD tasks with defined FH, FC, or calendar intervals are included.

2. **Downtime as a Linear Resource:**

The complex calculation of aircraft downtime hours is linearly modelled as a fixed resources using formula that approach its output result when compare to real-world dynamic execution [15], [16]. Integration of above parameter to the simplified formula will help the simplified formula to better representing the dynamic model shift execution complexity.

3. **Fixed Manpower Representation:**

Labor availability is aggregated into a single capacity parameter (M_{cap}) per shift, combined with a fixed labor cost (C_{Labor}). Detailed mechanic skill differences are not modeled to maintain tractability.

4. **Deterministic**

Data:

All inputs (task durations, interval limits, forecasted utilization, cost values, and capacity parameters) are treated as fixed and known during each optimization run. This ensures that the MILP produces an exact, verifiable solution for each scenario without introducing stochastic uncertainty.

3.2. **Objective Function Formulation**

The primary objective of this research model is to determine the optimal packaging arrangement of maintenance tasks for a single aircraft to minimize the total associated costs within the defined planning horizon, while adhering to all regulatory and operational constraints. As established in Chapter 2, existing literature often simplifies cost structures to manage computational complexity. The objective function derived for this study is a comprehensive synthesis design to capture all major direct and indirect economic drivers, including labor, material, overhead, and aircraft downtime.

The optimization model balances compliance with airworthiness directives against efficient resources utilization, operational management, and the value of maximizing task intervals. Importantly each cost components formula will be adjusted to MILP-based system but still respect to how real-world execution of maintenance tasks are. The total cost function (Z_{total}) is defined as the summation of Direct Maintenance Cost (DMC), Indirect Maintenance Cost (IMC), and economic penalties, as formulated in Equation 3.1 to ensures the optimal packaging assignment is the most cost-efficient while adhering to all constraints.

$$\begin{aligned} \text{Minimize } (Z_{total}) \\ = C_{Labor} + C_{Material} + C_{Overhead} + C_{Downtime} + C_{Penalty} \\ + C_{Line} \end{aligned} \quad 3.1$$

The subsequent sections detail each cost component, its operational relevance, and its implementation within Mixed-Integer Linear Programming (MILP) model.

3.2.1. **Parameters and Variables**

- **Indices and Set**
 - **Aircraft Planning Horizon:** H
 - **Tasks:** I indexes tasks $i \in I$
 - **A-interval candidates:** A indexes discrete A-check candidates $a \in A$
 - **C-interval candidates:** C indexes discrete C-check candidates $c \in C$

- **Candidate bins:** T indexes bin endpoints $t \in T$ where:
$$T = \{t \in \{1, \dots, H\} \mid t \text{ is multiple of some } a \in A \text{ or } c \in C\}$$
- **Allowed assignment domain:** $T_i = \{t \in T \mid t \leq I_i\}$
- **Parameters:**
 - MH_i : Man-hours required for Task i (deterministic value from total manhours component)
 - I_i : Minimum Interval available for Task i (in flight hours, a deterministic minimum value from all interval value of a task after normalization with average value of utilization forecast data to represent maximum dues for when the task must be executed before due date)
 - M_{cap} : Manpower capacity per shift of Package t
- **Decision Variables**
 - **A/C interval selection:**

$$s_a^A \in \{0,1\} \quad \forall a \in A, \quad \text{with } \sum_{a \in A} s_a^A = 1$$

$$s_c^C \in \{0,1\} \quad \forall c \in C, \quad \text{with } \sum_{c \in C} s_c^C = 1$$
 - **Bin opening and assignment:**

$$v_t \in \{0,1\}, \quad \forall t \in T \text{ (bin open indicator)}$$

$$x_{i,t} \in \{0,1\}, \quad \forall i \in I, \forall t \in T_i \text{ (task } i \text{ assign to bin } t)$$
 - **Line decision:**

$$y_i^{line} \in \{0,1\} \text{ (1 if task } i \text{ left to line; only allowed for } cat_i = 1)$$
 - **Classification variables**

$$isA_t \in \{0,1\} \text{ and } isC_t \in \{0,1\} \text{ (bin type indicators)}$$

$$z_t^A \in \{0,1\} \text{ and } z_t^C \in \{0,1\} \text{ (overhead trigger)}$$

Table 3.1 Symbol and Parameters for MILP-based Model Formulation

Symbol	Description	Notes
I_i	Task Interval	Sources from MPD [25]
MH_i	Task Man-hours	Sources from MPD [25]
ϕ	Material Cost Factor	0.33 [31].

Symbol	Description	Notes
φ	Labor Rates	USD 60 [29], [30].
ω	Overhead Cost per Events	Section 3.2.3
ρ	Revenue Rate per hours	USD 14,000 [32], [33]
I_t	Bin Assignment Interval	Input and Program Computation
I_A, I_C	Interval A Check and C Check chosen	Output of Program
d_T	Downtime Hours	Section 3.2.4
$e_{i,t}$	Early Execution Penalty (in Flight Hours)	Compute variable
y_i^{Line}	Line Task	Decision variable
M_i, M_t	Manpower availability for task i and package t respectively	10 per shift [34]
H	Observed Planning Horizon	Input (Section 4.1)

3.2.2. Labor Cost (C_{Labor}) and Material Cost ($C_{Material}$)

These components represent direct operating cost component and the primary variable operating costs of maintenance execution. These are derived from how maintenance task executed and assumption as mention, calculated as sums of the resources consumed. These formulas will be calculated for each task:

$$C_{Labor} = \sum_{i \in I} \sum_{t \in T} \varphi \cdot MH_i \cdot x_{i,t} \quad 3.2$$

$$C_{Material} = \phi \cdot C_{Labor} \quad 3.3$$

Where $x_{i,t}$ is the binary decision variable assigning task i to available bin/package t .

3.2.3. Overhead Cost ($C_{Overhead}$)

Overhead cost represents fixed logistical and administrative costs incurred per maintenance event, independent of the number of maintenance packages executed during that event. In the optimization model, overhead is approximated using bin-activation proxies, while the exact overhead cost is recomputed from the number of maintenance events. Overhead reflects indirect costs linked to the environment where the event takes

place. Events requiring hangar access (base maintenance tasks) activate higher overhead, while apron-based events incur lower overhead as detail in Table 3.1.

Table 3 2 Overhead Cost Components

Components	Line	Base	Note
Hangar/Storage	-	USD 200,000	[31]
Administration & Tools	USD 40,000	USD 100,000	[31], [36]
Towing (Apron to Hangar)	-	USD 400	[8], [35]
Total	US 40,000	USD 300,400	

This cost components also captures of how different treatment regarding tasks with special environment requirement (hangar facilities) as mention in Section 2.2.3. Base maintenance tasks will be mark later on in preprocessing process and explain in detail at Section 3.5. As this cost component calculated for each maintenance events, if in the event no task needs hangar arrangement then the event will be executed in Apron and Overhead Cost will be calculated accordingly. This formulation is accordance to objective of minimize total cost and respect real-world execution situation. Based on the list on Table 3.1 that happen for each maintenance event and respecting how real-world situation work:

$$C_{Overhead} = \sum_t \omega_{Apron} \cdot z_t^A + \sum_t \omega_{Hangar} \cdot z_t^C \quad 3.4$$

3.2.4. Downtime Cost ($C_{Downtime}$)

The downtime cost is the primary cost component that signify loss of revenue of grounding an aircraft. It quantifies the lost of revenue that the aircraft would have generated if it were operational [8], [27]. This metric ensures that the economic impact of grounding the aircraft is fully internalized by the optimization model.

The core challenge lies in accurately estimating the duration of the maintenance package for integration into the MILP model. This study employs a simplified linear formula which utilizes engineering principle to provide robust estimation of total

maintenance downtime (d_T). Calculated as the revenue rate per hour multiplied by the total maintenance downtime (duration of the grounding event) [38], [39]:

$$C_{Downtime} = \sum_{t \in T} \rho \cdot d_T \quad 3.5$$

$$d_T = \frac{MH_t}{U_{eff} \cdot M_t} \quad 3.6$$

Where $U_{eff} = 0.3$, U_{eff} represent wrench time [40] and 9 hours is shift length [41]. $M_t = 10$ that represent manpower availability per shift [34]. Wrench time (also called *tool time*) measures how much of a mechanic's day is spent physically working on equipment [40].

3.2.5. Penalty Cost ($C_{Penalty}$)

This component penalizes the early execution of maintenance tasks relative to their optimal due date. This cost component is the primary opportunity cost model to the system representing trade-offs of chosen package assignment instead of its best assignment for global optimum result objective (total cost) to task packaging model formulation. Penalty cost measures the loss of interval life when a task is executed earlier than its allowable limit:

$$C_{Penalty} = \rho \cdot \sum_{i \in I} \sum_{t \in T_i} e_{i,t} \cdot x_{i,t} \cdot \left(\frac{H}{I_t}\right) \quad 3.7$$

$$e_{i,t} = I_i - I_t \quad 3.8$$

This formulation economically discourages “pulling tasks forward” unless doing so reduces overall cost through savings in downtime or overhead.

3.2.6. Line Cost (C_{Line})

Some tasks cannot be efficiently packaged, especially those with short intervals or operational classifications (line-routine tasks). These may be executed individually:

$$C_{Line} = \sum_{i \in I} y_i^{Line} \cdot \left(\frac{H}{I_i}\right) \cdot ((1 + \phi) \cdot \varphi \cdot MH_i + \frac{MH_i}{M_i} \cdot \rho + \omega_{Apron}) \quad 3.9$$

The model compares this cost with the cost of packaging, allowing flexible determination of the most economical option.

3.2.7. Interaction and Model Behavior

Based on all cost components definition and how its mathematical formula, the final form of total cost and objective of this research formulation model is expressed as follow:

$$\begin{aligned}
Z = & \underbrace{(1 + \phi) \cdot \sum_{i \in I} \sum_{t \in T} \varphi \cdot MH_i \cdot x_{i,t}}_{DMC} + \underbrace{\sum_{t \in T} (\omega_{Apron} \cdot z_t^A + \omega_{Hangar} \cdot z_t^C)}_{Overhead\ Cost} \\
& + \underbrace{\rho \cdot \sum_{t \in T} \frac{MH_t}{U_{eff} \cdot M_t}}_{Downtime\ Cost} + \underbrace{\rho \cdot \sum_{i \in I} \sum_{t \in T_i} e_{i,t} \cdot x_{i,t} \cdot \left(\frac{H}{I_t}\right)}_{Penalty\ Cost} \\
& + \underbrace{\sum_{i \in I} y_i^{Line} \cdot \left(\frac{H}{I_i}\right) \cdot ((1 + \phi) \cdot \varphi \cdot MH_i + \frac{MH_i}{M_i} \cdot \rho + \omega_{Apron})}_{Line\ Cost}
\end{aligned} \tag{3.10}$$

This mathematical formula will better represent how interplay and correlation of each cost components and its intended behavior. From interplay between these components drives the optimization behavior. High *Downtime* and *Overhead* costs incentivize packaging tasks into fewer, consolidated events. Simultaneously, the *Penalty (Opportunity)* cost prevents suboptimal “pulling forward” of tasks into the earliest possible package, pushing the solution toward maximizing utilized intervals. The *Line* cost offers a pathway to unpackage tasks if their inclusion significantly increases the duration or penalty costs of main package. This dynamic balance ensures the optimized packaging arrangement is both cost-efficient and operationally robust. The flowchart of this behaviour is planned and its implementation described in Figure 3.2.

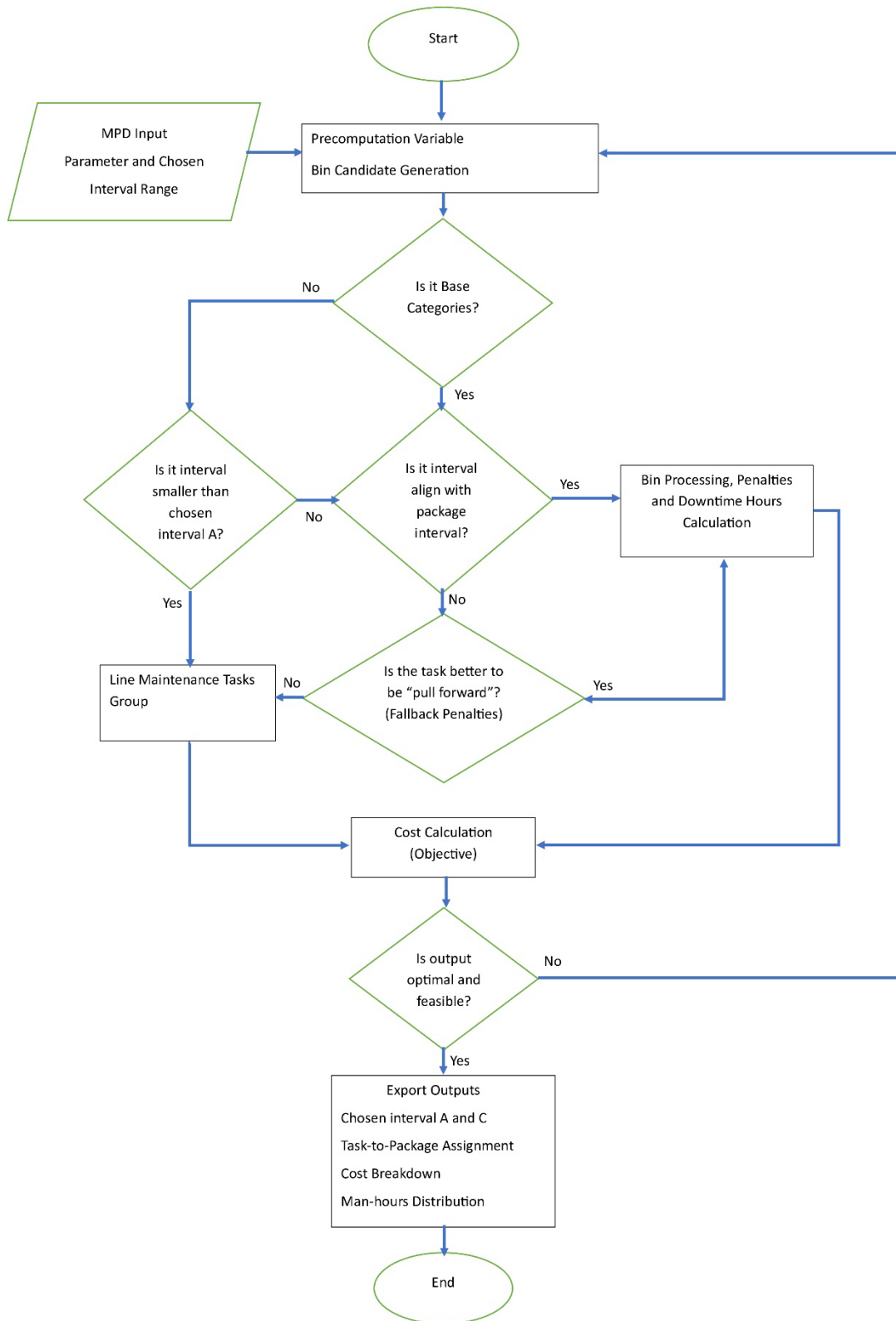


Figure 3.2 Conceptual Logic Flow of the Optimization Model

Figure 3.2 illustrates the sequential decision logic implemented in the optimization model, starting from MPD task inputs and selected interval ranges, and ending with the final optimized packaging output. The process begins with the generation of precomputed variables and candidate bins based on the chosen A- and C-check intervals. These bins represent all possible maintenance opportunities within the planning horizon.

The model first checks whether a task belongs to a base-maintenance category. Base tasks can only be considered for packaging into A- or C-check events; line-maintenance tasks bypass this step and proceed directly to the classification stage for potential line execution. For base-maintenance tasks, the model examines whether the task's interval aligns with the selected package interval. If the task naturally fits into a bin according to its interval relationship, it proceeds to bin processing where its man-hours, penalty implications, and downtime contributions are calculated.

When a task does not align cleanly with the selected package interval, the model evaluates whether performing the task earlier—i.e., “pulling forward”—is preferable. This evaluation considers the penalty mechanism that represents the opportunity cost of consuming interval life prematurely. If early execution is economically acceptable, the task is processed in the same manner as aligned tasks, including penalty and downtime evaluation. If early execution is not justified, the task is allocated to the line-maintenance group, meaning it will not be packaged into an A- or C-check event.

Once all tasks are processed, the model aggregates the contributions of every chosen bin into the objective function, which includes labor cost, material factor cost, overhead, downtime cost, penalty cost, and line execution cost. The solver then evaluates whether the resulting configuration is both feasible (respecting intervals and resource limits) and optimal with respect to total cost. If not, the model loops back, adjusting decisions until an optimal and feasible solution is found.

When an optimal solution is achieved, the model exports all outputs: the chosen A- and C-check intervals, the task-to-package assignments, the detailed cost breakdown by component, and the distribution of man-hours across generated maintenance events. This solution represents the most cost-efficient and technically compliant task packaging arrangement for the given scenario.

3.3. Output Design Rationale and Method Suitability

The model's design was driven by engineering utility and visualization of its output: they must be auditable, traceable, and directly usable in aircraft maintenance task packaging assignment. Outputs are structured to provide discrete assignment decisions, clear visualization, and cost decomposition that can be verified against regulatory requirements. This section explains why outputs are designed in this way, how they justify the use of Mixed-Integer Linear Programming (MILP), and how they correlate with the broader model formulation.

3.3.1. Discrete vs. Continuous Results

Aircraft maintenance programs operate under **discrete regulatory intervals** (e.g., A-checks at 600 FH, C-checks at 6000 FH). Continuous optimization of check timing would yield fractional or non-divisible results that cannot be executed in practice. The model therefore produces **binary assignment decisions**:

$$x_{i,t} \in \{0,1\}, \quad \text{"Task } i \text{ is assigned to bin } t\text{"}.$$

This ensures outputs answer the engineering question in precise terms: "*Does Task A go to Package 1 (Yes/No)?*" rather than producing ambiguous fractional allocations.

- **Engineering justification:** Discrete outputs align with maintenance manuals and airworthiness regulations, where checks are defined at fixed flight-hour multiples.
- **Method suitability:** MILP is the natural framework for such combinatorial decisions, as it can guarantee global optimality across discrete candidate sets.

3.3.2. A- and C-interval selection

The choice of the "best" A- and C-intervals (I_A and I_C respectively) are also modelled as a **discrete selection problem**. This discrete model is to be tailored with MILP since a continuous will make a more complex arrangement, so it approach by making candidate selection with certain range and increment to be closely reflect a continuous result, since with increment that can be modified then to make a more precise result the increment can be adjust with trade-offs solver time because of the expansion of search space. The solver must commit to exactly one $a^* \in A$ and one $c^* \in C$. This design reflects engineering reality: operators adopt fixed policies for A- and C-checks, not continuous or fractional intervals.

- **Engineering justification:** Discrete interval selection ensures that the resulting packages are consistent with actual maintenance programs and can be certified.

- **Method suitability:** MILP allows endogenous policy selection within the same optimization, ensuring that interval choice and task packaging are jointly optimal.

3.3.3. Traceability and Auditability

Outputs are designed as **matrix visualizations** (task-to-package assignment tables) and **locked schedules** (occurrence lists). These formats allow certifying engineers to verify every task's placement against regulatory requirements.

- **Engineering justification:** Maintenance planning must withstand regulatory audit; outputs must be traceable from decision variables to occurrence schedules.
- **Method suitability:** MILP's binary assignment structure translates directly into auditable tables, avoiding the opacity of heuristic or continuous approaches.

3.3.4. Correlation with Model Design

Each output designed to corresponds directly to a modelling construct, it correlate to objective function (Section 3.2), Model's Constraints (Section 3.4) and Model's formulation:

- **Task-to-package matrix** ↔ indivisible packaging constraint.
- **Locked schedule** ↔ divisibility-based bin classification and occurrence propagation.
- **Package summary** ↔ capacity and workload aggregation constraints.
- **Cost decomposition** ↔ proxy vs exact reconciliation in the objective function.

This correlation ensures that outputs are not post-hoc approximations but direct reflections of the mathematical formulation, reinforcing their engineering credibility.

Figure 3.3 provides a conceptual visualization of how the model organizes task assignments, making the relationship between interval limits, bin availability, and assignment outcomes easier to interpret. Rather than merely showing a technical decision, the diagram highlights the structural patterns that emerge when maintenance opportunities are viewed as discrete containers with cost implications.

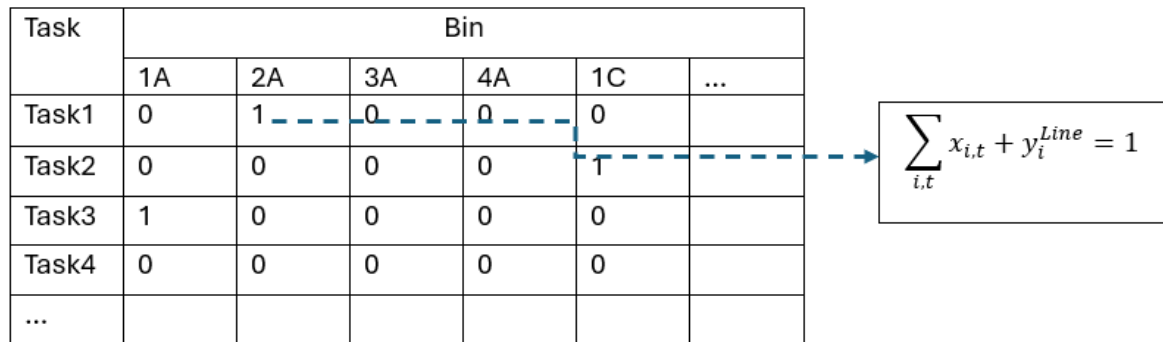


Figure 3.3 Design Output for Optimization Model

As shown in Figure 3.3, tasks are mapped to their feasible bins, and the binary assignment variable $x_{i,t}$ determines whether they are included in a specific maintenance event. If no assignment is made (e.g., Task 4), the model indicates that the task is better executed individually. This visualization helps reveal packaging behavior:

- which tasks naturally cluster,
- which tasks require special conditions,
- and where interval interactions force trade-offs.

By interpreting these patterns, planners gain a refined understanding of the underlying logic inside the optimization, supporting the research's intent to present a more structured and dynamic view of task packaging behavior.

3.4. Constraints for Design System (MILP-based)

The following constraints guarantee the feasibility and compliance of the maintenance task packaging execution:

3.4.1. Policy Selection

The model captures strategic interval selection by choosing exactly one A-Check and C-Check interval that best suits the task portfolio and economics over the horizon, rather than optimizing assignment under a fixed policy.

$$\sum_{a \in A} s_a^A = 1, \quad \sum_{c \in C} s_c^C = 1 \quad 3.11$$

This constraint forces the solver to commit to single A and C interval globally, enabling the MILP to co-optimize interval choice and packaging.

3.4.2. Bin Classification

The model keeps bin semantics consistent with chosen intervals, avoiding static pre-labeling that could misclassify bins when A-check and C-check intervals change.

$$isA_t = \sum_{a \in A} s_a^A \cdot 1\{a|t\}, \quad isC_t = \sum_{c \in C} s_c^C \cdot 1\{c|t\}, \quad \forall t \in T \quad 3.12$$

This constraint classifies each bin endpoint t as A-type or C-type depending on the chosen intervals (A-check and C-check intervals exact divisibility). This keeps bin semantics consistent with maintenance policy after interval selection.

3.4.3. Overhead Trigger Linearization

The model uses z_t^A, z_t^C to avoid bilinear cost terms while retaining exact logic: overheads are charged only when a bin is both opened and of the corresponding type.

$$\begin{aligned} z_t^A &\leq isA_t, & z_t^A &\leq v_t, & z_t^A &\geq isA_t + v_t - 1 \\ z_t^C &\leq isC_t, & z_t^C &\leq v_t, & z_t^C &\geq isC_t + v_t - 1 \end{aligned} \quad 3.13$$

This constraint enforces $z_t^A = isA_t \cdot v_t$ and $z_t^C = isC_t \cdot v_t$ without bilinear terms, so overhead costs apply only when a compatible bin is opened. This will preserve linearity and exact logic.

3.4.4. Bin Compatibility Constraints

To prevent “cheat” happen by the solver, the model design to only open compatible bins and assign task i only to compatible open bin t .

$$\begin{aligned} v_t &\leq isA_t + isC_t, & \forall t \in T \\ x_{i,t} &\leq isA_t + isC_t, & \forall i \in I, \quad \forall t \in T_i \end{aligned} \quad 3.14$$

These constraints prevent opening bins that are incompatible with the selected A/C intervals, strengthening the LP relaxation and eliminating non-sensical openings, also ensuring tasks are assigned only to valid A/C bins under the chosen intervals, preserving operational feasibility.

3.4.5. Task Assignment to Opened Bins

The model is design to prevent task to closed bin. This constraint coupled with bin compatibility constraints strengthen the LP relaxation and reducing branch-and-bound effort.

$$x_{i,t} \leq v_t, \quad \forall i \in I, \quad \forall t \in T_i \quad 3.15$$

This constraint prohibits assignment to closed bins, coupling workload decisions with bin opening costs, aligning costs and capacity with actual task packaging assignment.

3.4.6. Packaging and Line Decisions

The model restricting line to $cat_i = 1$ enforces operational boundaries. The line proxy captures expected recurring cost over planning horizon (H), making the packaging vs line trade-off transparent and economically grounded.

$$\sum_{t \in T_i} x_{i,t} + y_i^{Line} = 1, \quad \forall i \in I \quad 3.16$$

This constraint ensure each task is either packaged once (anchored to a single bin $\leq I_i$) or entirely left to line. This single anchor propagates occurrences across the horizon in post-solve validation.

3.4.7. Interval Compliance Constraints (Due-Date Rule)

Tasks are assigned before their intervals, in other word its bin interval (bin/package where the task is assigned) must not exceed due date (task interval).

$$x_{i,t} = 0, \quad \text{if } I_t > I_i \quad 3.17$$

This constraint enforces strict compliance by construction—variables $x_{i,t}$ are only created for $I_t \leq I_i$, preventing anchors beyond permissible intervals and simplifying the model.

3.4.8. Bin Workload Aggregation

This constraint defines a linear aggregate used by the downtime proxy in the objective, maintaining convexity and capturing workload intensity.

$$MH_t = \sum_{i \in I} MH_i \cdot x_{i,t}, \quad \forall t \in T \quad 3.18$$

3.4.9. Line Eligibility Restriction

This constraint enforces operational policy that only line-eligible tasks can be left to line maintenance, preventing unrealistic deferrals. The model will assigned task with $I_i \leq I_A$ automatically treated as Line Task.

$$\begin{aligned} T_i = \emptyset &\Rightarrow y_t^{Line} = 1 \\ cat_i \neq 1 &\Rightarrow y_t^{Line} = 0 \end{aligned} \quad 3.19$$

Figure 3.4 summarizes how the constraints interact to form a coherent decision-support framework. Rather than presenting constraints as isolated rules, the diagram shows how interval selection, bin classification, task assignment, and cost logic flow together into a structured optimization sequence. This visualization helps illustrate how the mathematical formulation reflects the engineering decision process.

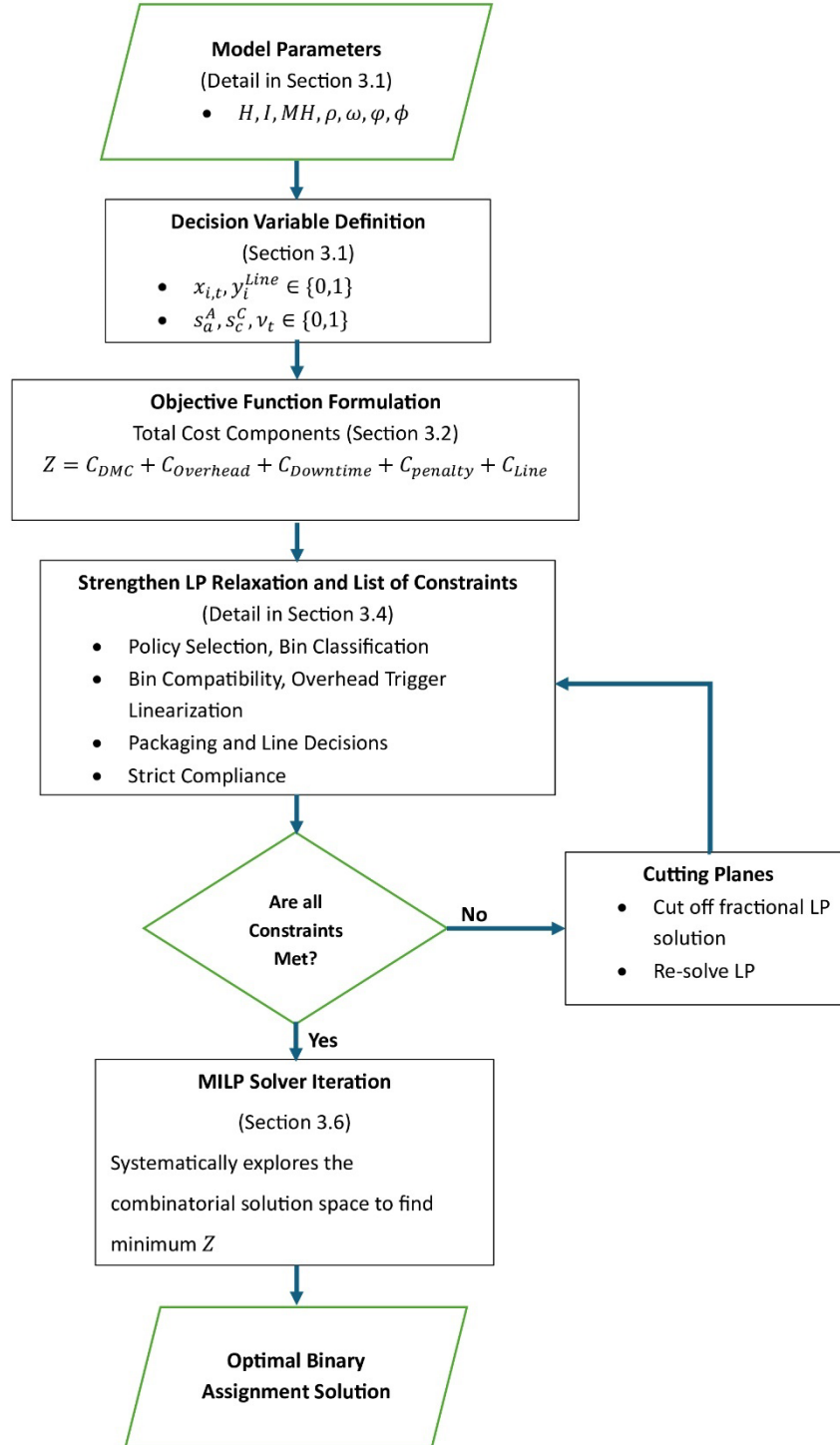


Figure 3.4 MILP Implementation on Design Decision-Support System

As shown in Figure 3.4, the constraint system creates a layered decision structure. The model begins by selecting interval policies, which then shape bin identity and determine which maintenance opportunities are feasible. Tasks are evaluated against these opportunities, and the model considers packaging, early execution, or line maintenance as valid alternatives. Cost

components then guide the solver toward the configuration that best balances operational feasibility and economic efficiency. This flow reinforces the research’s perspective that maintenance task packaging can be studied as a structured, interactive decision system rather than a purely procedural activity.

3.5. Data Acquisition and Preprocessing Methodology

The methodology for preparing data inputs is crucial for ensuring integrity and operational realism. The preprocessing pipeline was designed to transform raw Maintenance Planning Document (MPD) spreadsheets into normalized, auditable datasets suitable for optimization. Each step was engineered to preserve traceability, enforce engineering validation, and align with utilization assumptions defined in Section 3.1.

- **Data Sources:** Raw MPD data and historical maintenance records are sourced from the MRO’s maintenance tracking system [25]. These inputs are validated by maintenance engineers to ensure engineering compliance and operational relevance.
- **Preprocessing Steps:** Data is cleaned and validated to ensure compliance with engineering standards.
 - **Column normalization:** Raw headers are mapped to canonical names (e.g., *TASK NUMBER* → *TaskID*, *TASK M.H.* → *TaskMH*) to establish a consistent schema.
 - **Interval conversion:** Heterogeneous thresholds and intervals (YE, MO, DY, FC, FH) are parsed and converted into flight-hour equivalents using utilization constants from *Utilization Forecast Data* (*FH per day*, *FH per cycle*, *FH per month*). Flags such as NOTE and CE are extracted to preserve engineering metadata. (NOTE: Refer to the note in the task description; and CE(Customizable Equipment): Consult the vendor's documentation for this interval/threshold) [10].
 - **Manhour aggregation:** Multiple manhour-related columns (TaskMH, AccessMH, PrepMH, Manhours) are normalized and summed into a single runtime labor field (*ManhoursRT*).
 - **Technician count:** The *Men* field is coerced to numeric, with a *MenZero* flag indicating missing or zero values.
 - **Environment eligibility:** The *Check_Categories* field is mapped to a binary *Categories* flag: Line/apron-executable tasks are encoded as 1, while Base/Hangar-only tasks are encoded as 0. This distinction is critical for downstream cost modeling.

- **Traceability:** A sequential *RowID* is assigned to each record, ensuring reproducibility across CSV and Excel outputs.
- **Canonical interval field:** A unified *IntervalFH* column is created, representing the normalized flight-hour interval for each task.
- **Outputs:**
 - **Cleaned CSV:** A full normalized dataset is exported for archival and audit purposes.
 - **Optimizer input subset:** A filtered dataset containing only relevant fields (*RowID*, *TaskID*, *IntervalFH*, *ManhoursRT*, *Men*, *Categories*, *Zone*) is exported to `optimizer_input.csv`. This ensures the optimizer operates only on validated, model-ready parameters.
 - **Excel export:** An optional formatted Excel file is generated for engineering review, with auto-width columns and styled headers for readability

Figures 3.5 illustrate data acquisition and preprocessing module. This rigorous preprocessing (detailed further in **Appendix A** and **Appendix B**) ensures that the mathematical model operates on reliable, engineering-approved data. By explicitly encoding environment eligibility (*Categories*), the methodology captures overhead implications: tasks executable on the apron (*Categories* = 1) minimize facility overhead, while Base/Hangar tasks (*Categories* = 0) incur additional costs. This distinction is essential for accurate cost modeling and ensures that optimization results reflect real-world execution constraints.

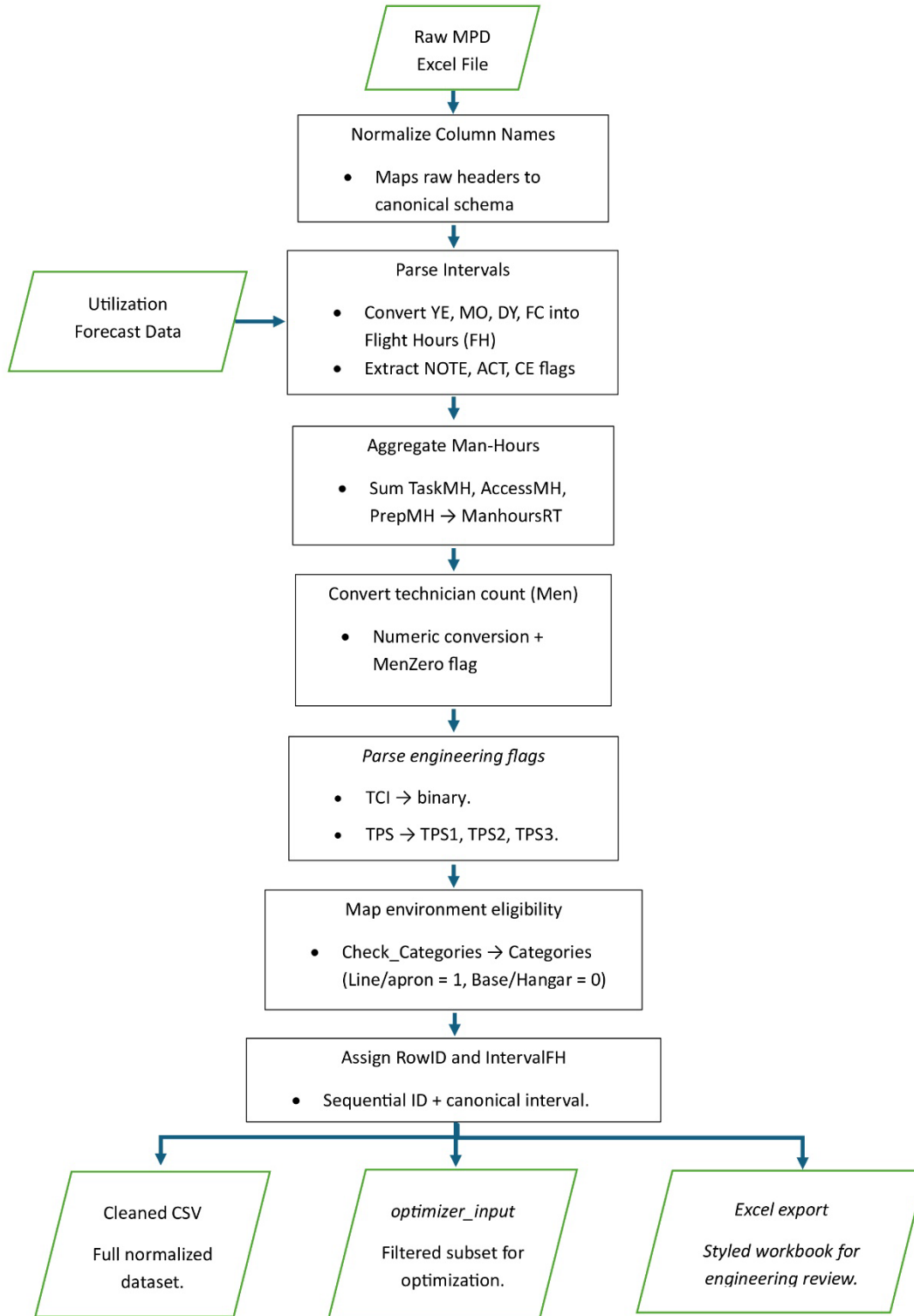


Figure 3.5 Conceptual Logic Flow of Data Acquisition Process

3.6. Heuristic Model

Although the main focus of this study is the MILP formulation (Section 3.4), a heuristic model was also developed to provide a fair point of comparison. This model reflects traditional

maintenance philosophies discussed in Section 2.2 (block and equalized methods) and allows the performance of the MILP to be evaluated against a transparent, rule-based benchmark.

The heuristic model operates by enumerating candidate combinations of A- and C-check intervals across the planning horizon (see Section 3.5 for data preparation). For each candidate pair, tasks are mapped according to simple rules:

- In block mode, tasks are bundled into the nearest C-multiple, emulating the classic letter-check system.
- In equalized mode, tasks are distributed across A-bins within each C-window to balance workload peaks.

Once candidate mappings are generated, the model applies a greedy heuristic to test whether certain line-eligible tasks (Category 1) should be removed from packages. If unpackaging a task reduces total cost, it is left as a line task, reflecting operational pragmatism in line/base contexts (Section 2.2.3).

Costs are computed using the same decomposition as the MILP objective (Section 3.2), including direct labor, material, overhead, downtime, line costs, and opportunity penalties. This ensures that the comparison in Chapter 4 (Section 4.3.1) isolates methodological differences rather than data or cost assumptions. Outputs of the baseline include validation sheets, package summaries, and workload visualizations. Key code snippets and implementation details are documented in **Appendix B**. Together, these provide a reproducible audit trail and demonstrate how the MILP achieves measurable improvements over heuristic method.

3.7. System Architecture and User Interface Procedure

This section describes the practical implementation of the optimization model develop in Section 3.1–3.5, detailing the architecture and step-by-step procedure for utilizing the costum built decision-support system software application.

3.7.1. System Architecture Overview

This practical implementation results in an application strategy with procedure as guidance that follows a data-to-decision pipeline architecture designed for transparency and reproducibility. Figure 3.4 illustrates the system’s architecture.

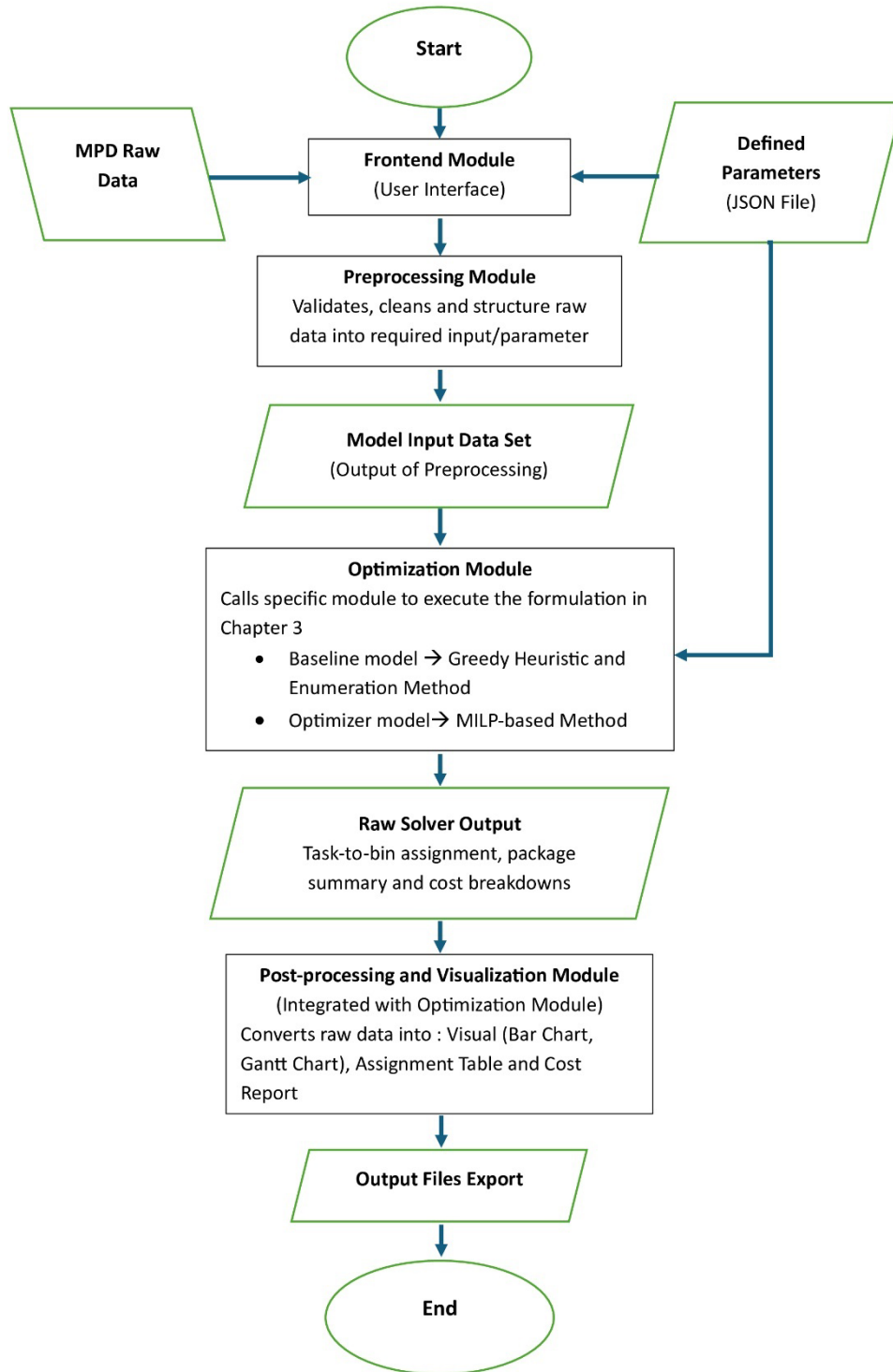


Figure 3.6 Engineering Data-to-Decision Pipeline Architecture

As illustrate by Figure 3.4, the optimization system is a modular application design to provide a structured workflow for maintenance planners. It consists of the following key modules:

- **Frontend Module (mtp_frontend):** A Graphical User Interface (GUI) that manage user interaction, data input/upload (MPD data, parameter JSON file, utilization forecasts), and process initiation. The interface requires specific data:
 - **MPD Raw Data:** A CSV or Excel files containing the complete list of mandatory maintenance tasks, their descriptions (Task ID, Description, Task Code), engineering-defined data (task intervals, man-hours, minimum manpower to execute the task), and other information (zones access, skill code).
 - **Parameters JSON File:** A file containing baseline cost parameter values (*labor rate, material cost factor, Overhead cost per package*) as described in Section 3.2.
 - **Average Utilization Forecast:** An adjust input value used for planning various scenarios (e.g., average flight hours per month)
- **Preprocessing Module (mtp_preprocessing):** This module cleans, validates, and transforms raw input data into standardized format required by the optimization solvers. The methodology within this module is detailed in Section 3.5.
- **Optimizer Modules:**
 - **Baseline Model (mtp_heuristic_baseline):** Implements traditional industry heuristic logic for comparative analysis
 - **MILP-based Model (mtp_milp_optimizer):** Executes the primary optimization logic formulated in Section 3.2, employing the constraints defined in Section 3.4.
- **Post-processing (Integrated):** Integrated within the optimizer modules, this handles output generation (CSV files for task-to-package matrix, cost breakdowns) and visualization (figures and charts)

3.7.2.Step-by-Step User Procedure

The following procedure outlines the detailed explanation of Figure 3.4, how an engineer or planner utilizes the system to generate an optimized task packaging strategy for a case study aircraft (e.g., A320).

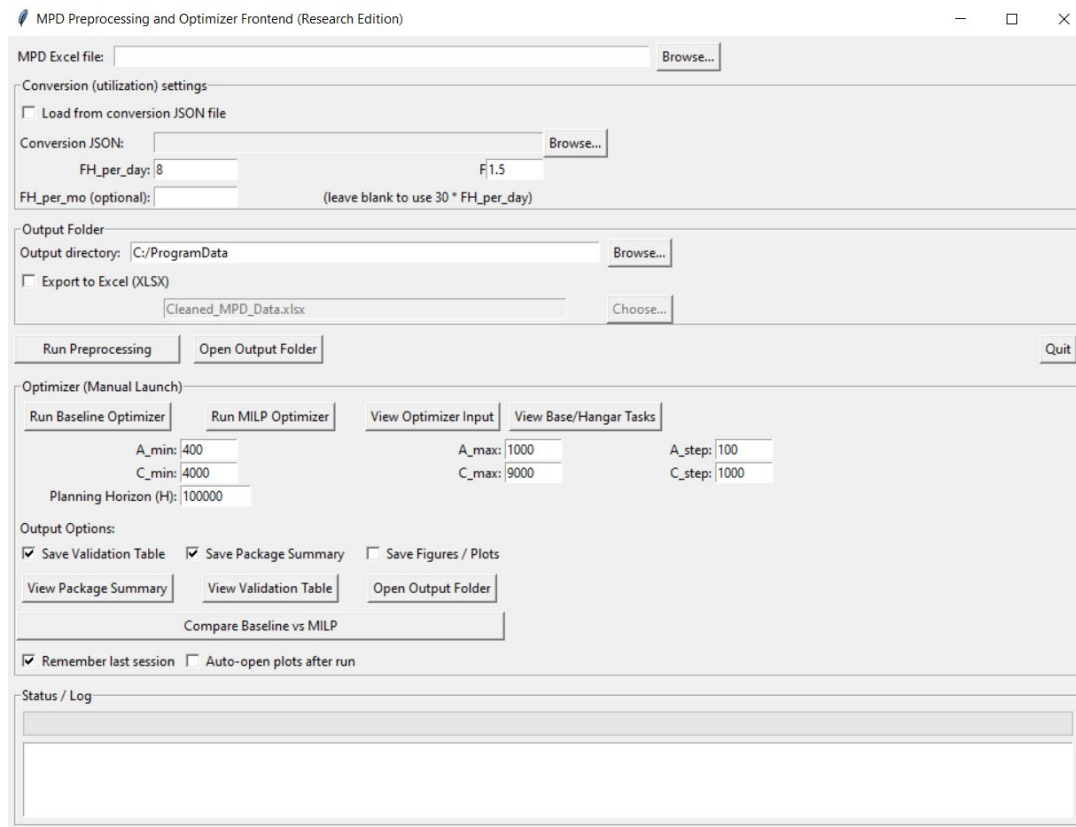


Figure 3.7 Graphical User Interface of Costum Built Decision-Support System

- Step 1: Launch the Application and Input Raw Data**

The user initiates the process by running the `mtp_frontend` module. The GUI prompts the user to input or upload the necessary raw data:

- Upload the raw MPD data file
- Upload the JSON parameter file.
- Input the average utilization forecast

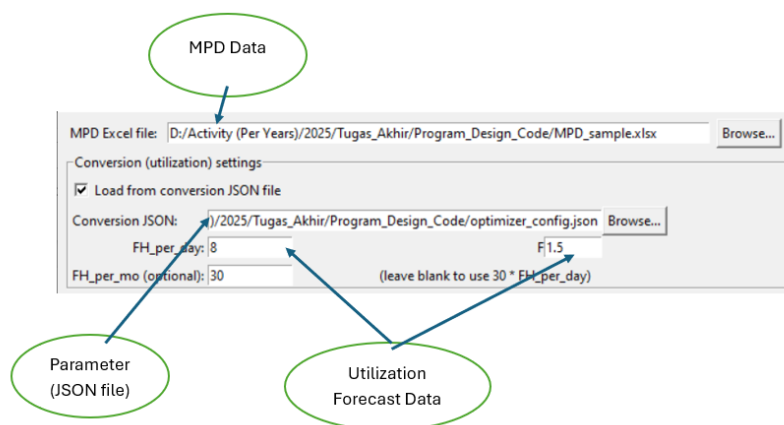


Figure 3.8 Input Data Process to System

- **Step 2: Generate Optimizer Input Data (Preprocessing)**

The user selects the option to execute the `mtp_preprocessing` module. This crucial step prepares the data for the solvers, as detailed in Section 3.5. The user can optionally review the pre-processed data output before proceeding.

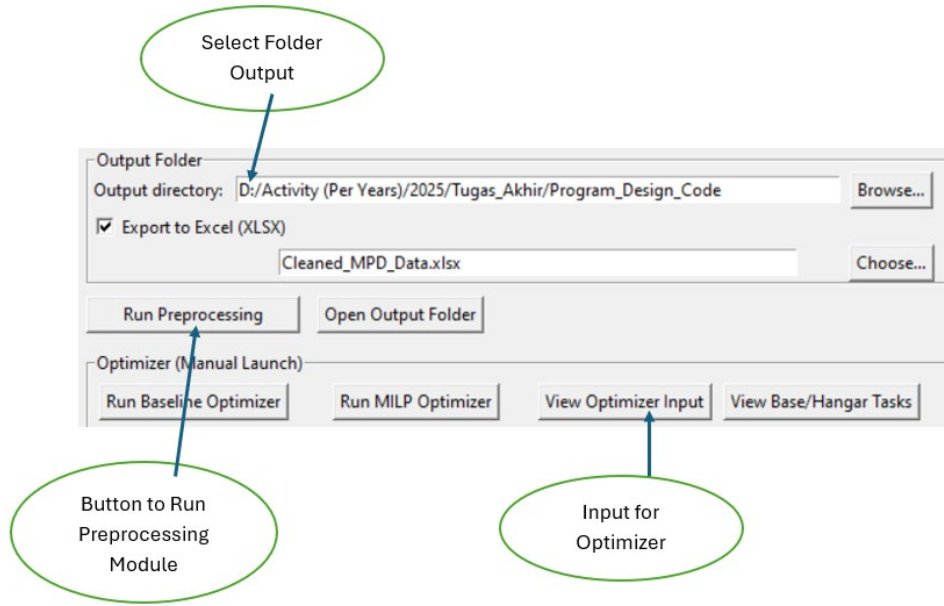


Figure 3.9 Run Preprocessing Module

- **Step 3: Define Interval Candidates and Parameters**

The user defines the discrete range of potential check intervals within the GUI. As detailed in Section 3.1, this ensures the model outputs operationally relevant, standardized intervals.

- **A-Check Range:** Input A_{min} , A_{max} , and A_{step} (e.g., 400 FH, 1000 Fh, 100 FH increment)
- **C-Check Range:** Input C_{min} , C_{max} , and C_{step} (e.g., 4000 FH, 9000 Fh, 1000 FH increment)

The system uses the parameters uploaded in Step 1 to influence the cost calculations for the optimization.

- **Step 4: Execute Optimization Model**

The user selects the desired model to run (either the baseline model or the primary **MILP-based model** for the study's results).

Upon execution, the system log within the GUI provides real-time feedback. Notifications confirm a successful run, or annotations indicate specific errors if constraints are violated or input data is invalid.

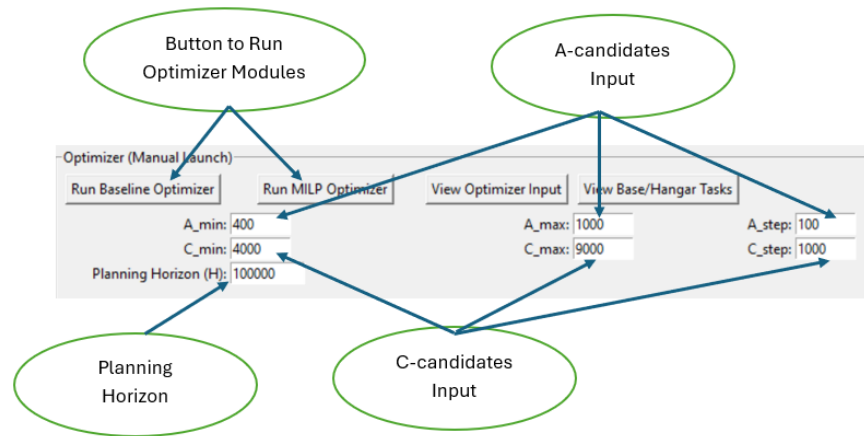


Figure 3.10 Run Optimizer Modules

- **Step 5: Review Results and Access Outputs**

Upon successful completion, a summary of key metrics appears in the log, providing immediate insights:

- Optimal A-Check Interval
- Optimal C-Check Interval
- Total Cost (optimized value)
- Number of Unpackaged Task (Task that judge better be done individually to minimize cost)
- Average Cost per Planning Horizon

Simultaneously, the post-processing routine saves detailed outputs to the local directory:

- **CSV Files:** Contain the complete Task-to-Package Matrix, detailed cost component breakdowns, and compliance reports.
- **Visualization Files:** Generated figures (e.g., bar charts of man-hours distribution, cost component value analysis, Gant Chart for Event across horizon) that are further analyzed in Chapter 4.

This procedure ensures the methodology is repeatable and functional, bridging the gap between the theoretical model and its practical application.

Chapter 4

Result and Analysis

Building upon the mathematical formulation detailed in Chapter 3, this chapter shifts focus to practical execution of the MILP model. It covers the specific software system developed, the implementation strategy, and the presentation of illustrative optimal results derived from a case study. The section concludes with comparative analysis of the findings against a baseline heuristic model, interpreting the results within the engineering context and addressing the research objectives defined in Chapter 1.

4.1. System Implementation and Experimental Setup

This section describes how the generated optimal outputs are applied and integrated into the operational engineering workflow, leveraging the system described in Section 3.6. To execute the MILP model efficiently and bridge the gap between complex optimization algorithms and practical application, a custom software system was developed in Python, utilizing Pyomo library and Gurobi solver API (Section 3.6) [26].

4.1.1. Software Architecture and Solver Configuration

The optimization model was implemented as a Python-based system using the Pyomo modeling framework. Pyomo provides a structured interface for expressing MILP decision variables, constraints, and objective functions exactly as formulated in Section 3.2 and Section 3.4. The optimization tasks were solved using the commercial Gurobi solver through the Gurobi–Pyomo API (Section 3.7).

The software environment used for all computational experiments consisted of:

- **Programming language:** Python 3.11
- **Modeling library:** Pyomo, Pandas, Numpy, Matplotlib
- **Solver:** Gurobi (with fallback support for CBC or GLPK)
- **Machine specification:** Standard engineering workstation (Intel i7 CPU, 8 GB RAM)

To ensure solution consistency and tractability within the single-aircraft scope, the solver was configured with the following settings:

- **Time limit:** 600 seconds
- **MIP relative gap tolerance:** 0.01%
- **Default branching and presolve parameters**

These solver parameters were fixed across all experiments to ensure that differences observed in Chapter 4 arise solely from model behavior rather than computational tuning.

4.1.2. Case Study Inputs and Data Preparation Summary

All input data were derived from the A320-family MPD dataset and preprocessed using the pipeline described in Section 3.5. This includes interval normalization, man-hour aggregation, line/base eligibility classification, and assignment of unified interval units.

Table 4.1 Case Study Configuration

Parameter	Value
Planning horizon (flight hours)	4 year (15000 FH)
Total number of MPD tasks	300
Utilization Forecast Data	8 FH/Day [31], [33], [34]
Labor rate (φ)	USD 60/ hour
Material fraction (ϕ)	1/3
Overhead A (ω_{Apron})	USD 50,000 per activation
Overhead C (ω_{Hangar})	USD 300,000 per activation
Revenue (ρ)	USD 14,000 per hour
Man & Shift hour	10 & 9 hours

This configuration defines the experimental environment for evaluating the MILP and heuristic models. Across all experiments in Chapter 4, the task set, MPD intervals, and cost parameters remain fixed; only utilization inputs and resulting interval conversions vary.

4.2. Auditable Outputs of the Optimization Model

A key requirement of any maintenance packaging tool is that its outputs must be transparent, traceable, and suitable for engineering audit. The MILP model developed in this research produces outputs that directly satisfy this requirement by expressing every decision in a structured, regulator-verifiable format. Table 4.2 presents a representative subset of the task-to-package assignment matrix to illustrate the binary allocation structure produced by the optimization model.

The subset shown is selected to include packaged A-check tasks, line-maintenance tasks, and recurring tasks with different interval lengths, thereby demonstrating the full range of assignment outcomes produced by the model rather than emphasizing any specific maintenance level. These fields allow verification not only the optimization result but also the compliance logic behind it.

Table 4.2 Representation of Task-to-Package Assignment

task_id	Task interval	task_mh	task_description	Assigned bin	Assigned level	Assigned package	compliance	300	600	900	1200	1500	1800	2100
...														
46	1200	0.25	212300-04-1	1200	A	4A	TRUE	0	0	0	1	0	0	0
47	3000	0.1	212600-01-1	3000	A	10A	TRUE	0	0	0	0	0	0	0
48	8640	0.1	212600-02-1			Line	TRUE	0	0	0	0	0	0	0
49	600	1.03	212643-01-1	600	A	2A	TRUE	0	1	0	1	0	1	0
50	1000	1.03	212643-02-1			Line	TRUE	0	0	0	0	0	0	0
51	600	0.19	212643-06-1	600	A	2A	TRUE	0	1	0	1	0	1	0
53	3000	0.16	212643-09-1	3000	A	10A	TRUE	0	0	0	0	0	0	0
54	3000	0.16	212643-10-1	3000	A	10A	TRUE	0	0	0	0	0	0	0
55	500	0.19	212643-11-1			Line	TRUE	0	0	0	0	0	0	0
56	500	0.23	212742-01-1			Line	TRUE	0	0	0	0	0	0	0
57	1800	1	215000-03-1	1800	A	6A	TRUE	0	0	0	0	0	1	0
58	1200	0.25	233300-03-1	1200	A	4A	TRUE	0	0	0	1	0	0	0
59	1000	0.1	242000-01-1			Line	TRUE	0	0	0	0	0	0	0
63	800	0.72	242000-23-1			Line	TRUE	0	0	0	0	0	0	0
64	800	0.46	242000-24-1			Line	TRUE	0	0	0	0	0	0	0
65	800	0.46	242000-24-1			Line	TRUE	0	0	0	0	0	0	0
66	300	0.06	242100-02-1	300	A	1A	TRUE	1	1	1	1	1	1	1
67	300	0.06	242100-02-1	300	A	1A	TRUE	1	1	1	1	1	1	1

The information contained in Table 4.2 can be interpreted as follows:

1. Task-Level Traceability

Each record preserves the original MPD reference number (`task_description`), allowing inspectors and planners to cross-check assignments directly against the approved Maintenance Planning Document. This ensures that the optimization result is fully traceable to the regulatory source document.

2. Compliance Verification

The compliance column confirms whether the execution timing of each task satisfies its MPD interval limit (I_i). A value of TRUE indicates that the task's assigned occurrence does not exceed its allowable interval, validating that the optimization has maintained full regulatory compliance. This satisfies the engineering requirement that no optimization decision may violate airworthiness constraints.

3. Assignment Logic

Two forms of assignment are visible:

- **assigned_bin (numeric value in FH)**
Indicates the specific bin where the task is first anchored.
Example: Task interval = 1000 FH → assigned_bin = 1000 means it is packaged at the 1000-hour opportunity.
- **assigned_package (A or C classification)**
Specifies whether the bin corresponds to an A-check sequence (e.g., "2A") or a C-check window. Tasks left unassigned to a bin are marked as *Line*, meaning the model determined that packaging the task is less cost-efficient than executing it individually, consistent with the line-task logic described in Section 3.2.7.

The table illustrates assignment structure and compliance logic rather than the relative prevalence of A-check, C-check, or line-maintenance tasks within the overall solution.

4. Occurrence Mapping Across the Planning Horizon

The binary columns (300, 600, 900, ...) indicate when the task will occur across the full planning horizon. A value of 1 indicates task execution at that horizon point, while 0 indicates no execution.

For packaged tasks, recurring occurrences are propagated automatically based on the assigned bin interval (e.g., a "2A" assignment results in execution every 600 FH). For line-maintenance tasks, occurrences follow the original MPD-defined interval. This

mapping provides a complete, horizon-wide record of task execution and compliance, which is essential for regulatory audit and maintenance planning verification.

4.3. Comparative Results and Cost Analysis

The MILP model was executed using the A320 case study data (**Appendix A**) and the program (**Appendix B**). Following the procedure in Section 3.6, both the heuristic baseline and the MILP model were evaluated under identical cost assumptions, task datasets, and interval candidate sets. This ensures that differences between the two approaches arise purely from methodological differences rather than data inconsistencies. The MILP solver successfully identified an optimal task packaging configuration while maintaining full compliance with all interval and engineering constraints

4.3.1. Comparison Framework and Evaluation Scope

The comparative analysis in this section evaluates three maintenance packaging outcomes: a baseline execution scenario reflecting actual fixed A-check and C-check intervals, a heuristic approach, and the proposed MILP-based optimization model. All results are generated using real input data derived from an A320-type aircraft case study, ensuring full reproducibility of the numerical values presented.

To ensure a fair and meaningful comparison, all observable cost components—direct maintenance cost, fixed overhead, downtime, and line task execution cost—are computed using the same cost formulations described in Chapter 3. A penalty-based term is additionally included as an internal modeling construct to represent inefficiencies associated with underutilization of remaining task life; however, this term does not correspond to an accounting cost reported in operational or financial records.

To avoid misinterpretation of economic performance, penalty-based cost components are excluded from reported total maintenance cost and average cost per flight hour metrics. While penalty terms play a critical role within the optimization objective by discouraging inefficient utilization of remaining task life, they do not represent observable expenditures recorded during maintenance execution. Accordingly, economic comparisons in this chapter are based exclusively on observable cost components, ensuring consistency with practical maintenance cost accounting.

Accordingly, interpretation of results in this section focuses primarily on observable cost components, while the penalty term is used solely to explain structural differences between task packaging approaches.

4.3.2. Interval Selection and Structural Difference

The first stage of comparison evaluates the **total cost** and its decomposition across five major components: Direct Cost, Penalty Cost, Fixed Overhead, Downtime Cost, and Line Task Cost.

Before viewing the numerical tables, it is important to highlight that:

- The **heuristic** approximates classic industry logic (Section 2.2) and explores interval combinations sequentially.
- The **MILP** evaluates the entire search space simultaneously and guarantees global optimality.

Table 4.3 summarizes the resulting A-check and C-check interval selections and corresponding cost components for each approach. The baseline scenario adopts fixed intervals derived from actual execution practice, whereas both the heuristic and optimization approaches independently select intervals that minimize their respective objective functions.

Table 4.3 Detail Cost Components from Comparative result

Cost Component	Heuristic (USD)	Optimization (USD)	Baseline (USD)
Key Parameters			
A_interval	500	300	1000
C_interval	9000	5000	12000
Cost Components			
Direct Cost	11926.73333	14282.4	46203.6
Penalty Cost	0	0	6120520000
Fixed overhead	1950000	2260000	860000
Downtime	103070.535	123428.1481	399290.3704
Line task	34429640.4	31759409.8	22424835.4
Total cost	36498573.49	34161833.54	6144265577
Average Cost per FH	2433.238233	2277.455569	409617.7051

The optimization model selects shorter A-check and C-check intervals relative to the baseline, reflecting its ability to align task execution more closely with interval utilization constraints. Compared to the heuristic, the optimization further refines interval selection by jointly considering package activation, task grouping, and cost interactions, rather than evaluating candidate interval combinations independently.

4.3.3. Observable Cost Component

Focusing on observable cost components, the optimization model achieves a reduction in line task execution cost relative to the heuristic, indicating more effective task grouping and reduced redundancy in maintenance execution. Although the optimization incurs higher fixed overhead and direct cost due to increased maintenance event frequency, these increases are offset by improvements in task packaging efficiency and lower average cost per flight hour.

Compared to the baseline execution scenario, both the heuristic and optimization approaches substantially reduce downtime and line task costs. This reflects the inefficiency of fixed-interval execution when task grouping is not optimized, particularly under the utilization conditions considered in this case study.

To evaluate the economic implications of each packaging approach, Figure 4.1 compares the observable maintenance cost components, excluding penalty-based terms. This comparison focuses on cost elements that are directly incurred and recorded during maintenance execution.

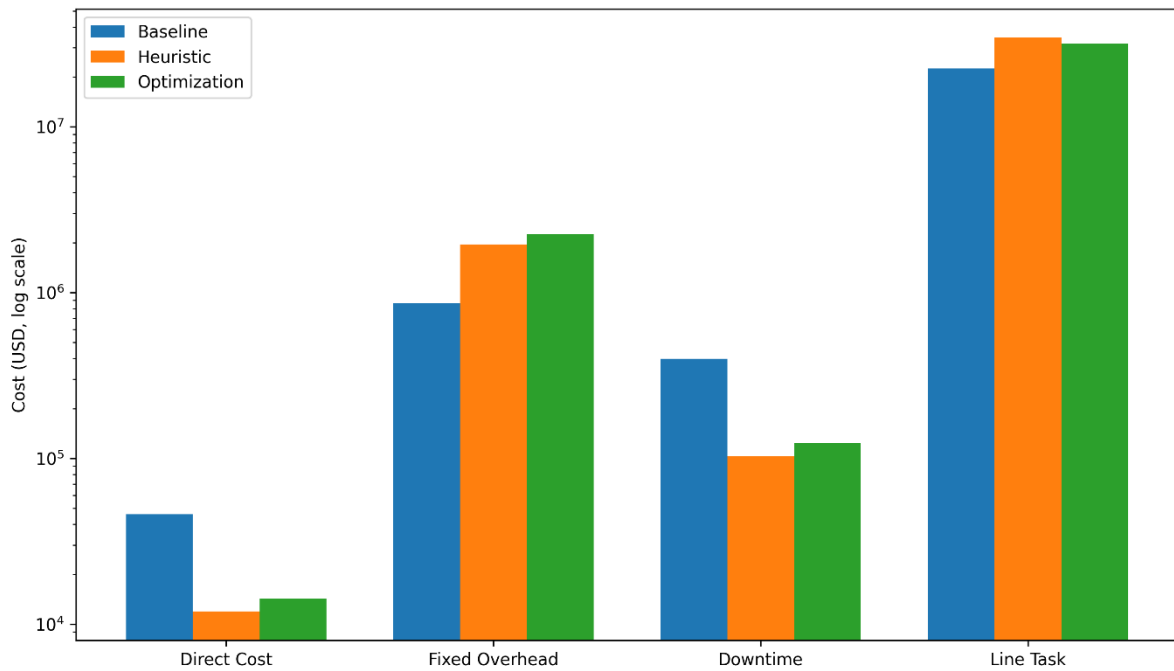


Figure 4.1 Comparison of observable maintenance cost components for baseline execution, heuristic method, and MILP optimization using real A320 case study data. Penalty-based cost terms are excluded. A logarithmic scale is used to improve visual clarity across cost components of different magnitudes.

Figure 4.1 shows that both the heuristic and optimization approaches significantly reduce line task execution cost and downtime relative to the baseline execution scenario. While the optimization model incurs higher fixed overhead and direct costs due to more

frequent maintenance events, these increases are offset by improved task grouping efficiency. The net effect is a more balanced and efficient cost structure compared to both baseline execution and heuristic method. To facilitate comparison across approaches under the same utilization conditions, overall economic performance is further evaluated using average maintenance cost per flight hour, as shown in Figure 4.2.

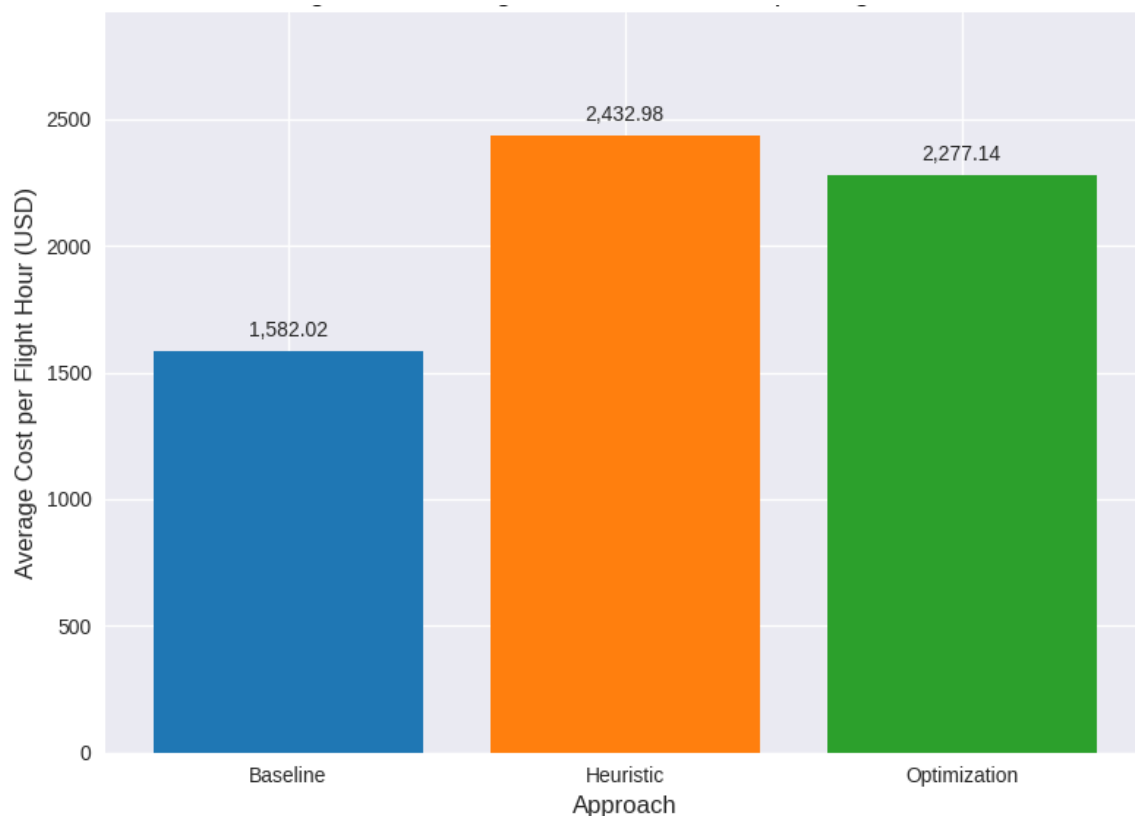


Figure 4.2 Average maintenance cost per flight hour for baseline, heuristic, and MILP optimization approaches under identical utilization conditions

Figure 4.2 presents the average maintenance cost per flight hour based on observable cost components. Although the baseline execution exhibits the lowest normalized value, this result reflects the exclusion of long-term inefficiencies associated with unused remaining task life rather than superior packaging performance. Both the heuristic and optimization approaches incur higher fixed and execution-related costs due to more frequent and structured maintenance events; however, these costs are incurred to improve task utilization and reduce inefficiencies that are not captured by observable accounting metrics alone. Consequently, the optimized solution demonstrates improved economic robustness when evaluated alongside utilization efficiency rather than normalized cost magnitude in isolation.

4.3.4. Interpretation of Penalty-Based Component

The penalty cost component shown in Table 4.3 represents a model-internal measure of inefficiency associated with unused remaining maintenance task life. In the baseline scenario, fixed interval execution leads to significant accumulation of unused task life, resulting in a large penalty value. In contrast, both the heuristic and optimization models eliminate this penalty by packaging tasks in a manner consistent with interval utilization constraints.

It is emphasized that the magnitude of the penalty term should not be interpreted as a direct economic saving. Instead, it serves as an indicator of structural inefficiency in maintenance task packaging and is used to guide solution behavior within the optimization framework. For this reason, penalty-based values are excluded from economic performance claims and are not used to quantify cost savings in this study.

4.3.5. Observable Cost Component

Overall, the comparative results demonstrate that the proposed optimization framework achieves a more balanced maintenance cost structure by improving task aggregation and utilization of available intervals, rather than by simply minimizing individual cost components. While the baseline execution may appear competitive when considering observable costs alone, it fails to account for inefficiencies associated with unused remaining task life, which are captured internally through penalty mechanisms within the optimization model. These findings confirm that the optimization approach provides a more robust and utilization-responsive maintenance task packaging reference under varying operational conditions.

Although the baseline execution may exhibit lower observable total cost when penalty terms are excluded, this does not indicate superior packaging performance. Instead, it reflects the inability of observable cost metrics alone to capture long-term inefficiencies associated with suboptimal task utilization, which are addressed by the penalty formulation within the optimization model.

4.4. Downtime Behaviour and Event-level Analysis

This section analyzes how the MILP optimization model structures maintenance events in terms of event-level downtime across the planning horizon. Downtime is a critical component of the objective function (Section 3.2), representing the opportunity cost associated with aircraft grounding and serving as a primary driver for efficient task packaging. Examining downtime patterns provides insight into how the model balances workload aggregation, interval compliance, and operational stability.

4.4.1. Downtime Distribution Across the Planning Horizon

Figure 4.3 presents the maintenance event downtime timeline generated by the MILP optimization over a four-year planning horizon (15,000 FH). Each bar represents a single maintenance event, with the bar height indicating total downtime and annotations indicating the number of packages executed within that event.

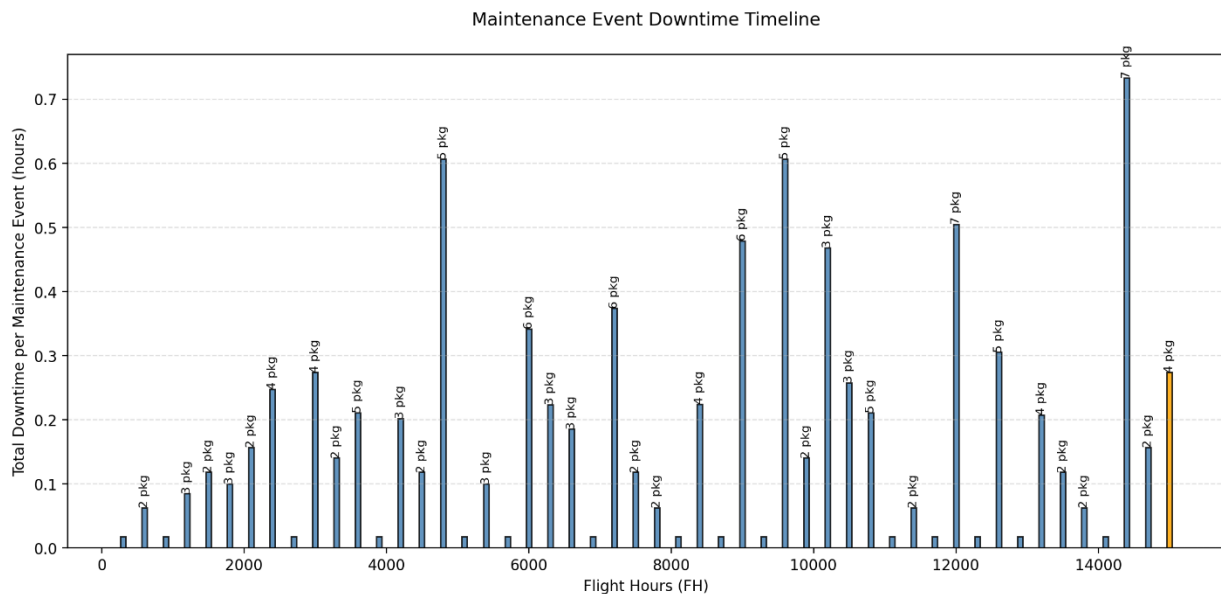


Figure 4.3 Distribution of Downtime Hours Across Planning Horizon (MILP Result)

Figure 4.3 illustrates the event-level downtime timeline produced by the MILP optimization over the 15,000 FH planning horizon, highlighting both the magnitude of grounding durations and the number of task packages executed per event. More detailed data shown in **Appendix C**. The results show that downtime remains consistently bounded across the planning horizon, with most events exhibiting low grounding durations corresponding to light workload aggregation. Events with higher downtime coincide with periods where multiple packages are intentionally aggregated, reflecting the direct relationship between aggregated maintenance workload and grounding duration. Importantly, no anomalous spikes or uncontrolled growth in downtime are observed, even toward the end of the planning horizon.

This behavior confirms that the simplified downtime formulation (Equation 3.6) remains stable under realistic task aggregation scenarios and produces operationally plausible event durations.

4.4.2. Relationship between Workload Aggregation and Downtime

The observed downtime patterns are directly linked to the bin-based workload aggregation mechanism (Equation 3.18). As shown in the event summary, increases in total man-hours (MH) and package count lead to proportional increases in downtime, indicating that grounding duration is governed by aggregated workload rather than by arbitrary scheduling decisions.

At the same time, strict enforcement of task interval constraints (Section 3.4) prevents excessive late clustering of tasks, which would otherwise inflate workload and grounding duration near the end of the planning horizon. In parallel, the penalty mechanism (Section 3.2) discourages premature execution of tasks, avoiding unnecessary workload concentration in early maintenance events. These combined mechanisms ensure that workload distribution remains balanced across time.

To further illustrate how workload aggregation is formed at the package level, Figure 4.4 visualizes the interaction between individual maintenance tasks and their assigned packages across the planning horizon. The figure links task-level selection decisions with package-level manhour accumulation and resulting downtime.

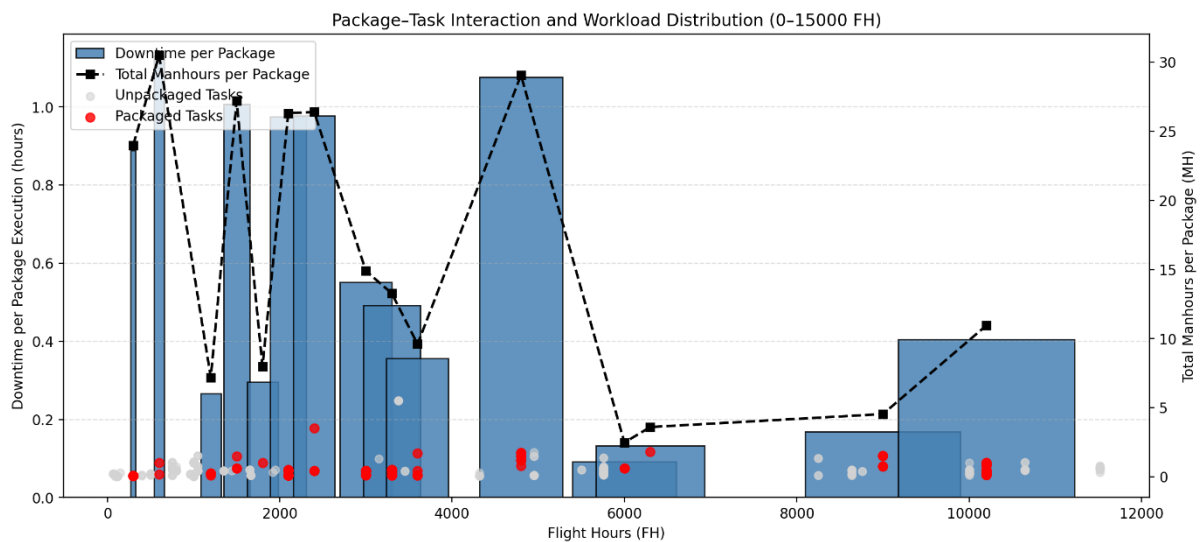


Figure 4.4 Package-Interaction and Workload Distribution

Figure 4.4 shows that package-level downtime is directly driven by the total manhours aggregated from selected tasks, confirming the structural linkage between Equations 3.16 and 3.18. Tasks selected into packages form concentrated clusters that increase workload and grounding duration, while non-selected tasks remain deferred until more favorable aggregation

conditions are met. This interaction demonstrates that downtime is not an arbitrary scheduling outcome but an emergent property of task selection and bin-based packaging decisions. The presence of un-packaged tasks further reflects the effect of line-task filtering and penalty mechanisms, which prevent inefficient task inclusion and promote balanced workload distribution across the planning horizon.

4.4.3. Stability and Operational Realism

The resulting event-level downtime structure demonstrates a high degree of stability, with predictable variations driven by intentional workload aggregation decisions rather than model artifacts. The line-task filtering logic (Section 3.2.7) further contributes to this stability by removing inefficient or non-value-adding tasks from maintenance packages, reducing unnecessary workload inflation.

As a result, the optimization model naturally converges toward a maintenance plan characterized by fewer extreme events, controlled workload variability, and grounding durations that remain consistent with engineering expectations for A-check level maintenance. This provides a robust foundation for subsequent scenario-based analysis, where changes in utilization levels alter interval compression and trigger systematic shifts in the optimal maintenance packaging strategy.

4.4.4. Analysis and Discussion for Downtime

It is important to note that the maintenance events generated in this simulation consist exclusively of A-level task packages, with no C-level packages executed within the analyzed planning horizon. This outcome does not contradict the conceptual distinction between A-check and C-check maintenance. In the proposed model, maintenance levels are defined by task classification and interval constraints rather than by the magnitude of aggregated workload or resulting downtime.

Although certain A-level maintenance events exhibit relatively higher downtime due to intentional aggregation of multiple A-packages, these events do not represent C-checks in operational terms. C-checks are characterized by fundamentally different access requirements, inspection depth, and certification processes, which are captured in the model through task-level definitions and interval thresholds. Since no C-interval constraints are reached within the simulated horizon under the given utilization conditions, C-level maintenance is optimally deferred beyond the analysis window.

This behavior reflects realistic maintenance task packaging practice, where heavy A-check events may occur without triggering C-check execution, and confirms that the

optimization model preserves the structural separation between maintenance levels while allowing flexible workload aggregation within each level.

4.5. Utilization Scenario Analysis

Utilization affects the optimization indirectly through the conversion of calendar-based MPD intervals into flight-hour equivalents; the task definitions themselves remain unchanged. This section evaluates how the MILP optimization model performs under different utilization forecast scenarios. Since utilization is the primary driver for converting MPD interval units (YE, MO, DY, FC) into flight-hour equivalents (Section 3.5), changes in utilization directly affect the frequency of event activation, workload distribution, and total cost.

The analysis here demonstrates:

1. How the optimal packaging arrangement changes when utilization fluctuates
2. How the optimal A-check and C-check intervals shift under different operational environments
3. How downtime (A-check and C-check) behaves across scenarios
4. How these observations validate the robustness and engineering realism of the MILP model

This section also shows how the outputs can be used by maintenance planners to design resilient strategies during operational disruptions—such as COVID-19 flight reductions (Section 1.2).

4.5.1. Influence of Utilization on Cost and Downtime

Utilization influences the optimization indirectly through the conversion of calendar-based MPD intervals into flight-hour equivalents, while task definitions remain unchanged. Variations in utilization therefore affect the timing and frequency of maintenance event activation rather than the intrinsic workload of individual tasks.

Figure 4.5 illustrates the response of the optimized maintenance plan to variations in aircraft utilization, showing the resulting average cost per flight hour together with the average downtime associated with A-check and C-check events. The analysis covers utilization levels from 5 to 15 FH/day, reflecting both reduced and intensified operational environments.

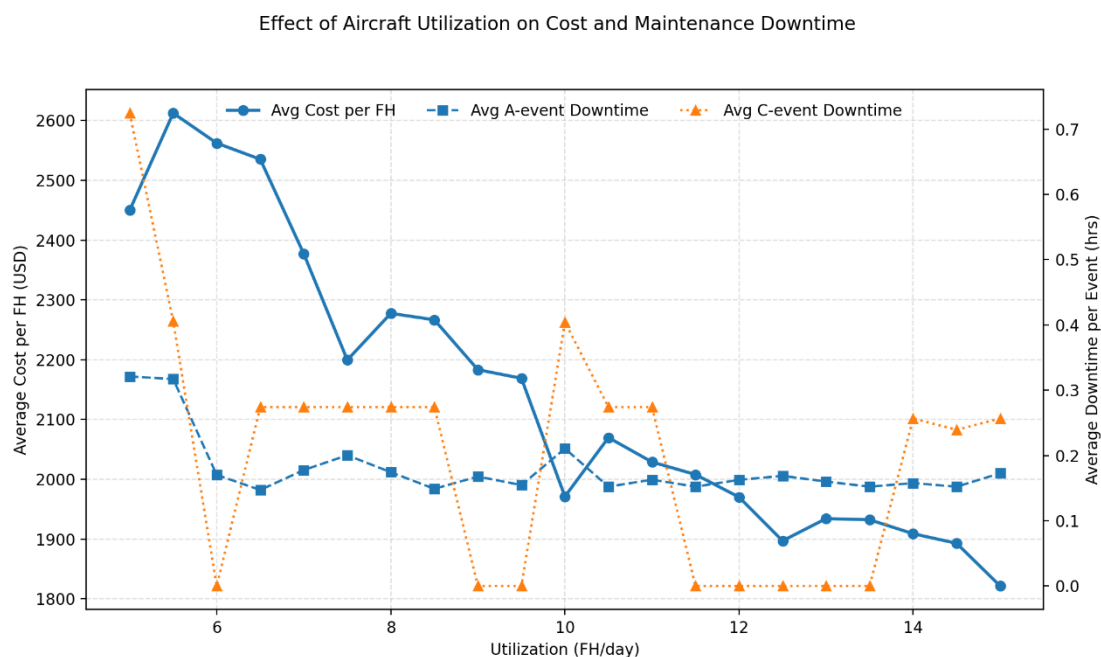


Figure 4.5 Effect of Aircraft Utilization on Average Cost and Maintenance Downtime

As shown in Figure 4.5, average cost per flight hour generally decreases with increasing utilization, confirming that higher aircraft usage improves cost efficiency by spreading fixed and event-based costs over a larger number of flight hours. A-check downtime exhibits relatively smooth and stable behavior across all scenarios, reflecting the short-interval nature of A-level maintenance and its consistent presence within the planning horizon.

In contrast, C-check downtime appears intermittently and remains zero in several utilization scenarios. This behavior is expected, as C-level maintenance is governed by long interval thresholds and is only triggered when the converted flight-hour limits are reached within the planning horizon. When a C-check is executed, its associated downtime appears as a discrete increase rather than a gradual trend, reflecting the event-based nature of heavy maintenance.

The results indicate that average cost per flight hour decreases as utilization increases, demonstrating improved cost efficiency under higher operational intensity. A-check downtime remains relatively stable across scenarios, reflecting its short-interval structure and consistent execution. C-check downtime, however, is observed only in scenarios where the converted C-interval is reached within the planning horizon, resulting in intermittent rather than continuous behavior.

4.5.2. Optimal Intervals Adjustment for Maintenance Task Packaging

To complement the trend-based visualization in Figure 4.5, Table 4.4 summarizes representative numerical results across selected utilization scenarios. The table highlights the adjustment of optimal intervals, the stability of A-check downtime, and the utilization-dependent activation of C-check events. A complementary table summarizes how the MILP model selects optimal A and C intervals under each utilization scenario:

Table 4.4 Summary of MILP Results Across Selected Utilization Scenarios

Utilization (FH/day)	Optimal A Interval (FH)	Optimal C Interval (FH)	Avg A Downtime (hrs)	Avg C Downtime (hrs)	Avg Cost per FH (USD)
5.0	300	5000	0.32	0.72	2449.8
6.0	300	10000	0.17	0.00	2562.2
7.5	300	5000	0.20	0.27	2199.5
9.0	300	10000	0.17	0.00	2183.2
12.0	300	8000	0.16	0.00	1970.1
15.0	300	5000	0.17	0.26	1821.4

Interpretation for Table 4.4 explained as follow:

1. **Avg Cost per FH** drops steadily from **\$2449.8 at 5 FH/day** to **\$2183 at 9 FH/day**, confirming that higher utilization improves cost efficiency.
2. **Avg A-Check Downtime** remains relatively stable, with a slight dip at higher utilization levels.
3. Zero average C-check downtime indicates that the optimized C-interval is not reached within the planning horizon for the corresponding utilization scenario, rather than the absence of C-level tasks.
4. Total cost increases with decrease of utilization, as expected.
Lower utilization → more frequent maintenance → higher cost.

This alignment between model behavior and operational intuition confirms that the MILP formulation reflects realistic maintenance packaging dynamics.

4.5.3. Interpretation of A-Check Stability and C-Check Sparsity

The observed disparity between A-check and C-check behavior across utilization scenarios reflects fundamental differences in their maintenance roles. A-check tasks are

characterized by short intervals and are consistently triggered regardless of utilization level, resulting in stable and continuous downtime patterns.

C-check tasks, in contrast, are governed by long interval thresholds and are therefore highly sensitive to the interaction between utilization rate and planning horizon. In several scenarios, the converted C-interval is not reached, leading to the absence of C-check execution and zero observed C-downtime. When C-checks do occur, their associated downtime appears as discrete events rather than smooth trends.

This behavior confirms that the model correctly distinguishes between frequent light maintenance and infrequent heavy maintenance, preserving engineering realism while allowing utilization-driven variability in event activation.

4.5.4. Model Validation Under Utilization Variability

The utilization scenario analysis demonstrates that the MILP model responds coherently to changes in operational intensity by adjusting maintenance timing, event frequency, and cost outcomes in a manner consistent with engineering expectations. As utilization decreases, calendar-based task intervals are converted into shorter flight-hour equivalents, leading to more frequent maintenance opportunities and increased sensitivity of cost components such as overhead and downtime.

The results show that A-check behavior remains stable across utilization scenarios, reflecting the repetitive and lightweight nature of line-maintenance tasks. In contrast, C-check execution becomes utilization-dependent, with extended optimal C-intervals leading to sparse or absent C-events within the planning horizon under certain scenarios. This behavior does not indicate a modeling deficiency, but rather confirms that the optimization appropriately defers heavy maintenance when interval limits are not reached.

These findings validate the model's suitability as a decision-support reference for anticipating maintenance behavior under utilization disruptions, such as those experienced during irregular operational periods. By quantifying how utilization shifts influence task aggregation, interval selection, and cost efficiency, the MILP model fulfills the third research objective of providing a robust and realistic framework for maintenance task packaging under variable operational conditions.

Chapter 5

Conclusion

This chapter concludes the study by summarizing the main technical achievements of the MILP-based aircraft maintenance task packaging model and discussing practical recommendations aligned with the scope defined throughout the thesis. The chapter answers the research objectives stated in Chapter 1, reflects on the methodological contributions, and outlines development steps that remain consistent with the framework established in Chapters 2–4

5.1. Conclusion

This research successfully developed and validated a high-fidelity Mixed-Integer Linear Programming (MILP) model for **aircraft maintenance task packaging**, explicitly targeting task-level optimization under strict airworthiness and operational constraints. In doing so, the study addressed a recognized gap in the existing literature, where most maintenance optimization approaches rely on heuristic or approximate methods when dealing with task-level decisions. The primary objectives of the research were fully achieved, as summarized below:

5.1.1. MILP-based mathematical model for Maintenance Task Packaging

The study achieved this objective by establishing a complete Mixed-Integer Linear Programming (MILP) formulation that captures the structural and operational characteristics of maintenance task packaging. The model incorporates:

- task-level decisions based on MPD-defined intervals (Section 3.1),
- regulatory and engineering constraints associated with maintenance operations (Equation 3.11 – 3.19),
- and economic cost components reflecting labor, material, overhead, downtime, penalty, and line execution (Equation 3.10).

The comparative analysis in Chapter 4 demonstrated that, when considering **observable cost components**—direct, overhead, downtime, and line task costs—the Baseline configuration produced lower average maintenance cost per flight hour compared to both heuristic and optimization approaches.

However, this apparent cost advantage must be interpreted with caution. The Baseline results exclude the **penalty cost formulation**, which represents the remaining usage life of tasks lost when maintenance events are triggered prematurely. Penalty costs are not directly monetizable in financial terms and therefore were not included in the observable totals.

The exclusion of penalty costs explains why the Baseline appears cheaper in observable terms, while heuristic and optimization approaches show higher values. From an analytical perspective, this highlights the **limitations of observable cost comparisons** and underscores the need for models that incorporate both measurable financial costs and non-monetized efficiency indicators.

The MILP model contributes to this discussion by providing auditable outputs, demonstrating compliance with regulatory constraints, and offering a structured framework for analyzing packaging decisions under utilization variability. While the model does not yet fully capture penalty costs in economic terms, it establishes a foundation for integrating hidden inefficiencies into future evaluations. The results confirm that optimization-based approaches can generate interpretable outputs aligned with operational logic, even if observable costs alone do not reflect their full value.

Through this analysis result and discussion, the research demonstrates that aircraft maintenance task packaging—often treated manually—can be expressed as a rigorous combinatorial optimization problem with mathematically guaranteed optimality using MILP method. The formulation provides a generalizable framework that future studies may adapt when addressing maintenance task packaging problems requiring precise interval compliance and minimize total maintenance cost.

5.1.2. A Develop MILP-based Computational Program for Maintenance Task Packaing

This objective was fulfilled by designing and implementing a complete computational pipeline that translates the mathematical model into an operational decision-support tool. The program produces transparent, engineering-ready outputs, including:

- **task-to-package assignment matrices** based directly on binary MILP decision variables (Table 4.2),
- **A- and C-check interval selections** chosen endogenously by the model (Table 4.3 and 4.4),
- **Cost decompositions** that quantify the contribution of each cost components (Table 4.3).

These outputs provide full traceability from input parameters to optimized decisions, enabling regulatory verification and engineering review. The implementation demonstrates that MILP-based task packaging can be embedded into a practical, auditable computational workflow—an important contribution for future researchers building tools for maintenance optimization or scenario evaluation.

5.1.3. Model Transparency and Validation:

The model was validated using actual MPD and operational data from A320 family aircraft, confirming that the formulation and program can reliably handle real-world task structures and parameter variation. More importantly, the model demonstrated consistent and interpretable behavior under different utilization scenarios:

- shifts in flight-hour forecasts directly influenced A- and C-check spacing (Figure 4.5 and Table 4.4),
- package clustering patterns changed in response to compressed or expanded task maturities,
- total average cost components **reduction** within the defined scope when utilization intensity increase.

This scenario responsiveness shows that the model is suitable not only as an optimization tool but also as a **reference framework** for analyzing maintenance task packaging behavior under utilization disruptions—such as those observed during irregular operational periods or demand shocks. From the results, the same methodology may be adopted when evaluating how maintenance task packaging interacts with utilization variability.

5.2. Recommendations

The recommendations presented below are strictly aligned with the assumptions, boundaries, and findings of this thesis. They focus on enhancing the precision, applicability, and interpretability of the existing single-aircraft model—without extending the scope beyond what has been discussed.

1. Refine Cost Components Using Additional Empirical Data

The linearized cost functions (downtime proxy, overhead triggers, material factor) performed well, but future work may strengthen accuracy by integrating more empirical data from repeated maintenance cycles.

The linearized cost representations developed in Chapter 3 (downtime proxy, overhead triggers, material-to-labor ratio) performed well during validation; however, their accuracy can be further improved by incorporating empirical observations from repeated maintenance events.

Future work may refine:

- downtime estimation using historical maintenance duration distributions,
- overhead cost differentiation across apron vs. hangar tasks,
- material cost scaling based on component types or zones.

- Penalty cost can better represent how unused remaining task life affect maintenance task packaging execution

These refinements do not alter the mathematical structure but allow the model to reflect a richer engineering context.

2. Expand Scenario and Sensitivity Analysis

Chapter 4 demonstrates that utilization forecasts significantly influence optimal packaging outcomes. Future research can extend this by systematically analyzing:

- small perturbations in FH/day assumptions,
- utilization disruptions of varying duration and magnitude

A more comprehensive scenario library would help maintenance engineers interpret how packaging patterns evolve under different operational conditions, reinforcing the model's relevance for packaging arrangement under uncertainty

3. Integrated Additional MPD Engineering Data

The current preprocessing pipeline already captures interval normalization, environment eligibility, and man-hour aggregation. Further enhancements may include:

- zone/access overlap information,
- enrich task data that need special environment for execution (like hangar) from historical data and expert review to make model approaching real-world situation better.

Incorporating these engineering attributes—already available in MPD datasets—would allow the MILP to recognize deeper structural relationships between tasks while staying fully consistent with the existing formulation.

4. Improve System Usability and Visualization

The decision-support interface may be expanded to include automatic compliance warnings, clearer cost-distribution dashboards, or side-by-side scenario comparisons. These enhancements do not alter the mathematics but help engineers interpret the optimization results more effectively.

References

1. European Commission, Commission Regulation (EU) No 1321/2014—Continuing Airworthiness, Official Journal of the European Union, 2014. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2014/1321/oj>. [Accessed: Dec. 3, 2025].
2. European Aviation Safety Agency, “Easy Access Rules for Continuing Airworthiness (Revision September 2025),” [Online]. Available: <https://www.easa.europa.eu/en/document-library/easy-access-rules/easy-access-rules-continuing-airworthiness>. [Accessed: Dec. 3, 2025].
3. Federal Aviation Administration, “Title 14 Code of Federal Regulations, Part 121—Operating Requirements: Domestic, Flag, and Supplemental Operations,” U.S. Department of Transportation, 2024. [Online]. Available: <https://www.ecfr.gov/current/title-14/chapter-I/subchapter-G/part-121>. [Accessed: Dec. 3, 2025].
4. Federal Aviation Administration, “Title 14 Code of Federal Regulations, § 25.571—Damage-Tolerance and Fatigue Evaluation of Structure,” U.S. Department of Transportation, 2024. [Online]. Available: <https://www.ecfr.gov/current/title-14/chapter-I/subchapter-C/part-25/section-25.571>. [Accessed: Dec. 3, 2025].
5. SKYbrary, “Maintenance Programme,” Flight Safety Foundation. [Online]. Available: <https://skybrary.aero/articles/maintenance-programme>. [Accessed: Dec. 3, 2025].
6. SKYbrary, “Aircraft Maintenance,” Flight Safety Foundation. [Online]. Available: <https://skybrary.aero/articles/aircraft-maintenance>. [Accessed: Dec. 3, 2025].
7. European Union Aviation Safety Agency (EASA), *Review of Aviation Safety Issues Arising from the COVID-19 Pandemic*, Version 2, Cologne, Germany, Apr. 2021. [Online]. Available: <https://www.easa.europa.eu/en/downloads/127172/en>. [Accessed: Dec. 3, 2025].
8. Marvella, *Optimization for Aircraft Maintenance Task Packaging for Boeing 737-800 NG*, Undergraduate Thesis, Bandung Institute of Technology, Bandung, Indonesia, 2023.
9. Federal Aviation Administration (FAA), *Advisory Circular AC 120-16G: Air Carrier Maintenance Programs*, Washington, DC, USA, Jan. 2016. [Online]. Available: https://www.faa.gov/documentLibrary/media/Advisory_Circular/AC_120-16G.pdf
10. Airbus, *Maintenance Planning Document (MPD) A320 Family*, Rev. R47, Blagnac, France, Aug. 7, 2025. [Online]. Available: [62](https://portail-</div><div data-bbox=)

- navigabilite.online/espace_inspecteurs/uploads/mpda320v1_r47_i00.pdf [Accessed: Dec. 3, 2025].
11. I. Tseremoglou, M. P. Scarpin, and V. Bertolini, “A comparative study of optimization models for condition-based maintenance scheduling of an aircraft fleet,” *Aerospace*, vol. 10, no. 2, p. 120, 2023. [Online]. Available: <https://doi.org/10.3390/aerospace10020120>. [Accessed: Dec. 3, 2025]
 12. R. A. Rahn, M. Schuch, K. Wicke, B. Sprecher, C. Dransfeld, and G. Wende, “Beyond flight operations: Assessing the environmental impact of aircraft maintenance through life cycle assessment,” *Journal of Cleaner Production*, vol. 453, Art. no. 142195, 2024. Available: <https://doi.org/10.3390/aerospace8050140>. [Accessed: Dec. 3, 2025]
 13. Oliver Wyman, *Global Fleet and MRO Market Forecast 2023–2033*, Feb. 2023. [Online]. Available: <https://www.oliverwyman.com/our-expertise/insights/2023/feb/global-fleet-and-mro-market-forecast-2023-2033.html>. [Accessed: Dec. 3, 2025].
 14. D. P. Pereira, I. L. R. Gomes, R. Melicio, and V. M. F. Mendes, “Planning of aircraft fleet maintenance teams,” *Aerospace*, vol. 8, Art. no. 140, 2021. [Online]. Available: <https://doi.org/10.3390/aerospace8050140>. [Accessed: Dec. 3, 2025].
 15. R. Saltoğlu, N. Humaira, and G. İnalan, “Aircraft scheduled airframe maintenance and downtime integrated cost model,” *Advances in Operations Research*, vol. 2016, pp. 1–12, 2016. Available: <https://doi.org/10.1155/2016/2576825>. [Accessed: Dec. 3, 2025].
 16. J. N. da M. Filho, A. C. P. de Mesquita, F. T. M. Abrahão, and G. C. Rocha, “An innovative method to solve the maintenance task allocation and packing problem,” *J. Quality Maintenance Eng.*, vol. 30, no. 1, pp. 284–305, 2024. [Online]. Available: <https://doi.org/10.1108/JQME-08-2023-0069>
 17. M. Witteman, Q. Deng, and B. F. Santos, “A bin packing approach to solve the aircraft maintenance task allocation problem,” *European Journal of Operational Research*, vol. 294, no. 1, pp. 365–376, Jan. 2021. [Online]. Available: <https://doi.org/10.1016/j.ejor.2021.01.027>
 18. H.-C. Vu, K. D. Tran, V. H. Tran, and K. P. Tran, “Predictive maintenance optimization based on genetic algorithms for future industrial systems,” in *Artificial Intelligence for Safety and Reliability Engineering*, Springer Series in Reliability Engineering, pp. 25–47, Sep. 2024. [Online]. Available: https://doi.org/10.1007/978-3-031-71495-5_3

19. J. P. Vielma, "Mixed integer linear programming formulation techniques," *SIAM Review*, vol. 57, no. 1, pp. 3–57, Jan. 2015. [Online]. Available: <https://dspace.mit.edu/handle/1721.1/96480>
20. M. Delorme, M. Iori, and S. Martello, "Bin packing and cutting stock problems: Mathematical models and exact algorithms," *European Journal of Operational Research*, vol. 255, no. 1, pp. 1–20, 2016. doi: 10.1016/j.ejor.2016.04.030. [Online]. Available: <https://doi.org/10.1016/j.ejor.2016.04.030>. [Accessed: Dec. 3, 2025].
21. K. Ali, D. Rudinskas, and V. Leonavičiūtė, "Optimizing aircraft maintenance tasks allocation using mixed integer linear programming," *Aviation*, vol. 29, no. 3, pp. 164–173, 2025. [Online]. Available: <https://doi.org/10.3846/aviation.2025.24535>
22. P. Andrade, C. Silva, B. Ribeiro, and B. F. Santos, "Aircraft maintenance check scheduling using reinforcement learning," *Aerospace*, vol. 8, no. 4, p. 113, Apr. 2021. [Online]. Available: <https://www.mdpi.com/2226-4310/8/4/113>. [Accessed: Dec. 3, 2025].
23. S. F. Pazhooh and H. S. Shemirani, "An efficient continuous-time MILP for integrated aircraft hangar scheduling and layout," *arXiv:2508.02640* [math.OC], Sep. 2025. [Online]. Available: <https://arxiv.org/pdf/2508.02640>. [Accessed: Dec. 3, 2025].
24. Aircraft Commerce, "737NG maintenance analysis & budget," *Aircraft Commerce Technical Journal*, no. 70, pp. 21–32, 2010. [Online]. Available: https://www.aircraft-commerce.com/wp-content/uploads/aircraft-commerce-docs/Aircraft%20guides/737NG-600-700-800-900/ISSUE70_737NG_MTCE.pdf. [Accessed: Dec. 3, 2025].
25. PT XYZ, "Maintenance Planning Document of Airbus A320 Family," Confidential Report, 2025. [Classified – not publicly available].
26. Gurobi Optimization, LLC, *Gurobi Optimizer Reference Manual v10.0*, Houston, TX, USA, 2024. [Online]. Available: <https://www.gurobi.com/documentation/>. [Accessed: Dec. 3, 2025].
27. A. K. Muchiri, *Maintenance Planning Optimisation for the Boeing 737 Next Generation*, MSc thesis, Delft Univ. of Technology, Delft, The Netherlands, May 2002.
28. AVA Aero Trading, "A, B, C and D checks in aviation maintenance: What they mean and the ground support equipment that makes them possible," *AVA Aero Trading Blog*, 2023. [Online]. Available: <https://www.avaet.com/post/a-b-c-and-d-checks-in-aviation-maintenance-what-they-mean-and-the-ground-support-equipment-that-m>. [Accessed: Dec. 3, 2025].

29. D. Scholz, "Aircraft Maintenance Costs," Hamburg University of Applied Sciences, 2004. [Online]. Available: <https://www.fzt.haw-hamburg.de/pers/Scholz/materialAFS/MaintenanceCosts.pdf>. [Accessed: Dec. 3, 2025].
30. Federal Aviation Administration, "Economic Values for FAA Investment and Regulatory Decisions: Section 7 – Labor Cost Factors," U.S. Department of Transportation, 2016. [Online]. Available: https://www.faa.gov/sites/faa.gov/files/regulations_policies/policy_guidance/benefit_cost/econ-value-section-7-labor-cost-factors.pdf. [Accessed: Dec. 3, 2025].
31. International Air Transport Association, "Maintenance Cost Executive Report FY2023," IATA, 2023. [Online]. Available: https://www.iata.org/contentassets/8437020db31a4717b70677d9b06b1a45/fy2023-mcx-report_public.pdf. [Accessed: Dec. 3, 2025].
32. Airbus, "A320 Family Facts and Figures, April 2025," Airbus, 2025. [Online]. Available: <https://www.airbus.com/sites/g/files/jlcpta136/files/2025-04/Airbus-A320-Family-Facts-and-Figures-April-2025.pdf>. [Accessed: Dec. 3, 2025].
33. Aviation Week, "Flight Friday: 2025 Flight Hours Soar 11% Above Pre-Pandemic Levels," Aviation Week Network, Apr. 2025. [Online]. Available: <https://aviationweek.com/air-transport/aircraft-propulsion/flight-friday-2025-flight-hours-soar-11-above-pre-pandemic-levels>. [Accessed: Dec. 3, 2025].
34. Aircraft Commerce, "A320 Family Maintenance Guide, Issue 44," *Aircraft Commerce*, 2006. [Online]. Available: <https://www.aircraft-commerce.com/wp-content/uploads/aircraft-commerce-docs/Aircraft%20guides/A320%20FAMILY/ISSUE%2044-A320%20MTCE.pdf>. [Accessed: Dec. 3, 2025].
35. Munich Airport, "EFM Munich Airport Price List 2025–2026," Munich Airport, 2025. [Online]. Available: https://www.munich-airport.com/_b/00000000000000035386020bb68cd45ff/2025-2026-efm-muc-price-list.pdf. [Accessed: Dec. 3, 2025].
36. Limble CMMS, "CMMS Costs: How Much Does a CMMS Cost?" Limble, 2024. [Online]. Available: <https://limble.com/learn/cmms/cost/>. [Accessed: Dec. 3, 2025].
37. EUROCONTROL, "Aircraft Operating Costs," ANS Performance – Standard Inputs for Economic Analyses, 2025. [Online]. Available: https://ansperformance.eu/economics/cba/standard-inputs/latest/chapters/aircraft_operating_costs.html. [Accessed: Dec. 3, 2025].

38. I. Alomar and N. Diallo, “Managing Operational Efficiency and Reducing Aircraft Downtime by Optimization of Aircraft On-Ground (AOG) Processes for Air Operator,” *Applied Sciences*, vol. 15, no. 9, p. 5129, May 2025. [Online]. Available: <https://doi.org/10.3390/app15095129>. [Accessed: Dec. 3, 2025].
39. Umbrex, “Aircraft Maintenance Planning and Downtime Reduction,” Umbrex Industry Analyses, 2023. [Online]. Available: <https://umbrex.com/resources/industry-analyses/how-to-analyze-an-airline-company/aircraft-maintenance-planning-and-downtime-reduction/>. [Accessed: Dec. 3, 2025].
40. MaintBoard, “Wrench Time: Definition, Measurement, and Best Practices,” MaintBoard, 2024. [Online]. Available: <https://maintboard.com/wrench-time>. [Accessed: Dec. 3, 2025].
41. SKYbrary, “Maintenance Shift Change/Turnover,” SKYbrary Aviation Safety, 2022. [Online]. Available: <https://skybrary.aero/articles/maintenance-shift-changeturnover>. [Accessed: Dec. 3, 2025].

Appendix A

Detail of Case Study and Raw Data

This appendix presents the structured input data used throughout the research. The datasets shown here are representative and formatted according to the requirements described in Chapter 3. These inputs serve as the foundation for preprocessing, optimization, and post-processing procedures.

A.1. MPD Raw Data

The MPD raw dataset contains mandatory maintenance tasks for the Airbus A320 case study. The data include task identifiers, engineering intervals, manpower estimates, and additional attributes relevant to packaging decisions.

TASK NUMBER	ZONE	DESCRIPTION	SKILL CODE	TASK CODE	SAMPLE THRESH OLD	SAMPLE INTERVAL	100% THRESH OLD	100% INTERVAL	SOURCE	TCI	TPS	MEN	TASK M.H.	ACCESS M.H.	PREP. M.H.
321101-01-2	731	MAIN GEAR	AF	DET			6 YE	6 YE	MRB CPCP		NO	1	0.40		4.20
321101-01-2	741	MAIN GEAR	AF	DET			6 YE	6 YE	MRB CPCP		NO	1	0.40		4.20
321141-01-2	731	MAIN LANDING GEAR COMPONENTS	AF	SDI			8 YE	8 YE	MRB CPCP	T	NO	0	N/A		38.00
321141-01-2	741	MAIN LANDING GEAR COMPONENTS	AF	SDI			8 YE	8 YE	MRB CPCP	T	NO	0	N/A		38.00
521104-01-1	831	FWD PAX/CREW DOOR INTERNAL STRUCTURE	AF	DET			12 YE	6 YE	MRB CPCP		1+2	1	0.50	0.02	0.60
521104-01-1	841	FWD PAX/CREW DOOR INTERNAL STRUCTURE	AF	DET			12 YE	6 YE	MRB CPCP		1+2	1	0.50	0.02	0.60
521304-01-1	832	AFT PAX/CREW DOOR INTERNAL	AF	DET			12 YE	6 YE	MRB CPCP		1+2	1	0.50	0.02	0.60

The MPD dataset contains **300 tasks** for the A320, each defined by a unique Task ID and associated engineering details. The interval represents the required inspection frequency in flight hours. Man-hours and minimum manpower determine labor loading inside each package. Category codes indicate whether a task is line-eligible or must be performed in a controlled environment (A-check or C-check). This dataset forms the basis for constraint generation during model formulation.

A.2. Full Data Input (Optimizer Input)

Below are data used on this research after conversion using utilization FH/day = 8, because of MPD raw data is confidential, attachment for data used will be shown from data that process by the optimizer (data after preprocessing). Here is list of data:

RowID	TaskID	IntervalFH	ManhoursRT	Men	Categories	Zone
1	200001-01-1	8640	0.5	1	1	571
2	200001-01-1	8640	0.5	1	1	671
3	200413-02-1	5760	0.31	1	1	413
4	200413-02-1	5760	0.26	1	1	414
5	200413-02-1	5760	0.39	1	1	423
6	200413-02-1	5760	0.24	1	1	424
7	200413-04-1	11520	0.75	1	1	413
8	200413-04-1	11520	0.58	1	1	414
9	200413-04-1	11520	0.78	1	1	423
10	200413-04-1	11520	0.61	1	1	424
11	200413-05-1	11520	0.55	1	1	413
12	200413-05-1	11520	0.55	1	1	414
13	200413-05-1	11520	0.55	1	1	423
14	200413-05-1	11520	0.58	1	1	424
15	200415-01-1	11520	0.62	1	1	415
16	200415-01-1	11520	0.6	1	1	425
17	200415-02-1	11520	0.62	1	1	415
18	200415-02-1	11520	0.6	1	1	425
19	200435-01-1	5760	0.15	1	1	435
20	200435-01-1	5760	0.17	1	1	436
21	200435-01-1	5760	0.15	1	1	445

22	200435-01-1	5760	0.17	1	1	446
23	200435-02-1	5760	0.52	1	1	435
24	200435-02-1	5760	0.54	1	1	436
25	200435-02-1	5760	0.52	1	1	445
26	200435-02-1	5760	0.54	1	1	446
27	200435-03-1	5760	0.52	1	1	435
28	200435-03-1	5760	0.54	1	1	436
29	200435-03-1	5760	0.52	1	1	445
30	200435-03-1	5760	0.54	1	1	446
31	200453-01-1	5760	0.19	1	1	453
32	200453-01-1	5760	0.33	1	1	454
33	200453-01-1	5760	0.19	1	1	463
34	200453-01-1	5760	0.33	1	1	464
35	200453-02-1	5760	0.56	1	1	453
36	200453-02-1	5760	0.7	1	1	454
37	200453-02-1	5760	0.56	1	1	463
38	200453-02-1	5760	0.7	1	1	464
39	200453-03-1	5760	0.58	1	1	453
40	200453-03-1	5760	0.74	1	1	454
41	200453-03-1	5760	0.58	1	1	463
42	200453-03-1	5760	0.74	1	1	464
43	200471-01-1	11520	0.32	1	1	471
45	212300-02-1	1000	0.2	1	1	200
46	212300-04-1	1200	0.25	1	1	200
47	212600-01-1	3000	0.1	1	1	210

48	212600-02-1	8640	0.1	1	1	210
49	212643-01-1	600	1.03	1	1	120
50	212643-02-1	1000	1.03	1	1	120
51	212643-06-1	600	0.19	1	1	120
53	212643-09-1	3000	0.16	1	1	120
54	212643-10-1	3000	0.16	1	1	120
55	212643-11-1	500	0.19	1	1	120
56	212742-01-1	500	0.23	1	1	127
57	215000-03-1	1800	1	1	1	190
58	233300-03-1	1200	0.25	1	1	200
59	242000-01-1	1000	0.1	1	1	210
60	242000-20-1	800	0.44	1	1	438
61	242000-20-1	800	0.44	1	1	448
62	242000-21-1	800	0.44	1	1	438
63	242000-23-1	800	0.72	1	1	448
64	242000-24-1	800	0.46	1	1	438
65	242000-24-1	800	0.46	1	1	448
66	242100-02-1	300	0.06	1	1	438
67	242100-02-1	300	0.06	1	1	448
68	256242-02-2	8760	0.4	2	1	193
69	256242-02-2	8760	0.4	2	1	194
70	282800-24-1	1000	1.1	1	1	150
71	324500-02-1	10000	0.5	1	1	148
72	324500-02-1	10000	0.5	1	1	210
73	792000-l4-2	1000	1.02	1	1	435

74	792000-I4-2	1000	1.02	1	1	445
75	534160-02-2	1650	0.52	1	1	152
76	534160-02-3	1650	0.52	1	1	152
77	242100-02-1	300	0.06	1	1	438
78	242100-02-1	300	0.06	1	1	448
79	242100-07-1	300	0.06	1	1	438
80	242100-07-1	300	0.06	1	1	448
81	242100-14-1	300	0.06	1	1	438
82	242100-14-1	300	0.06	1	1	448
83	256241-05-1	1665	0.1	1	1	221
84	256241-05-1	1665	0.1	1	1	222
85	256241-05-1	1665	0.1	1	1	261
86	256241-05-1	1665	0.1	1	1	262
87	261800-01-1	5760	0.35	1	1	200
88	261800-02-1	5760	0.13	1	1	200
89	261800-03-1	5760	0.67	1	1	200
90	261800-04-1	5760	0.47	1	1	200
91	272400-05-1	5760	0.14	1	1	325
92	272400-06-1	5760	0.16	1	1	325
93	273400-02-2	5760	1.38	2	1	334
94	273400-02-2	5760	1.38	2	1	344
95	273400-08-1	5760	0.36	1	1	334
96	273400-08-1	5760	0.36	1	1	344
97	278468-01-1	4800	1.5	1	1	520
98	278468-01-1	4800	1.5	1	1	620

99	282800-24-1	1000	1.1	1	1	150
100	292000-02-1	3600	0.12	2	1	210
101	292000-04-1	3600	1.72	2	1	190
102	292000-05-1	3600	0.12	2	1	210
103	292000-09-1	3600	0.44	1	1	190
104	324500-02-1	10000	0.5	1	1	148
105	324500-02-1	10000	0.5	1	1	210
106	335100-04-1	4320	0.3	1	1	200
107	342100-02-1	4800	0.79	1	1	120
108	383100-06-1	150	0.3	1	1	200
109	521000-01-1	112	0.02	1	1	831
110	521000-01-1	112	0.02	1	1	832
111	521000-01-1	112	0.02	1	1	841
112	521000-01-1	112	0.02	1	1	842
113	527100-02-1	4320	0.07	1	1	831
114	527100-02-1	4320	0.07	1	1	832
115	527100-02-1	4320	0.07	1	1	841
116	527100-02-1	4320	0.07	1	1	842
117	730000-C3-1	1200	0.1	1	1	210
118	753200-I1-1	1500	0.62	1	1	450
119	753200-I1-1	1500	0.62	1	1	460
120	781144-P2-1	4800	1.75	1	1	450
121	781144-P2-1	4800	1.75	1	1	460
122	792000-I4-2	1000	1.02	1	1	435
123	792000-I4-2	1000	1.02	1	1	445

124	532164-01-3	4950	1.78	1	1	200
125	533154-03-2	3375	5.52	1	1	147
126	533154-03-2	3375	5.52	1	1	148
127	533154-03-2	3375	5.5	1	1	734
128	533154-03-2	3375	5.5	1	1	744
129	534108-01-8	1950	0.5	1	1	100
130	534126-01-2	3150	1.3	1	1	100
131	534160-02-1	2100	0.52	1	1	152
132	534160-02-2	1650	0.52	1	1	152
133	534160-02-3	1650	0.52	1	1	152
134	534260-02-1	2100	0.52	1	1	152
135	545330-01-1	3450	0.42	1	1	415
136	545330-01-1	3450	0.4	1	1	425
137	572011-01-22	4800	1.2	1	1	540
138	572011-01-22	4800	1.2	1	1	640
139	572140-02-4	4950	0.1	1	1	500
140	572140-02-4	4950	0.1	1	1	540
141	572140-02-4	4950	0.1	1	1	550
142	572140-02-4	4950	0.1	1	1	560
143	572140-02-4	4950	0.1	1	1	600
144	572140-02-4	4950	0.1	1	1	640
145	572140-02-4	4950	0.1	1	1	650
146	572140-02-4	4950	0.1	1	1	660
147	572140-02-7	2100	0.1	1	1	500
148	572140-02-7	2100	0.1	1	1	540

149	572140-02-7	2100	0.1	1	1	550
150	572140-02-7	2100	0.1	1	1	560
151	572140-02-7	2100	0.1	1	1	600
152	572140-02-7	2100	0.1	1	1	640
153	572140-02-7	2100	0.1	1	1	650
154	572140-02-7	2100	0.1	1	1	660
155	572142-02-1	4950	1.47	1	1	500
156	572142-02-1	4950	1.47	1	1	600
157	572164-02-7	2100	0.1	1	1	500
158	572164-02-7	2100	0.1	1	1	600
159	575151-01-6	3000	0.44	1	1	573
160	575151-01-6	3000	0.42	1	1	574
161	575151-01-6	3000	0.44	1	1	673
162	575151-01-6	3000	0.42	1	1	674
163	575151-01-9	2100	0.44	1	1	573
164	575151-01-9	2100	0.42	1	1	574
165	575151-01-9	2100	0.44	1	1	673
166	575151-01-9	2100	0.42	1	1	674
167	712111-01-1	3300	0.12	1	1	435
168	712111-01-1	3300	0.14	1	1	436
169	712111-01-1	3300	0.12	1	1	445
170	712111-01-1	3300	0.14	1	1	446
171	712111-01-1	3300	0.36	1	1	451
172	712111-01-1	3300	0.36	1	1	461
173	712111-01-3	3300	0.52	1	1	450

174	712111-01-3	3300	0.52	1	1	460
175	712111-01-4	3300	0.52	1	1	450
176	712111-01-4	3300	0.52	1	1	460
177	ZL-171-01-1	5760	0.3	1	1	171
178	ZL-171-01-1	5760	0.31	1	1	172
179	ZL-171-01-2	5760	0.3	1	1	171
180	ZL-171-01-2	5760	0.31	1	1	172
181	ZL-171-01-3	5760	0.3	1	1	171
182	ZL-171-01-3	5760	0.33	1	1	172
183	ZL-300-02-1	5760	0.2	1	1	300
184	ZL-500-02-1	5760	0.34	1	1	500
185	ZL-500-02-1	5760	0.34	1	1	600
186	212742-01-1	500	0.23	1	1	127
187	255000-01-1	64	0.23	1	1	130
188	255000-01-1	64	0.22	1	1	150
189	255000-01-1	64	0.2	1	1	160
190	274000-02-1	750	0.1	1	1	210
191	281500-01-1	10000	0.1	1	1	210
192	282100-04-1	10000	1	2	1	210
193	291000-03-1	400	0.14	1	1	100
194	291000-13-1	500	0.22	1	1	100
195	291000-16-1	500	0.22	1	1	100
196	291200-03-1	400	0.07	1	1	100
197	301151-01-1	5500	0.5	1	1	522
198	301151-01-1	5500	0.5	1	1	622

199	323000-01-1	1000	0.02	1	1	711
200	323000-01-1	1000	0.02	1	1	731
201	323000-01-1	1000	0.02	1	1	741
202	324411-01-2	1000	0.04	1	1	148
203	324411-01-2	1000	0.02	1	1	210
204	525111-06-2	1440	0.42	1	1	220
205	730000-l1-1	750	0.1	1	1	210
206	730000-L1-1	500	0.1	1	1	210
207	730000-W1-1	750	0.1	1	1	210
208	731100-W1-1	750	0.74	1	1	450
209	731100-W1-1	750	0.74	1	1	460
210	732100-W1-1	750	1.07	1	1	450
211	732100-W1-1	750	1.07	1	1	460
212	783200-L3-1	10000	0.25	1	1	451
213	783200-L3-1	10000	0.41	1	1	452
214	783200-L3-1	10000	0.25	1	1	461
215	783200-L3-1	10000	0.41	1	1	462
216	783200-L4-1	10000	0.22	1	1	451
217	783200-L4-1	10000	0.24	1	1	452
218	783200-L4-1	10000	0.22	1	1	461
219	783200-L4-1	10000	0.24	1	1	462
220	534160-02-6	1050	0.52	1	1	152
221	575151-01-11	1050	0.44	1	1	573
222	575151-01-11	1050	0.42	1	1	574
223	575151-01-11	1050	0.44	1	1	673

224	575151-01-11	1050	0.42	1	1	674
225	575151-01-14	1050	0.44	1	1	573
226	575151-01-14	1050	0.42	1	1	574
227	575151-01-14	1050	0.44	1	1	673
228	575151-01-14	1050	0.42	1	1	674
229	575151-01-8	1350	0.44	1	1	573
230	575151-01-8	1350	0.42	1	1	574
231	575151-01-8	1350	0.44	1	1	673
232	575151-01-8	1350	0.42	1	1	674
233	282800-22-1	6300	1.8	1	1	150
234	292000-01-1	1500	1.48	2	1	190
235	753200-11-2	6000	0.62	1	1	450
236	753200-11-2	6000	0.62	1	1	460
237	571136-01-6	10650	1.05	1	1	141
238	571136-01-6	10650	1.05	1	1	142
239	572008-01-5	10650	0.54	1	1	574
240	572008-01-5	10650	0.54	1	1	674
241	572032-01-2	10200	0.2	1	1	500
242	572032-01-2	10200	0.2	1	1	600
243	572042-01-10	8250	1.35	1	1	471
244	572042-01-10	8250	1.35	1	1	481
245	572114-01-11	10200	0.47	1	1	540
246	572114-01-11	10200	0.47	1	1	640
247	572114-01-12	10200	0.47	1	1	540
248	572114-01-12	10200	0.47	1	1	640

249	572114-01-8	10650	0.47	1	1	540
250	572114-01-8	10650	0.47	1	1	640
251	572114-01-9	10650	0.47	1	1	540
252	572114-01-9	10650	0.47	1	1	640
253	572115-01-13	10200	1	1	1	540
254	572115-01-13	10200	0.38	1	1	550
255	572115-01-13	10200	0.16	1	1	560
256	572115-01-13	10200	0.85	1	1	640
257	572115-01-13	10200	0.34	1	1	650
258	572115-01-13	10200	0.16	1	1	660
259	572115-01-8	10200	1	1	1	540
260	572115-01-8	10200	0.38	1	1	550
261	572115-01-8	10200	0.16	1	1	560
262	572115-01-8	10200	0.85	1	1	640
263	572115-01-8	10200	0.34	1	1	650
264	572115-01-8	10200	0.16	1	1	660
265	572115-01-9	10200	1	1	1	540
266	572115-01-9	10200	0.38	1	1	550
267	572115-01-9	10200	0.16	1	1	560
268	572115-01-9	10200	0.85	1	1	640
269	572115-01-9	10200	0.34	1	1	650
270	572115-01-9	10200	0.16	1	1	660
271	572137-01-2	8250	0.12	1	1	500
272	572137-01-2	8250	0.12	1	1	600
273	725403-01-2	9000	0.76	1	1	455

274	725403-01-2	9000	0.76	1	1	465
275	725403-02-2	9000	1.51	1	1	455
276	725403-02-2	9000	1.51	1	1	465
277	ZL-129-01-1	1920	0.32	1	1	129
278	711000-C7-1	8640	0.48	1	1	440
279	291000-06-1	8640	0.1	1	1	210
280	291000-20-1	8640	0.25	1	1	210
281	711000-C7-1	8640	0.48	1	1	430
282	783201-I9-1	8640	0.21	1	1	430
283	783201-I9-1	8640	0.21	1	1	440
284	783200-W6-1	8640	0.47	1	1	450
285	783200-W6-1	8640	0.47	1	1	460
286	321000-01-1	750	0.52	2	1	731
287	321000-01-1	750	0.52	2	1	741
288	322000-01-1	750	0.54	2	1	710
289	353000-01-1	960	0.17	1	1	200
290	353000-05-1	960	0.2	1	1	200
291	781112-C1-1	3000	0.42	1	1	450
292	781112-C1-1	3000	0.42	1	1	460
293	725403-01-1	1050	0.76	1	1	455
294	725403-01-1	1050	0.76	1	1	465
295	725403-02-1	1050	1.51	1	1	455
296	725403-02-1	1050	1.51	1	1	465
297	242400-01-2	120	0.02	1	1	210
298	242400-01-3	90	0.02	1	1	210

299	242800-01-3	90	0.1	1	1	210
300	255000-01-2	80	0.23	1	1	130
301	274451-11-1	2400	3.52	1	1	310
302	275446-01-1	2400	0.44	1	1	530
303	275446-01-1	2400	0.44	1	1	630

A.3. Parameter Data (JSON file)

The optimization model requires a parameter JSON file containing baseline cost and configuration values. This file ensures that all solver runs use consistent assumptions.

Table A.2 — Cost and Model Parameters (JSON Structure)

Parameter Key	Value	Description
"labor_rate"	60	USD per man-hour
"material_factor"	0.33	Fraction of labor cost
"overhead_A"	40000	Fixed cost per A-check event
"overhead_C"	300000	Fixed cost per C-check event
"man_availability"	10	Manpower availability
"shift_hours"	9	Per shift hours work
"revenue_loss_per_hr"	14000	Lost revenue per downtime hour
"planning_horizon"	15000	Flight hours
"A_range"	[400,1000, increment =10]	Candidate A-intervals
"C_range"	[4000, 15000, increment =100]	Candidate C-intervals

This parameter file is loaded by the frontend module and injected into both the baseline heuristic and MILP optimizer. The values mirror the cost formulations in Chapter 3 and ensure consistent comparison across all scenarios tested in Chapter 4 and 5. All monetary assumptions are based on historical averages and are used for relative performance evaluation, not absolute cost forecasting.

Appendix B

Key Code Snippets and Codebase Link

To ensure reproducibility, transparency, and auditability, this appendix presents selected excerpts from the software modules developed in this study. The complete codebase is maintained separately, but only representative snippets are included here to illustrate the logic behind each stage of the workflow. These snippets are directly aligned with the methodological framework described in Chapter 3 and provide a traceable link between the mathematical formulation and its implementation.

The appendix is organized into three subsections:

- **B.1 Preprocessing Module** – shows how raw Maintenance Planning Document (MPD) data is cleaned, normalized, and converted into standardized optimizer inputs (Section 3.5).
- **B.2 Baseline Heuristic Model** – demonstrates the enumeration of candidate A/C combinations and the greedy heuristic for line-task unpackaging, reflecting traditional block and equalized philosophies (Section 3.6).
- **B.3 MILP Optimizer** – provides code excerpts for the objective function, constraints, and output design, corresponding to Sections 3.2–3.4, and ensuring full traceability of the optimization formulation.

Together, these subsections provide a clear audit trail from raw data preprocessing through heuristic benchmarking to MILP optimization, supporting the comparative analysis presented in Chapter 4.

The conceptual design of the maintenance task packaging framework, the problem formulation, modelling assumptions, system architecture, and interpretation of results presented in this thesis were developed solely by the author.

As part of the software implementation process, artificial intelligence-based coding assistance tools were used selectively to support code generation based on the author's explicit design specifications and optimization logic. These tools were employed as productivity aids during implementation and did not determine the problem formulation, optimization strategy, or analytical conclusions.

All generated source code was reviewed, adapted, and validated by the author to ensure consistency with the formulated Mixed-Integer Linear Programming (MILP) model and the engineering constraints described in this thesis. Responsibility for the correctness, assumptions, and limitations of the models and results remains entirely with the author.

The complete source code is publicly available and documented at the following link:

[sgeryandika/UndergraduatedThesis: Maintenance Packaging Problem](https://github.com/sgeryandika/UndergraduatedThesis: Maintenance Packaging Problem)

B.1. Preprocessing Module

This subsection documents the preprocessing pipeline (mtp_preprocessing.py) used to transform raw Maintenance Planning Document (MPD) data into a structured optimizer input file. As described in Section 3.5, preprocessing ensures that task attributes are normalized, intervals are converted into flight-hour equivalents, and categories are flagged for line or base maintenance. The outputs of this module form the foundation for both the heuristic baseline (Appendix B.2) and the MILP optimizer (Appendix B.3)

- Column Normalization and Parsing

The first step is to rename MPD columns into standardized names and parse threshold values into flight-hour equivalents. This ensures consistency across different MPD formats.

```
rename_map = {
    "TASK\\nNUMBER": "TaskID",
    "ZONE": "Zone",
    "TASK CODE": "TaskType",
    "TASK\\nM.H.": "TaskMH",
    "ACCESS\\nM.H.": "AccessMH",
    "PREP.\\nM.H.": "PrepMH",
    "MEN": "Men",
    "100%\\nINTERVAL": "Interval100",
    "THRESHOLD": "Threshold100",
    "INTERVAL": "Interval100",
}

def parse_threshold_with_flags(val):
    if pd.isna(val) or str(val).strip() == "":
        return {"fh": 0, "note": 0, "act": 0, "ce": 0}
    v = str(val).upper().replace("\\n", " ")
    matches = re.findall(r"(\d+)\s*(YE|MO|DY|FC|FH)", v)
    fh_values = []
    for num_s, unit in matches:
        num = int(num_s)
        if unit == "YE": fh_values.append(num * 365 * FH_per_day)
        elif unit == "MO": fh_values.append(num * FH_per_mo)
        elif unit == "DY": fh_values.append(num * FH_per_day)
        elif unit == "FC": fh_values.append(num * FH_per_fc)
        elif unit == "FH": fh_values.append(num)
    fh_val = int(min(fh_values)) if fh_values else 0
```

```

    return {"fh": fh_val, "note": ("NOTE" in v), "act": ("ACT" in v), "ce":
("CE" in v)}

```

This snippet shows how intervals expressed in calendar units (years, months, days, cycles) are converted into flight-hour equivalents, ensuring comparability across tasks.

- Category Flagging

Tasks are flagged as line-eligible (Category = 1) or base/hangar tasks (Category = 0). This classification is critical for both heuristic and MILP models (cf. Section 2.2.3).

```

def map_categories(v):
    if pd.isna(v): return 1
    s = str(v).strip().upper()
    if "LINE" in s: return 1
    if "BASE" in s or "HANGAR" in s: return 0
    return 1
df["Categories"] = df["Check_Categories"].apply(map_categories)

```

This snippet reflects the operational boundary between line and base maintenance, ensuring tasks are assigned to feasible environments.

- Interval Conversion

The canonical IntervalFH column is created by prioritizing parsed flight-hour values. This ensures that each task has a deterministic interval for packaging.

```

if "Interval100_FH" in df.columns:
    df["IntervalFH"] = pd.to_numeric(df["Interval100_FH"],
errors="coerce").fillna(0).astype(float)
elif "SampleInterval_FH" in df.columns:
    df["IntervalFH"] = pd.to_numeric(df["SampleInterval_FH"],
errors="coerce").fillna(0).astype(float)
else:
    df["IntervalFH"] = 0.0

```

- Export to Optimizer

Finally, the cleaned dataset is exported into two files:

- **Full cleaned CSV** (for audit and traceability)
- optimizer_input.csv (filtered subset containing only essential fields for optimization)

python

```

cols_for_optimizer =
["RowID", "TaskID", "IntervalFH", "ManhoursRT", "Men", "Categories", "Zone"]
df_opt = df_final[cols_for_optimizer]
optimizer_path = out_dir / "optimizer_input.csv"
df_opt.to_csv(optimizer_path, index=False)

```

This ensures reproducibility and provides a standardized input format for both heuristic and MILP solvers.

B.2. Baseline Heuristic Model

This subsection documents the baseline heuristic program (`mtp_heuristic_baseline.py`) used for comparative validation (see Section 3.6). The heuristic model emulates traditional block and equalized philosophies by enumerating candidate A/C interval combinations and applying simple rules for task assignment. It also incorporates a greedy heuristic for line-task unpacking. The excerpts below highlight the essential logic without reproducing the full source code.

- Candidate Enumeration and Task Mapping

The heuristic begins by generating candidate bins across the planning horizon and mapping tasks according to block or equalized rules (Section 2.2).

```
def map_task_occurrences_to_bins(tasks_df, A, C, H, mode='block'):
    bins = generate_bins(H, A, C)
    records = []
    for _, row in tasks_df.iterrows():
        interval = float(row['interval_fh']); mh = float(row['manhours'])
        if interval < A or interval <= 0: continue
        if interval < C:
            chosen_bin = max([b for b in bins if b <= interval and b % A ==
0])
            records.append((row['task_id'], interval, chosen_bin, mh, 'A'))
        else:
            if mode == 'block':
                chosen_bin = max([b for b in bins if b <= interval and b % C
== 0])
                records.append((row['task_id'], interval, chosen_bin, mh,
'C'))
            else:
                per_mh = mh / (C // A)
                for a_bin in range(A, C+1, A):
                    records.append((row['task_id'], interval, a_bin, per_mh,
'C-eq'))
    return pd.DataFrame(records, columns=['task_id', 'due', 'bin', 'mh', 'level'])
```

This function shows how tasks are mapped to candidate bins

- Greedy Heuristic for Line Task Unpacking

After initial assignment, the model applies a greedy heuristic to test whether line-eligible tasks (Category 1) should be removed from packages. If unpacking reduces total cost, the task is left as a line task (Section 2.2.3).


```
def greedy_unpackage_line_tasks(locked_df, tasks_df, params, A, C, H):
    base_costs = compute_costs_from_locked(locked_df, tasks_df, params, A, C,
H)
    base_total = base_costs['total_with_opportunity']
    for _, row in tasks_df[tasks_df['category']==1].iterrows():
        tid = str(row['task_id'])
        test_locked = locked_df[locked_df['task_id'] != tid]
        test_costs = compute_costs_from_locked(test_locked, tasks_df, params,
A, C, H)
        if test_costs['total_with_opportunity'] < base_total:
            locked_df = test_locked; base_total =
test_costs['total_with_opportunity']
    return locked_df
```

This snippet illustrates the greedy ununpacking logic, reflecting operational pragmatism in line/base contexts.

- Cost Computation Framework

The heuristic cost function mirrors the MILP objective decomposition (Section 3.2), ensuring comparability between heuristic and optimization results.

```
def compute_costs_from_locked(locked, tasks_df, params, A, C, H):
    total_mh = locked['mh'].sum()
    direct = params.labor_rate * total_mh
    material = params.material_fraction * params.labor_rate * total_mh
    overhead_total = sum(params.ohC if b % C == 0 else params.ohA for b in
locked['bin'].unique())
    downtime_total = sum((mh / men) * params.revenue_per_hour for mh, men in
zip(locked['mh'], tasks_df['men']))
    total_incurred = direct + material + overhead_total + downtime_total
    return {'total_mh': total_mh, 'direct': direct, 'material': material,
'overhead': overhead_total, 'downtime': downtime_total,
'total_incurred': total_incurred}
```

This ensures that both heuristic and MILP models report costs using the same decomposition.

B.3. MILP Optimizer

This subsection documents the Mixed-Integer Linear Programming (MILP) optimizer (mtp_milp_optimizer.py) developed for this study. As described in Section 3.4, the MILP formulation guarantees global optimality under discrete task-level constraints, with endogenous selection of A- and C-intervals. The excerpts below highlight the essential logic: the objective function, the constraint rules, and the output design.

- Objective Function

The MILP objective integrates all cost components defined in Section 3.2: direct labor, material, overhead, downtime, opportunity penalties, and line task costs. This ensures parity with the heuristic baseline while providing mathematically guaranteed optimality.

```
def objective_rule(m):
    # Overhead classification via zA/zC
    overhead = sum(params.ohA * m.zA[t] + params.ohC * m.zC[t] for t in
m.BINS)

    # Direct + material costs
    total_mh_expr = sum(allowed_mh.get((i, bin_to_fh[t]), 0.0) * m.x[i, t]
                        for i in m.TASKS for t in m.BINS)
    direct_cost = params.labor_rate * total_mh_expr
    material_cost = params.material_fraction * direct_lab

    # Downtime proxy per bin
    downtime_proxy = sum((m.mh_bin[t] / avg_men) * params.revenue_per_hour for
t in m.BINS)

    # Opportunity proxy
    opp_proxy = sum(params.revenue_per_hour *
                    occ_proxy.get((i, bin_to_fh[t]), 0) *
                    opp_slack.get((i, bin_to_fh[t]), 0.0) *
                    m.x[i, t]
                    for i in m.TASKS for t in m.BINS)

    # Line task proxy
    line_proxy = 0
    for i in m.TASKS:
        if cat_map[i] == 1:
            row_mh = mh_map.get(i, 0.0)
            row_int = interval_map.get(i, 1.0)
            men_eff = men_map.get(i, 0.0) if men_map.get(i, 0.0) > 0 else 1.0
            occ = math.ceil(H / row_int) if row_int > 0 else 0
            occ_lab = params.phi * params.labor_rate * row_mh
            occ_dt = (row_mh / men_eff) * params.revenue_per_hour
            occ_oh = params.ohL
            line_proxy += occ * (occ_lab + occ_dt + occ_oh) * m.y_line[i]

    return overhead + direct_cost + material_cost + downtime_proxy + opp_proxy
+ line_proxy
```

This function demonstrates how the MILP integrates all cost drivers into a single optimization objective, consistent with Section 3.2.

- Constraints (Correspond to Section 3.4)

The MILP enforces a comprehensive set of constraints to guarantee compliance and feasibility. Each constraint listed in Section 3.4 is represented here with its code counterpart.

- Policy Selection (3.4.1)

Ensure exactly one A-interval and one C-interval are chosen:

```
model.chooseA = pyo.Constraint(expr=sum(model.sA[a] for a in model.ACANDS) == 1)
model.chooseC = pyo.Constraint(expr=sum(model.sC[c] for c in model.CCANDS) == 1)
```

- Bin Classification

Classify bins as A or C depending on divisibility under chosen intervals:

```
def isAbin_rule(m, t): return m.isAbin[t] == sum(m.sA[a] for a in m.ACANDS if bin_to_fh[t] % a == 0)
def isCbin_rule(m, t): return m.isCbin[t] == sum(m.sC[c] for c in m.CCANDS if bin_to_fh[t] % c == 0)
```

- Overhead Trigger Linearization

Linearize overhead assignment via McCormick constraints:

```
def zA1(m, t): return m.zA[t] <= m.isAbin[t]
def zA2(m, t): return m.zA[t] <= m.v[t]
def zA3(m, t): return m.zA[t] >= m.isAbin[t] + m.v[t] - 1
```

- Bin Compatibility Constraints

Ensure bins are opened only if classified as A or C:

```
def open_compat_rule(m, t): return m.v[t] <= m.isAbin[t] + m.isCbin[t]
```

- Link assignment to opened bin

Tasks can only be assigned to bins that are opened:

```
def link_rule(m, i, t): return m.x[i, t] <= m.v[t]
```

- Packaging and Line Decisions

Each task is either packaged or left as line:

```
def assign_rule(m, i):
    allowed = allowed_bins_by_task.get(i, [])
    if len(allowed) == 0:
        return m.y_line[i] == 1
    return sum(m.x[i, t] for t in allowed) + m.y_line[i] == 1
```

- Task Due Date

Tasks may only be assigned to bins $\leq$ their interval: (*enforced via allowed_pairs construction in code*)

- Bin Workload Aggregation

Aggregate manhours per bin for downtime proxy:

```
def mh_bin_rule(m, t):
    fh = bin_to_fh[t]
```

```
return m.mh_bin[t] == sum(allowed_mh.get((i, fh), 0.0) * m.x[i, t] for i
in m.TASKS)
```

- Line Eligibility Constraints

Only Category 1 tasks can be assigned as line:

```
def line_restriction_rule(m, i):
    if cat_map[i] != 1:
        return m.y_line[i] == 0
    return pyo.Constraint.Skip
```

- Output Design

After solving, the MILP produces locked schedules, package summaries, and cost decompositions. These outputs are directly comparable to heuristic results (Appendix B.2) and feed into Chapter 4’s analysis.

```
# Build locked schedule by propagating chosen assignments
locked_records = []
for (tid, fh, mh_val) in chosen_pairs:
    multiples = np.arange(fh, H + 1, fh, dtype=int)
    for d in multiples:
        lvl = 'C' if (d % chosen_C == 0) else 'A'
        locked_records.append((str(tid), int(d), fh, float(mh_val),
                                'MILP-assigned', lvl, 0.0))
locked_df = pd.DataFrame(locked_records,
                        columns=['task_id', 'due', 'bin', 'mh', 'method', 'check_1
                                evel', 'slack'])

# Exact cost recomputation for audit
exact_costs = compute_costs_from_locked(locked_df, tasks_df, params, chosen_A,
                                         chosen_C, H)
```

This snippet shows how the MILP produces auditable outputs: locked task schedules, package summaries, and exact cost decompositions, ensuring traceability and comparability with heuristic results

B.4. Output Design and Visualization

This subsection documents the output design and visualization routines that are integrated into both the baseline heuristic model (Appendix B.2) and the MILP optimizer (Appendix B.3). While the optimization logic differs, both programs share a common philosophy: outputs must be **traceable, auditable, and interpretable** by engineering stakeholders.

- Task-to-Bin Matrix

The primary validation artifact is the **task-to-bin matrix**, which shows how each task is assigned to packages across the planning horizon. This corresponds to Section 3.3 (Output Design Rationale) and Section 4.2.2 (Optimal Task Assignment Plan).

```
def build_task_bin_matrix(locked_df, tasks_df, A, C, horizon):
    bins = sorted(set(list(range(A, horizon+1, A)) + list(range(C, horizon+1, C))))
    matrix = tasks_df[['task_code', 'interval_fh']].copy()
    matrix.rename(columns={'task_code': 'Task ID', 'interval_fh': 'Interval (FH)'}, inplace=True)
    for idx, b in enumerate(bins, start=1):
        col_name = f"Package {idx} ({b}FH)"
        matrix[col_name] = 0
    for _, row in locked_df.iterrows():
        tid = row['task_id']; b = row['bin']
        col_name = f"Package {bins.index(b)+1} ({b}FH)"
        matrix.loc[matrix['Task ID']==tid, col_name] = 1
    return matrix
```

This produces a tabular output where each row corresponds to a task and each column to a package event, with binary indicators (1/0) showing assignment.

The original programs generate internal validation sheets and package summaries for solver logic. For thesis reporting, however, a **presentation-friendly adaptation** was added to produce binary assignment tables in the format shown in Section 4.2.2. This adaptation does not alter solver logic; it is a separate reporting utility that consumes program outputs (locked_df, tasks_df) and formats them for readability.

```
def build_task_bin_matrix(locked_df, tasks_df, A, C, horizon):
    """
    Adapted reporting function for thesis outputs.
    Produces binary assignment matrix:
    Task ID | Task Description | Interval (FH) | Package 1 (400FH) | Package 2 (800FH) | ...
    """
    # Start with basic task info
    matrix = tasks_df[['task_code', 'interval_fh']].copy()
    matrix.rename(columns={'task_code': 'Task ID', 'interval_fh': 'Interval (FH)'}, inplace=True)

    # Add description if available
    if 'task_desc' in tasks_df.columns:
        matrix['Task Description'] = tasks_df['task_desc']
    else:
        matrix['Task Description'] = matrix['Task ID']

    # Define package bins (multiples of A and C up to horizon)
```

```

    bins = sorted(set(list(range(A, horizon+1, A)) + list(range(C, horizon+1,
C))))
    for idx, b in enumerate(bins, start=1):
        col_name = f"Package {idx} ({b}FH)"
        matrix[col_name] = 0

    # Fill assignment: mark 1 if task is scheduled in that bin
    for _, row in locked_df.iterrows():
        tid = row['task_id']; b = row['bin']
        col_name = f"Package {bins.index(b)+1} ({b}FH)"
        matrix.loc[matrix['Task ID']==tid, col_name] = 1

    return matrix

```

- Package Workload Histogram

Both models generate workload histograms to visualize total manhours per package. This supports Section 4.2.3 (Resource Utilization Analysis) and Section 4.2.4 (Downtime Distribution).

```

def plot_package_histogram(pkg_summary, A, C, viz_horizon):
    fig, ax = plt.subplots(figsize=(12,6))
    for _, row in pkg_summary.iterrows():
        color = 'orange' if row['assigned_level']=='C' else 'steelblue'
        ax.bar(row['assigned_bin'], row['total_mh'], color=color, alpha=0.7)
    ax.set_xlabel('Flight Hours (FH)')
    ax.set_ylabel('Total Manhours')
    plt.show()

```

Bars represent aggregated workload per package, color-coded by A- or C-check classification.

- Occurrence Histogram

The occurrence histogram shows the distribution of scheduled events across the horizon, highlighting peaks and downtime clustering.

```

def plot_occurrence_histogram(locked, C, viz_horizon):
    df = locked[locked['due'] <= viz_horizon].copy()
    occ = df.groupby(['due', 'bin'],
as_index=False)['mh'].sum().sort_values('due')
    plt.figure(figsize=(14,6))
    colors = ['orange' if (row['bin'] % C == 0) else 'steelblue' for _, row in
occ.iterrows()]
    plt.bar(occ['due'], occ['mh'], color=colors,
width=max(1,int(viz_horizon/200)), edgecolor='k')
    plt.xlabel('Flight Hours (FH)')
    plt.ylabel('Total Manhours scheduled (MH)')
    plt.show()

```



This visualization supports downtime analysis and helps identify workload peaks across the planning horizon.

- Integration in Program Codes

It is important to note that these visualization routines are **not standalone scripts** but are integrated directly into the heuristic and MILP program codes. This ensures that every optimization run automatically produces:

- **Validation sheets** (task-to-bin matrices)
- **Package summaries** (aggregated workload per event)
- **Cost decomposition reports**
- **Visualizations** (histograms and Gantt-style charts)

This integration guarantees reproducibility and provides immediate interpretability of results, aligning with the methodological emphasis on auditability (Section 3.3.3).

Appendix C

Detailed Output Visualization

This appendix contains more detailed information regarding result discuss on Chapter 4 and supplementary to discussion regarding optimization result in Chapter 4.

C.1. Package Summary Data Result

assigned_level	assigned_bin	bin_start_fh	bin_end_fh	total_mh	task_count
A	300	1	300	24	8
A	600	301	600	30.5	2
A	1200	901	1200	7.2	3
A	1500	1201	1500	27.2	3
A	1800	1501	1800	8	1
A	2100	1801	2100	26.32	16
A	2400	2101	2400	26.4	3
A	3000	2701	3000	14.9	9
A	3300	3001	3300	13.28	10
A	3600	3301	3600	9.6	4
A	4800	4501	4800	29.07	7
A	6000	5701	6000	2.48	2
A	6300	6001	6300	3.6	1
A	9000	8701	9000	4.54	4
A	10200	9901	10200	10.95	24

C.2. Event Summary Timeline

due_fh	packages_executed	package_count	total_mh	downtime_hours
300	1A	1	0.48	0.017777778
600	1A + 2A	2	1.7	0.062962963
900	1A	1	0.48	0.017777778
1200	4A + 1A + 2A	3	2.3	0.085185185
1500	5A + 1A	2	3.2	0.118518519
1800	6A + 1A + 2A	3	2.7	0.1
2100	7A + 1A	2	4.24	0.157037037
2400	4A + 8A + 1A + 2A	4	6.7	0.248148148
2700	1A	1	0.48	0.017777778
3000	5A + 10A + 1A + 2A	4	7.4	0.274074074
3300	1A + 11A	2	3.8	0.140740741
3600	4A + 6A + 1A + 12A + 2A	5	5.7	0.211111111
3900	1A	1	0.48	0.017777778
4200	7A + 1A + 2A	3	5.46	0.202222222
4500	5A + 1A	2	3.2	0.118518519
4800	4A + 8A + 1A + 16A + 2A	5	16.39	0.607037037
5100	1A	1	0.48	0.017777778
5400	6A + 1A + 2A	3	2.7	0.1

5700	1A	1	0.48	0.017777778
6000	4A + 5A + 10A + 1A + 20A + 2A	6	9.24	0.342222222
6300	7A + 1A + 21A	3	6.04	0.223703704
6600	1A + 11A + 2A	3	5.02	0.185925926
6900	1A	1	0.48	0.017777778
7200	4A + 6A + 8A + 1A + 12A + 2A	6	10.1	0.374074074
7500	5A + 1A	2	3.2	0.118518519
7800	1A + 2A	2	1.7	0.062962963
8100	1A	1	0.48	0.017777778
8400	4A + 7A + 1A + 2A	4	6.06	0.224444444
8700	2A	1	0.48	0.017777778
9000	5A + 6A + 10A + 1A + 2A + 30A	6	12.94	0.479259259
9300	1A	1	0.48	0.017777778
9600	4A + 8A + 1A + 16A + 2A	5	16.39	0.607037037
9900	1A + 11A	2	3.8	0.140740741
10200	24A + 1A + 2A	3	12.65	0.468518519
10500	5A + 7A + 1A	3	6.96	0.257777778
10800	4A + 6A + 1A + 12A + 2A	5	5.7	0.211111111
11100	1A	1	0.48	0.017777778
11400	1A + 2A	2	1.7	0.062962963
11700	1A	1	0.48	0.017777778
12000	4A + 5A + 6A + 10A + 1A + 20A + 2A	7	13.64	0.505185185
12300	1A	1	0.48	0.017777778
12600	6A + 7A + 1A + 2A + 21A	5	8.26	0.305925926
12900	1A	1	0.48	0.017777778
13200	4A + 1A + 11A + 2A	4	5.62	0.208148148
13500	5A + 1A	2	3.2	0.118518519
13800	1A + 2A	2	1.7	0.062962963
14100	1A	1	0.48	0.017777778
14400	4A + 6A + 8A + 1A + 12A + 16A + 2A	7	19.79	0.732962963
14700	7A + 1A	2	4.24	0.157037037
15000	5A + 10A + 1A + 2A	4	7.4	0.274074074

C.3. Task to Package Matrix

task_code	zone	Category	manhours	300	600	1200	1500	1800	2100	2400	3000	3300	3600	4800	6000	6300
292000-02-1	210	1	0.12	0	0	0	0	0	0	0	0	0	1	0	0	0
292000-04-1	190	1	1.72	0	0	0	0	0	0	0	0	0	1	0	0	0
292000-05-1	210	1	0.12	0	0	0	0	0	0	0	0	0	1	0	0	0
292000-09-1	190	1	0.44	0	0	0	0	0	0	0	0	0	1	0	0	0
342100-02-1	120	1	0.79	0	0	0	0	0	0	0	0	0	0	1	0	0
730000-C3-1	210	1	0.1	0	0	1	0	0	0	0	0	0	0	0	0	0
753200-l1-1	450	1	0.62	0	0	0	1	0	0	0	0	0	0	0	0	0
753200-l1-1	460	1	0.62	0	0	0	1	0	0	0	0	0	0	0	0	0
781144-P2-1	450	1	1.75	0	0	0	0	0	0	0	0	0	0	1	0	0
781144-P2-1	460	1	1.75	0	0	0	0	0	0	0	0	0	0	1	0	0
534160-02-1	152	1	0.52	0	0	0	0	0	1	0	0	0	0	0	0	0
534260-02-1	152	1	0.52	0	0	0	0	0	1	0	0	0	0	0	0	0
572011-01-22	540	1	1.2	0	0	0	0	0	0	0	0	0	0	1	0	0
572011-01-22	640	1	1.2	0	0	0	0	0	0	0	0	0	0	1	0	0
572140-02-7	500	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0

572140-02-7	540	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572140-02-7	550	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572140-02-7	560	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572140-02-7	600	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572140-02-7	640	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572140-02-7	650	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572140-02-7	660	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572164-02-7	500	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
572164-02-7	600	1	0.1	0	0	0	0	0	1	0	0	0	0	0	0	0
575151-01-6	573	1	0.44	0	0	0	0	0	0	0	1	0	0	0	0	0
575151-01-6	574	1	0.42	0	0	0	0	0	0	0	1	0	0	0	0	0
575151-01-6	673	1	0.44	0	0	0	0	0	0	0	1	0	0	0	0	0
575151-01-6	674	1	0.42	0	0	0	0	0	0	0	1	0	0	0	0	0
575151-01-9	573	1	0.44	0	0	0	0	0	1	0	0	0	0	0	0	0
575151-01-9	574	1	0.42	0	0	0	0	0	1	0	0	0	0	0	0	0
575151-01-9	673	1	0.44	0	0	0	0	0	1	0	0	0	0	0	0	0

575151-01-9	674	1	0.42	0	0	0	0	0	1	0	0	0	0	0	0	0
712111-01-1	435	1	0.12	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-1	436	1	0.14	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-1	445	1	0.12	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-1	446	1	0.14	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-1	451	1	0.36	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-1	461	1	0.36	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-3	450	1	0.52	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-3	460	1	0.52	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-4	450	1	0.52	0	0	0	0	0	0	0	0	1	0	0	0	0
712111-01-4	460	1	0.52	0	0	0	0	0	0	0	0	1	0	0	0	0
282800-22-1	150	1	1.8	0	0	0	0	0	0	0	0	0	0	0	0	1
292000-01-1	190	1	1.48	0	0	0	1	0	0	0	0	0	0	0	0	0
753200-l1-2	450	1	0.62	0	0	0	0	0	0	0	0	0	0	0	1	0
753200-l1-2	460	1	0.62	0	0	0	0	0	0	0	0	0	0	0	1	0
572032-01-2	500	1	0.2	0	0	0	0	0	0	0	0	0	0	0	0	0

572032-01-2	600	1	0.2	0	0	0	0	0	0	0	0	0	0	0	0	0
572114-01-11	540	1	0.47	0	0	0	0	0	0	0	0	0	0	0	0	0
572114-01-11	640	1	0.47	0	0	0	0	0	0	0	0	0	0	0	0	0
572114-01-12	540	1	0.47	0	0	0	0	0	0	0	0	0	0	0	0	0
572114-01-12	640	1	0.47	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-13	540	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-13	550	1	0.38	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-13	560	1	0.16	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-13	640	1	0.85	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-13	650	1	0.34	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-13	660	1	0.16	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-8	540	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-8	550	1	0.38	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-8	560	1	0.16	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-8	640	1	0.85	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-8	650	1	0.34	0	0	0	0	0	0	0	0	0	0	0	0	0

572115-01-8	660	1	0.16	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-9	540	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-9	550	1	0.38	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-9	560	1	0.16	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-9	640	1	0.85	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-9	650	1	0.34	0	0	0	0	0	0	0	0	0	0	0	0	0
572115-01-9	660	1	0.16	0	0	0	0	0	0	0	0	0	0	0	0	0
725403-01-2	455	1	0.76	0	0	0	0	0	0	0	0	0	0	0	0	0
725403-01-2	465	1	0.76	0	0	0	0	0	0	0	0	0	0	0	0	0
725403-02-2	455	1	1.51	0	0	0	0	0	0	0	0	0	0	0	0	0
725403-02-2	465	1	1.51	0	0	0	0	0	0	0	0	0	0	0	0	0
781112-C1-1	450	1	0.42	0	0	0	0	0	0	0	1	0	0	0	0	0
781112-C1-1	460	1	0.42	0	0	0	0	0	0	0	1	0	0	0	0	0
274451-11-1	310	1	3.52	0	0	0	0	0	0	1	0	0	0	0	0	0
275446-01-1	530	1	0.44	0	0	0	0	0	0	1	0	0	0	0	0	0
275446-01-1	630	1	0.44	0	0	0	0	0	0	1	0	0	0	0	0	0

212300-04-1	200	1	0.25	0	0	1	0	0	0	0	0	0	0	0	0	0
212600-01-1	210	1	0.1	0	0	0	0	0	0	0	1	0	0	0	0	0
212643-01-1	120	1	1.03	0	1	0	0	0	0	0	0	0	0	0	0	0
212643-06-1	120	1	0.19	0	1	0	0	0	0	0	0	0	0	0	0	0
212643-09-1	120	1	0.16	0	0	0	0	0	0	0	1	0	0	0	0	0
212643-10-1	120	1	0.16	0	0	0	0	0	0	0	1	0	0	0	0	0
215000-03-1	190	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0
233300-03-1	200	1	0.25	0	0	1	0	0	0	0	0	0	0	0	0	0
242100-02-1	438	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0
242100-02-1	448	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0
242100-02-1	438	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0
242100-02-1	448	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0
242100-07-1	438	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0
242100-07-1	448	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0
242100-14-1	438	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0
242100-14-1	448	1	0.06	1	0	0	0	0	0	0	0	0	0	0	0	0

278468-01-1	520	1	1.5	0	0	0	0	0	0	0	0	0	0	1	0	0
278468-01-1	620	1	1.5	0	0	0	0	0	0	0	0	0	0	1	0	0

C.4 Utilization Scenario Analysis Result

FH_per_day	A_interval	C_interval	avg_A_downtime	avg_C_downtime	total_cost	avg_cost_per_fh
5	300	5000	0.321050642	0.724444444	36747690.5	2449.846033
5.5	330	5100	0.316992144	0.405555556	39186400.87	2612.426725
6	300	10000	0.17017037	0	38433704.51	2562.246968
6.5	300	5000	0.147120181	0.274074074	38028825.47	2535.255032
7	300	5000	0.177656841	0.274074074	35656670.54	2377.111369
7.5	300	5000	0.20031746	0.274074074	32992390.53	2199.492702
8	300	5000	0.174331066	0.274074074	34161833.54	2277.455569
8.5	300	5000	0.148866213	0.274074074	34000361.39	2266.690759
9	300	10000	0.167666667	0	32747759.4	2183.18396
9.5	300	10000	0.15462963	0	32532609.39	2168.840626
10	300	5000	0.210513983	0.404444444	29566174.33	1971.078289
10.5	300	5000	0.152191988	0.274074074	31044463.59	2069.630906
11	300	5000	0.162834467	0.274074074	30432293.12	2028.819541
11.5	300	10300	0.152162963	0	30117714.79	2007.847653
12	300	8000	0.162592593	0	29550808.11	1970.053874
12.5	300	10300	0.168725926	0	28453779.19	1896.918613
13	300	14900	0.159985185	0	29012354.83	1934.156989
13.5	300	14900	0.152162963	0	28982857.79	1932.190519
14	300	5000	0.157656841	0.256666667	28632643.57	1908.842905
14.5	300	10000	0.152162963	0.239259259	28391424.53	1892.761635
15	300	5000	0.172668178	0.256666667	27320870.58	1821.391372